

企业级数据分析 Agent 系统 — 技术 RFC

版本: v1.1 | 日期: 2026-07-06 | 作者: 罗晓军团队

本文档为技术架构与实现方案，面向开发团队。

目录

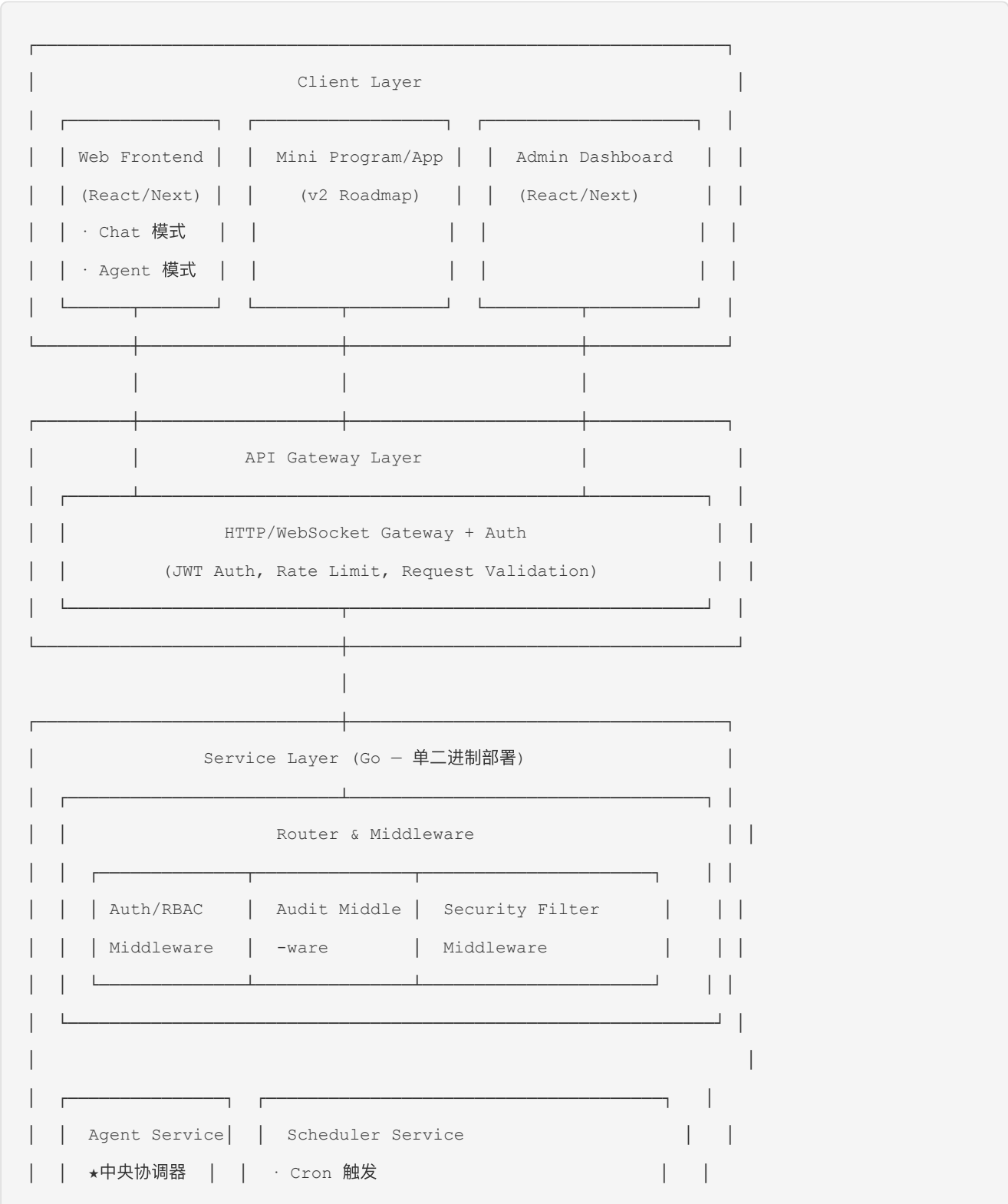
- 1. 架构总览
- 2. 技术选型与依赖
- 3. 业务时序图
- 4. 系统分层架构
- 5. 核心模块设计
- 6. Agent 系统设计
- 7. 数据模型设计
- 8. API 设计
- 9. Mock LLM Service (测试用)
- 10. Debug 日志规范
- 11. 安全架构
- 12. 会话与工作区管理
- 13. Artifact 管理 (基于 SeaweedFS)
- 14. 知识库设计
- 15. 审计系统设计
- 16. 异步任务与消息队列
- 17. 部署架构
- 18. 风险与缓解方案
- 19. 分阶段开发任务分解
- 20. AI 辅助开发流程规范
- 21. 团队分工建议
- 22. 附录

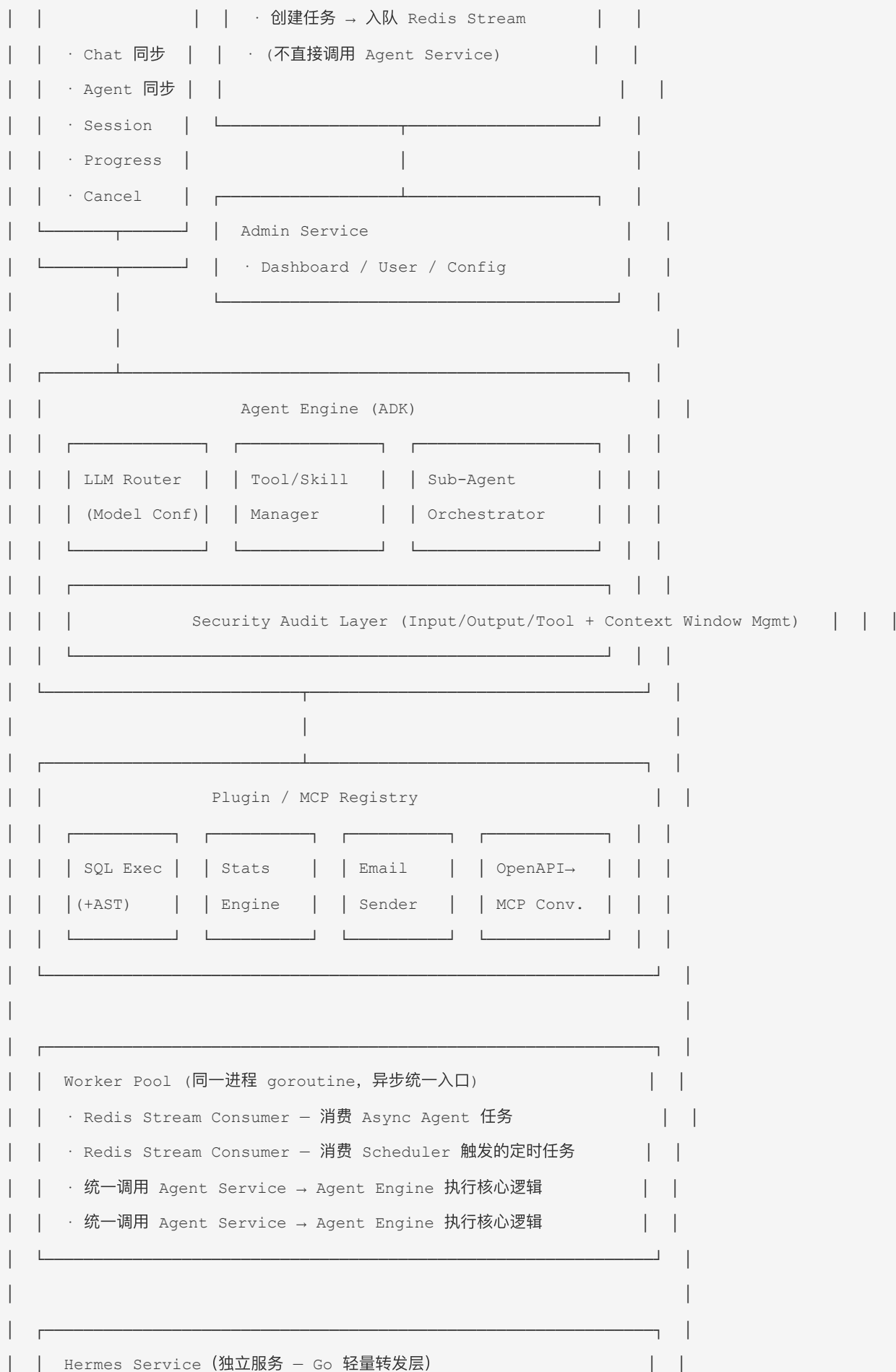
1. 架构总览

1.1 系统全景图

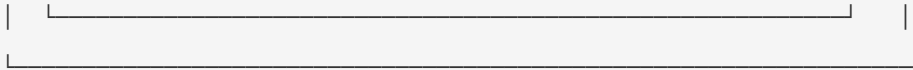
系统架构全景图

系统采用前后端分离的 B/S 架构，后端使用 Go 语言，分为以下核心层：









1.2 核心数据流

数据流对比图

核心架构约定: Agent Service 是系统的中央协调器，统一管理所有用户请求的 Session、模型调用和工具编排。前端用户无论通过 Chat 模式还是 Agent 模式，最终都调用 Agent Service。

Hermes Service 例外: 探索模式下，请求直接转发到 Hermes，不经过 Agent Service。**IM 模块集成在主二进制内**，通过 `internal/service/im/` 封装飞书 Webhook 和消息路由，Chat 消息经 Agent Service 处理，复用 Chat 模式的完整能力。

Chat 模式数据流 (同步即时交互):

```
User Input → API Gateway → Auth/RBAC Check
  → Security Filter (Input Sanitization)
  → Agent Service (Session Manager + ADK Agent Engine)
  → [LLM Router → Model Inference]
  → [Tool Calls via MCP/Skill Registry]
  → [Knowledge Base Search]
  → Agent Response → Security Filter (Output Sanitization)
  → Audit Log → SSE Stream → User
```

- Chat 模式直接调用 Agent Service，走同步路径
- Agent Service 管理 Session 生命周期（创建、上下文维护、超时清理）
- 响应通过 SSE (Server-Sent Events) 流式返回

Agent 模式数据流 (批量任务，支持同步/异步):

└ 同步模式 (立即执行) :

```
| User Task → API Gateway → Auth/RBAC Check
|   → Security Filter → Agent Service
|   → Agent Engine (ADK) → Plan & Execute Tools
|   → [Sub-Agent via A2A] → [Script Execution via Worker]
|   → Results → Store to MongoDB + SeaweedFS → Response
|
```

└ 异步模式 (队列消费) :

```
User Task → API Gateway → Auth/RBAC Check
```

```

→ Security Filter → Agent Service
→ Enqueue to Redis Stream (立即返回 task_id)
→ Worker consumes from Queue
→ Worker calls Agent Service (复用 Agent Engine)
→ Execute → Results → Store
→ Notification (Email/WS) → User

```

- Agent 模式统一通过 Agent Service 入口
- 同步模式: Agent Service 直接执行, 等待结果返回
- 异步模式: Agent Service 入队后立即返回 task_id, Worker 从 Redis Stream 消费任务后回调 Agent Service 执行
- **Worker 职责:** 从队列消费异步任务后回调 Agent Service 执行核心逻辑。Worker 是纯 Go goroutine, 不涉及 Python/R 等外部运行时

定时任务数据流:

```

Scheduler (Cron 触发时刻) → 创建 ScheduledTask (MongoDB)
→ XADD agent:tasks {type: scheduled}
→ Worker 消费 → 识别为 scheduled → 创建 AgentTask (MongoDB)
→ XADD agent:tasks {type: agent} ← 转换后重入同一条队列
→ Worker 再次消费 → 识别为 agent → 回调 Agent Service
→ Agent Engine Execute → Results → Persist + Notify

```

设计意图: ScheduledTask 不直接执行。Worker 消费后将其转换为标准 AgentTask 并重入同一 `agent:tasks` 队列, 复用 Agent 异步管道的全部执行逻辑和基础设施。Scheduler 和 Agent 异步模式共享 Worker Pool 和 `agent:tasks` 队列。

关键设计要点: – Agent Service 是唯一入口: Chat 和 Agent 两种模式共享同一 Agent Engine 实例 – Session 统一由 Agent Service 管理, Chat 和 Agent 任务的 Session 模型一致 – **Worker 是所有异步路径的统一入口:** 定时任务和异步 Agent 任务都经 Redis Stream → Worker → Agent Service, Worker 不与前端直接交互 – Agent Service 仅处理同步请求 (Chat + Agent 同步模式), 异步路径全部由 Worker 代为调用

2. 技术选型与依赖

2.1 核心技术栈

组件	选型	版本	选型理由
开发语言	Go	1.22+	高性能、并发原生、部署简单
Agent 框架	google.golang.org/adk	latest	Google 官方 Agent 开发套件，生态完善
业务数据库	MongoDB	7.0+	文档模型灵活，统一存储所有业务实体（用户、权限、知识库元数据、Artifact 元数据、分析结果、配置等）
向量数据库	Milvus	2.4+	高性能向量检索，CNCF 项目
记忆系统	Mem0	latest	开源记忆层，支持多粒度记忆管理
会话工作区 & Artifact	SeaweedFS	RELEASE.2024+	S3 兼容对象存储，开源；统一承载 Session 临时文件与 Artifact 持久文件
缓存 & 消息队列	Redis	7.2+	高性能缓存 + Stream 做轻量消息队列
敏感数据管理	HashiCorp Vault	1.18+	集中管理 JWT Secret、DB 密码、API Key、SMTP 密码等敏感配置，支持动态轮转和审计
前端框架	React / Next.js	18 / 14	生态丰富，SSR 支持好
WebSocket	Gorilla WebSocket	1.5+	Go 生态标准 WebSocket 库

2.2 关键开源依赖

组件	开源项目	用途
OpenAPI → MCP 转换	<code>openapi-mcp-server</code> (或自研)	将 OpenAPI 定义转换为 MCP Tool
安全审计	<code>go-audit</code> / <code>audit-go</code>	标准化审计日志库
A2A 协议	ADK 内置 <code>a2a</code> 包	Agent 间通信协议
JWT	<code>golang-jwt/jwt</code>	用户认证 Token
Cron	<code>robfig/cron</code>	定时任务调度
PDF 解析	<code>ledongthuc/pdf</code>	知识库文档解析
Excel 解析	<code>xuri/excelize</code>	表格文档解析
邮件发送	<code>gomail</code> / <code>go-mail</code>	SMTP 邮件
统计计算	<code>gonum</code>	Go 原生统计和数值计算库
飞书 SDK	<code>go-lark/lark</code>	Go 飞书开放平台 SDK，支持消息收发和事件订阅
Token 计数	<code>pkoukk/tiktoken-go</code>	OpenAI 兼容 token 计数 (MIT)，用于上下文窗口压缩阈值判断
结构化日志	<code>uber-go/zap</code>	高性能结构化日志，支持 Debug/Info/Warn/Error 级别
Mock LLM 存储	内嵌 Redis List (无额外依赖)	Mock Service 的 per-key 队列存储 (复用现有 Redis 实例)

2.3 开发工具链

- 版本控制: Git (内部 GitLab/GitHub)
- CI/CD: GitHub Actions / Jenkins
- 容器化: Docker + Docker Compose (开发/测试)
- 容器编排: Kubernetes (生产)
- API 文档: Swagger/OpenAPI 3.0 自动生成
- 代码质量: golangci-lint, SonarQube
- 测试: Go testing + testify + 端到端 Playwright

3. 业务时序图

系统四种核心交互模式的详细时序流程：

3.1 Chat 模式 (同步即时交互)



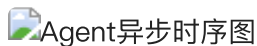
流程: User 发送自然语言 → API Gateway JWT+Rate Limit → Security Input Filter → Agent Service 创建 Session → ADK Agent Engine 推理 → Agent 自主调用 Skills (SQL/Stats/KB) → Security Output Filter + Audit → SSE 流式返回

3.2 Agent 同步模式



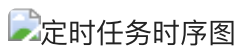
流程: User POST `/tasks {mode: sync}` → Gateway Auth → AgentService.CreateAndExecuteSync() → Agent Engine + Skills → 同步返回 Response。请求-响应模式，适用于中等复杂度的实时分析任务。

3.3 Agent 异步模式



流程: User POST `/tasks {mode: async}` → AgentService 保存任务(MongoDB) + XADD agent:tasks → 立即返回 task_id。Worker 从队列消费 → 回调 Agent Service → Agent Engine + Skills → 结果落库 → Email+WS 通知。

3.4 定时任务模式 (Scheduled → Convert → Re-enqueue)



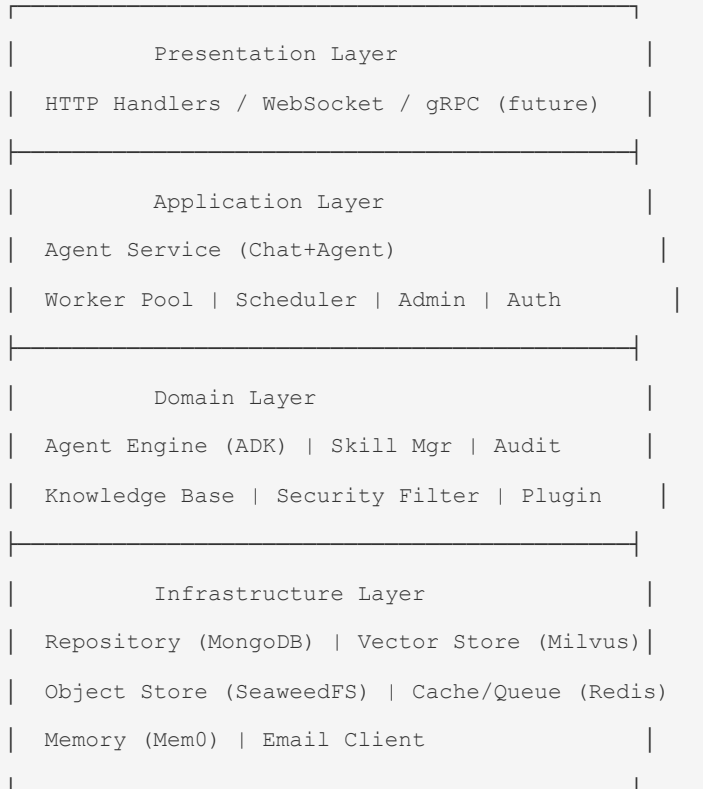
流程: Cron 触发 → Scheduler 创建 ScheduledTask(MongoDB) + XADD agent:tasks {type:scheduled} → Worker Dequeue → **转换为 AgentTask** → XADD agent:tasks {type:agent} (重入队) → Worker 再次 Dequeue → 回调 Agent Service → 复用 Agent 异步管道完成执行和通知。

设计意图: ScheduledTask 不直接执行。Worker 将其转换为标准 AgentTask 并重入同一 `agent:tasks` 队列，实现 Scheduler 和 Agent 异步模式的管道统一。

4. 系统分层架构

部署形态: 整个 Service Layer 编译为单个 Go 二进制文件部署，内部按逻辑分层（Handler → Service → Repository），不做物理微服务拆分。Worker 和 Scheduler 在同一进程中以独立 goroutine 运行，通过 Redis Stream 协调。

3.1 分层设计



请求处理流程 (Handler → Service → Repository):

所有 API 服务严格遵循三层架构，每层职责明确、不可越界：

Request → Handler (入口层) → Service (逻辑层) → Repository (存储层)

- | | | |
|-----------|----------|-----------|
| | | |
| · 参数解析与校验 | · 业务逻辑编排 | · 数据持久化 |
| · 权限前置检查 | · 跨模块协调 | · 缓存操作 |
| · 响应格式化 | · 事务管理 | · 外部API调用 |

入口层 (Handler) 必须完成的校验:

校验项	说明	示例
参数存在性	必填字段非空检查	<code>title</code> 不能为空
参数类型	字段类型匹配	<code>retry_count</code> 必须为整数
参数范围	数值/枚举范围检查	<code>report_type</code> 必须在预置枚举中
参数长度	字符串长度限制	<code>content</code> 不超过 100KB
参数格式	格式合规检查	<code>email</code> 符合邮箱正则
权限校验	当前用户是否有操作权限	RBAC <code>agent:create</code> 权限

Handler 层校验失败直接返回 4xx，不进入 Service 层。校验通过后的参数以 Domain Model 传递到 Service 层。

3.2 Go 项目目录结构

```
.
├── cmd/
│   ├── server/           # 主服务入口 - 单二进制部署
│   │   └── main.go       # 启动 API Server + Worker Pool + Scheduler (goroutines)
│   └── cli/              # 运维 CLI 工具 (migration/seed/admin, 非 Worker)
├── internal/
│   ├── api/              # HTTP Handler 层
│   │   ├── handler/     # 请求处理器 (参数校验 + 响应格式化)
│   │   ├── middleware/  # 中间件
│   │   └── router/       # 路由定义
│   ├── logic/            # ★ Logic 层 (Skill 与 Service 的共用业务逻辑)
│   │   ├── artifact.go  # Artifact 创建/删除 (save_artifact & save_report 共用)
│   │   ├── report.go    # Report CRUD (save_report 与 admin handler 共用)
│   │   └── session.go   # Session 清理 (agent service & cleanup goroutine 共用)
│   ├── service/          # 业务逻辑层 (Agent Service 内部模块)
│   │   ├── chat/        # Chat 模式业务逻辑 (SSE 流式、提示词)
│   │   ├── agent/       # Agent 批量任务模式业务逻辑
│   │   ├── scheduler/
│   │   ├── auth/
│   │   ├── admin/
│   │   └── knowledge/
│   └── domain/           # 领域模型
```

```

|   |   └─ model/           # 数据模型
|   |   └─ agent/           # Agent 引擎
|   |   └─ skill/           # Skill/MCP 注册中心
|   |   └─ audit/           # 审计系统
|   |   └─ security/        # 安全审查
|   └─ worker/              # Agent Worker Pool (goroutine, 非独立进程)
|   |   └─ pool.go          # 1. Worker 池管理 → Redis Stream 消费
|   |   └─ consumer.go      # 2. 异步 Agent 任务消费
|   |   └─ executor.go      # 3. 消费回调 Agent Service (纯 Go)
|   └─ infra/              # 基础设施层
|   |   └─ mongo/           # MongoDB Repository
|   |   └─ milvus/          # Milvus Client
|   |   └─ seaweedfs/       # SeaweedFS Client (S3 compatible)
|   |   └─ redis/           # Redis Client
|   |   └─ mem0/            # Mem0 Client
|   |   └─ email/           # 邮件客户端
|   └─ config/             # 配置管理
└─ pkg/                   # 可复用公共库
|   └─ errcode/            # 错误码
|   └─ jwt/                # JWT 工具
|   └─ logger/             # 日志
└─ skills/                # Skill 定义与实现
|   └─ sql-executor/
|   └─ stats-engine/
|   └─ email-sender/
|   └─ openapi-converter/
└─ configs/               # 配置文件
└─ docs/                  # API 文档
└─ scripts/               # 部署/运维脚本
└─ docker-compose.yml
└─ Makefile

```

4.3 Logic 层 — Skill/Controller/Service 共用逻辑抽取

问题: Skill (Agent 侧的业务入口)、Controller (HTTP API Handler)、Service (内部业务逻辑) 三者之间存在重复逻辑。例如： - `save_artifact` Skill 和 `save_report` Skill 都需要「上传文件到 SeaweedFS + 写入 MongoDB」 - Admin API 的「下载 Report」和 Agent Skill 的「下载 Report」走同一逻辑 - Workspace 清理在 Session 过期回调和 Admin 手动清理中重复

方案: 在 `internal/logic/` 中抽取纯业务函数，供上述三层共同调用：



Logic 层设计原则:

1. **无状态** — Logic 结构体不持有请求级状态，可安全并发调用
2. **接收 SkillContext 或 context.Context** — 不绑定 HTTP 或 Skill 特化上下文
3. **返回 Domain Model** — 不返回 HTTP 响应格式，由上层自行组装
4. **幂等性内置** — MongoDB upsert 在 Logic 层实施，上层无需关心重复调用

```

// internal/logic/artifact.go

type ArtifactLogic struct {
    seaweedfs *seaweedfs.Client
    mongo      *mongo.Collection
}

// CreateArtifact 创建 Artifact (幂等)
// 被 save_artifact Skill 和 save_report Skill 共同调用
func (l *ArtifactLogic) CreateArtifact(ctx context.Context, sc SkillContext, input
CreateArtifactInput) (*ArtifactRecord, error) {
    // 上传 SeaweedFS → MongoDB upsert → 返回记录
}

// DeleteArtifact 删除 Artifact (幂等)
// 被 Artifact GC 和 Admin API 共同调用
func (l *ArtifactLogic) DeleteArtifact(ctx context.Context, artifactID string) error {

```

```
// SeaweedFS 删除（不关心文件是否存在）→ MongoDB 标记删除 → 返回 nil（已删除也视为成功）
}
```

4.4 幂等性设计规范

系统中所有资源创建和删除操作必须保证幂等性，支持安全重试。

4.4.1 创建幂等

原则: 相同参数多次调用 `CreateXxx` 不会产生多条记录，不会报错。

实现方式 — MongoDB `$setOnInsert` upsert:

```
// internal/logic/idempotent.go

// IdempotentCreate 通用幂等创建 Helper
// uniqueKey 为唯一索引字段组合（如 {task_id, report_type}）
func IdempotentCreate(ctx context.Context, coll *mongo.Collection, uniqueKey bson.M, doc
any) (created bool, err error) {
    filter := bson.M{}
    for k, v := range uniqueKey {
        filter[k] = v
    }

    // 设置标准时间字段（如果 doc 有这些字段则用 $setOnInsert，已存在则不更新）
    update := bson.M{"$setOnInsert": doc}
    opts := options.Update().SetUpsert(true)

    result, err := coll.UpdateOne(ctx, filter, update, opts)
    if err != nil {
        return false, fmt.Errorf("idempotent_create: %w", err)
    }
    return result.UpsertedCount > 0, nil
}
```

事务回滚模式（跨资源操作，如 report 先创建 artifact 再创建 report）:

```
// Multi-step creation with rollback
func (l *ReportLogic) CreateReport(ctx context.Context, sc SkillContext, input
CreateReportInput) (*Report, error) {
```

```

// 1. 创建 Artifact

artifact, err := l.artifactLogic.CreateArtifact(ctx, sc, CreateArtifactInput{...})
if err != nil {
    return nil, err
}

// 2. 创建 Report

report, created, err := l.createReportRecord(ctx, sc, input, artifact.ID)
if err != nil {
    // ★ 回滚: 删除已创建的 Artifact (best-effort)
    l.artifactLogic.DeleteArtifact(ctx, artifact.ID)
    return nil, fmt.Errorf("create_report: %w", err)
}

// 3. 如果 Report 已存在 (幂等返回), 删除多余 Artifact
if !created {
    l.artifactLogic.DeleteArtifact(ctx, artifact.ID)
}

return report, nil
}

```

4.4.2 删除幂等

原则: 删除一个不存在或已被删除的资源直接返回成功 (HTTP 200 / 无错误), **绝不返回 404**。

```

// 应用层 Delete 模式

func (l *ArtifactLogic) DeleteArtifact(ctx context.Context, artifactID string) error {
    // 1. 标记删除 (MongoDB)
    _, err := l.mongo.UpdateOne(ctx,
        bson.M{"_id": artifactID},
        bson.M{"$set": bson.M{"deleted_at": time.Now()}}),
    )
    if err != nil {
        return fmt.Errorf("delete_artifact: %w", err)
    }
    // ★ MatchedCount = 0 (记录不存在) → 不报错, 直接返回 nil

    // 2. 删除 SeaweedFS 文件 (best-effort, 文件不存在也不报错)

```

```
l.seaweedfs.Delete(ctx, artifactID) // 内部吞掉 "not found" 错误

return nil

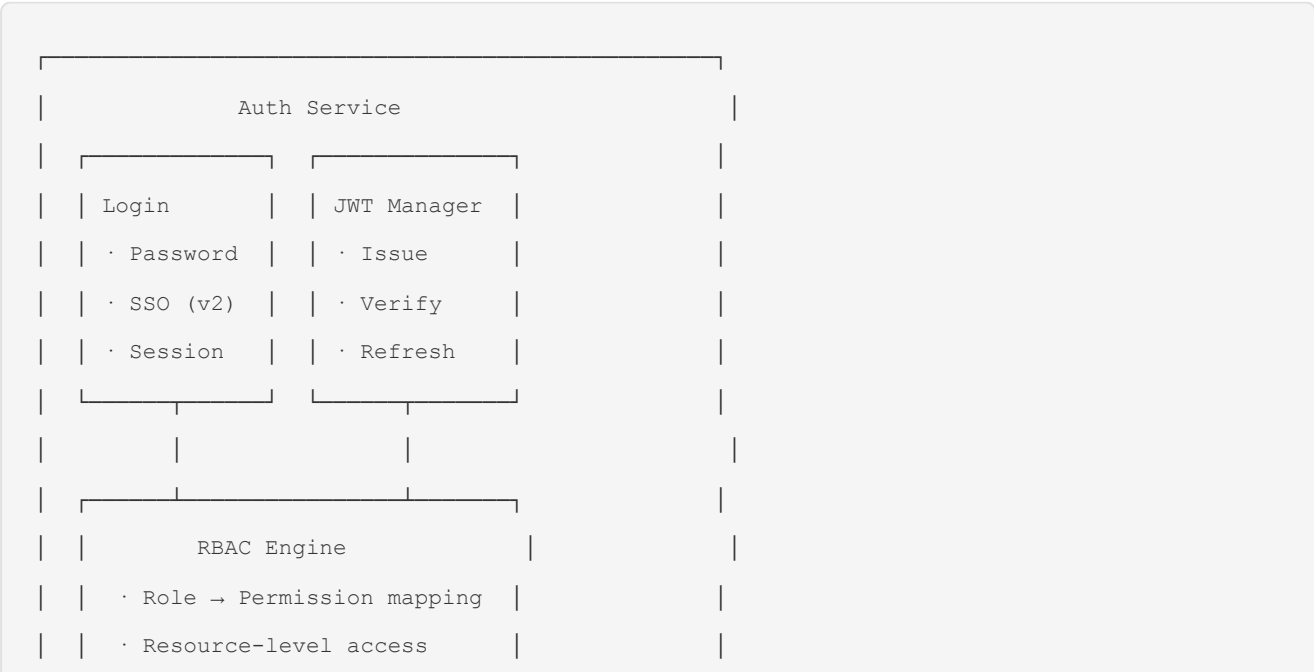
}
```

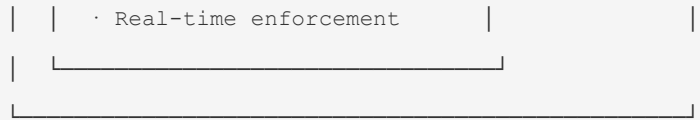
幂等删除规则汇总:

操作	资源不存在时行为
DELETE /api/v1/artifacts/:id	返回 {"deleted": true}，不返回 404
DELETE /api/v1/reports/:id	同上
DELETE /api/v1/schedules/:id	同上
DELETE /api/v1/sessions/:id	同上
DELETE /api/v1/knowledge/docs/:id	同上
Workspace 清理 (CleanupExpiredSessions)	目录不存在跳过，不报错
SeaweedFS 文件删除	文件不存在吞掉错误

5. 核心模块设计

4.1 认证与授权模块





RBAC 数据模型 (MongoDB):

```

// roles collection
{
  "_id": "role_analyst",
  "name": "Analyst",
  "description": "数据分析师",
  "permissions": [
    "chat:read", "chat:write",
    "agent:create", "agent:cancel",
    "knowledge:read", "knowledge:search",
    "data:read", "data:analyze",
    "schedule:create", "schedule:manage"
  ],
  "created_at": "2026-07-01T00:00:00Z",
  "updated_at": "2026-07-01T00:00:00Z"
}

// users collection
{
  "_id": "user_001",
  "username": "zhangsan",
  "password_hash": "$2a$10$...",
  "display_name": "张三",
  "email": "zhangsan@company.com",
  "roles": ["role_analyst", "role_viewer"],
  "status": "active", // active | disabled
  "last_login_at": "2026-07-01T08:30:00Z",
  "created_at": "2026-07-01T00:00:00Z"
}

```

权限校验流程:

```

Request → JWT Verify → Extract User + Roles
        → RBAC Engine.Check(user, resource, action)

```

```
→ [Pass] → Handler
→ [Deny] → 403 + Audit Log
```

4.2 Agent 服务 — Chat 模式

说明: Chat 模式是 Agent Service 的同步交互模式，**不是独立服务**。前端通过 `/api/chat` 端点发送请求 → Gateway → Auth/RBAC → Agent Service (同一组件) → ADK Agent Engine → SSE 流式返回。与 §4.3 的 Agent 批量任务模式共享同一个 Agent Service 实例和 Session Manager。

功能: – 即时对话式查询和分析 – 快捷提示词管理 – 多轮对话上下文维护 (Session 统一管理) – SSE 流式返回响应 – 结果缓存 (Redis, 相同查询 5 分钟内直接返回)

请求流程:

```
POST /api/v1/chat/message
→ Auth Middleware (JWT + RBAC)
→ Security Filter (Input Sanitization)
→ AgentService.HandleChatMessage() // ★ 同一组件
→ SessionManager.GetOrCreate()
→ ADK Agent Engine.Run() // LLM 推理 + Tool 调用
→ Security Filter (Output PII)
→ Audit Log
→ SSE Stream: data: {"type":"content","text":"..."}
```

Chat Session 模型:

```
{
  "_id": "session_chat_001",
  "user_id": "user_001",
  "type": "chat",
  "messages": [
    {
      "role": "user",
      "content": "昨天华东区销售额?",
      "timestamp": "2026-07-01T09:00:00Z"
    },
    {
      "role": "assistant",
      "content": "...",

```

```

        "tool_calls": [...],
        "timestamp": "2026-07-01T09:00:02Z"
    }
],
"context": {
    "data_sources": ["sales_db"],
    "preferences": {}
},
"workspace_id": "ws_chat_001",
"created_at": "2026-07-01T09:00:00Z",
"last_active_at": "2026-07-01T09:05:00Z",
"expires_at": "2026-07-01T09:35:00Z" // 30 min inactivity
}

```

快捷提示词模型:

```

{
    "_id": "prompt_001",
    "category": "sales",
    "text": "本月销售额TOP10产品",
    "roles": ["viewer", "analyst"],
    "sort_order": 1,
    "is_system": true,
    "created_by": "system"
}

```

4.2.1 上下文窗口管理

LLM 上下文窗口有限（通常 128K tokens），多轮对话 + 知识库检索 + 长报告生成容易超出限制。采用 **tiktoken-go + LLM 摘要** 三层策略，无需引入额外 Python 服务。

1. 对话摘要压缩（tiktoken-go 计数 + 轻量模型摘要）

压缩阈值由当前使用的模型配置 `max_context_tokens` 动态决定，而非硬编码：

```

import "github.com/pkoukk/tiktoken-go"

type ContextCompressor struct {
    tke *tiktoken.Tiktoken
    // 摘要模型：轻量低成本（GPT-4o-mini / Claude 3.5 Haiku）
    SummaryModel string
}

```

```

}

// Compress 根据当前模型配置决定是否需要压缩
// modelCfg 从 MongoDB model_configs 查询, 含 max_context_tokens
func (c *ContextCompressor) Compress(messages []Message, modelCfg *ModelConfig) ([]Message,
error) {
    // 阈值 = 模型上下文窗口 × 50% (预留给 system prompt、工具结果、LLM 输出)
    threshold := modelCfg.MaxContextTokens / 2

    // tiktoken-go 精确计数
    tokens := 0
    for _, msg := range messages {
        tokens += len(c.tke.Encode(msg.Content, nil, nil))
    }
    if tokens <= threshold {
        return messages, nil
    }

    // 保留最近 4 轮对话原文 (约 2K-4K tokens)
    recent := messages[len(messages)-4:]
    older := messages[:len(messages)-4]

    // 轻量模型生成摘要 (~100 tokens, 成本约 ¥0.001/次)
    summary := c.llmRouter.Chat(ctx, &ChatRequest{
        Model:      c.SummaryModel,
        Messages: append(older, Message{Role: "user", Content: "请用200字以内总结以上对话的关键信息和结论"}),
    })

    return append([]Message{{Role: "system", Content: "历史对话摘要:\n" + summary}},
recent...), nil
}

```

阈值计算: $\text{threshold} = \text{max_context_tokens} \times 50\%$, 例如 GPT-4o (128K) → 64K, GPT-4o-mini (128K) → 64K, Claude 3.5 Sonnet (200K) → 100K。剩余 50% 预留给 system prompt、知识库检索结果、工具输出和 LLM 回复。

2. 知识库结果截断 – 向量搜索结果按相似度排序，取 top-5 条 – 每条 chunk 截断至 800 tokens（用 tiktoken-go 精确计算） – $5 \times 800 = 4000$ tokens 上限

3. 长报告分段生成合并

```
// 报告预计长度超过阈值时，分段生成后合并
func (c *ReportBuilder) BuildLongReport(prompt string, estimatedTokens int) (string, error)
{
    if estimatedTokens <= c.MaxOutputTokens {
        return c.llm.Generate(prompt) // 单次生成
    }
    // 分段生成
    sections := c.splitPromptBySections(prompt)
    var parts []string
    for _, section := range sections {
        part := c.llm.Generate(section)
        parts = append(parts, part)
    }
    return strings.Join(parts, "\n\n"), nil
}
```

4.3 Agent 服务 — Agent 模式（批量任务）

定位: 与 §4.2 Chat 模式共享同一个 `AgentService` 组件。区别在于：Chat 是同步即时交互，Agent 模式支持同步/异步两种执行路径。Agent Service 是系统唯一的中央协调器。

功能: – 统一入口：Chat 实时对话和 Agent 批量任务均通过 Agent Service – Session 生命周期管理（创建、上下文维护、超时清理） – Chat 模式：接收用户消息，调用 ADK Agent Engine 进行同步推理，SSE 流式返回 – Agent 模式 — 同步：直接执行 Agent Engine，等待结果后返回 – Agent 模式 — 异步：将任务入队 Redis Stream，立即返回 task_id；由 Worker 消费后回调 Agent Service 执行 – 任务状态跟踪与进度上报 – 任务取消（Goroutine context cancel） – 子 Agent 编排（A2A） – 结果持久化和通知

Agent 任务模型:

```
{
  "_id": "task_001",
  "user_id": "user_001",
  "type": "batch_analysis", // batch_analysis | scheduled | sub_agent
  "status": "running", // pending | queued | running | completed | failed | cancelled
  "input": {
```

```

    "query": "对过去3年销售数据做回归分析和预测",
    "data_sources": ["sales_db"],
    "analysis_types": ["regression", "time_series"],
    "parameters": {}
  },
  "progress": {
    "percentage": 45,
    "current_step": "执行回归分析中...",
    "steps": [
      {"name": "数据提取", "status": "completed", "duration_ms": 5200},
      {"name": "数据清洗", "status": "completed", "duration_ms": 3100},
      {"name": "回归分析", "status": "running", "duration_ms": null},
      {"name": "结果解读", "status": "pending", "duration_ms": null}
    ]
  },
  "result": null,
  "error": null,
  "sub_agents": ["task_002", "task_003"],
  "parent_task_id": null,
  "workspace_id": "ws_agent_001",
  "created_at": "2026-07-01T09:00:00Z",
  "started_at": "2026-07-01T09:00:05Z",
  "completed_at": null,
  "cancellable": true
}

```

任务取消机制:

```

// 基于 context 的优雅取消
type AgentTask struct {
    ctx    context.Context
    cancel context.CancelFunc

    // ...
}

func (s *AgentService) CancelTask(taskID string) error {
    task := s.getTask(taskID)

    if task.Status != "running" {
        return ErrTaskNotRunning
    }
}

```

```

    }

    task.cancel() // 触发 context cancellation
    // Agent Engine 中所有 goroutine 检查 ctx.Done()

    task.Status = "cancelled"

    s.saveTask(task)

    return nil
}

```

4.4 定时任务服务 (Scheduler Service)

调度方式: Scheduler 不直接调用 Agent Service。Cron 触发时，创建任务记录 (MongoDB) 并写入 Redis Stream，由 Worker 统一消费后回调 Agent Service 执行。这与异步 Agent 模式走同一条路径，保持架构一致性。

调度器设计:

```

type SchedulerService struct {
    cron          *cron.Cron
    repo          *SchedulerRepo
    redisStream   *RedisStreamClient // 写入 Redis Stream, 不直接调 AgentSvc
}

type ScheduledTask struct {
    ID          string    `bson:"_id"`
    UserID      string    `bson:"user_id"`
    Name        string    `bson:"name"`
    CronExpr    string    `bson:"cron_expr"` // "0 8 * * *"
    AgentInput  AgentInput `bson:"agent_input"`
    Status      string    `bson:"status"` // active | paused | deleted
    LastRunAt   *time.Time `bson:"last_run_at"`
    LastStatus  string    `bson:"last_status"`
    RetryCount  int       `bson:"retry_count"` // 最大重试次数
    ValidFrom   *time.Time `bson:"valid_from"`
    ValidUntil  *time.Time `bson:"valid_until"`
    CreatedAt   time.Time `bson:"created_at"`
}

// onCronTrigger: Cron 到达触发时刻

```

```

func (s *SchedulerService) onCronTrigger(task ScheduledTask) {

    // 1. 创建任务执行记录 (MongoDB)

    execID := s.repo.CreateExecution(task)

    // 2. 写入 Redis Stream (与异步 Agent 任务共用队列)

    msg := map[string]interface{}{

        "task_id":    execID,

        "task_type":  "scheduled",

        "user_id":    task.UserID,

        "agent_input": task.AgentInput,

    }

    s.redisStream.XAdd(ctx, "agent:tasks", msg)

    // Worker 消费后:

    // 1. Dequeue → 识别 type=scheduled

    // 2. 创建 AgentTask (MongoDB) → XADD agent:tasks {type:agent}

    // 3. Worker 再次 Dequeue → 识别 type=agent

    // 4. 调用 AgentService.ExecuteTask() → 结果落库 + 通知

}

```

Worker 分流处理逻辑:

```

// Worker processTask - 根据 task_type 分流

func (w *AgentWorker) processTask(ctx context.Context, taskID string) {

    msg := w.loadMessage(taskID)

    switch msg.TaskType {

    case "scheduled":

        // == ScheduledTask: 转换 → 重入队 ==

        sched := w.loadScheduledTask(msg.ScheduledTaskID)

        // 创建 AgentTask (复用用户手动创建任务的同构模型)

        agentTask := &AgentTask{

            UserID:    sched.UserID,

            Query:     sched.AgentInput.Query,

            Mode:       "async",

            Source:     "scheduled",           // 标记来源

            SourceID:  sched.ID,

            CreatedAt: time.Now(),

        }
    }
}

```



```

    }

    w.mongo.CreateAgentTask(agentTask)

    // 重入同一条队列 (type=agent)
    w.redis.XAdd("agent:tasks", map[string]string{
        "task_type": "agent",
        "agent_task_id": agentTask.ID,
    })

case "agent":
    // === AgentTask: 标准异步执行 ===
    task := w.mongo.GetAgentTask(msg.AgentTaskID)

    taskCtx, cancel := context.WithCancel(ctx)
    defer cancel()

    w.updateStatus(task.ID, "running")
    result, err := w.agentSvc.ExecuteTask(taskCtx, task)
    if err != nil {
        w.updateStatus(task.ID, "failed")
        w.notifier.Notify(task.UserID, task.ID, err)
        return
    }
    w.saveResult(task.ID, result)
    w.updateStatus(task.ID, "completed")
    w.notifier.Notify(task.UserID, task.ID, nil)
}
}

```

4.5 管理后台服务 (Admin Service)

功能模块:

```

Admin Service
├── Dashboard Handler
│   ├── Agent Stats (调用量/成功率/耗时分布)
│   ├── Business Stats (分析指标聚合)
│   └── System Health (CPU/Mem/QPS)
└── User Management

```

```

|   ├── CRUD Users
|   ├── Role Assignment
|   └── Account Status (enable/disable)
└── Permission Management
    ├── Role CRUD
    └── Permission Mapping
└── Model Configuration
    ├── Model Selection
    ├── Context Length Limit
    └── Temperature/Top-P Settings
└── Task Management
    ├── Task List (running/history)
    ├── Cancel Task
    └── Retry Failed Task
└── Knowledge Base Management
    ├── Document Upload/Delete
    ├── Index Status Tracking
    └── Permission Settings
└── Audit Log Viewer
    ├── Filter by Time/User/Action
    └── Export Logs

```

4.6 报告格式校验 Skill (`save_analysis_report`)

功能: Agent 通过调用 `save_analysis_report` Skill 保存分析报告。Skill 在保存前自动校验报告格式，校验通过才执行保存；不通过时返回缺失项反馈，由 Agent 自行修正后重新调用。

设计原则: 报告校验不属于审计/安全层，而是 Agent 工具调用链中的一个环节。Agent 自主决定何时保存、是否重试、何时放弃。

调用流程:

```

Agent generates report content
|
▼
Agent calls save_analysis_report(report_type, content)
|
▼

```



```

        return "校验并保存分析报告。先创建 Artifact 上传到 SeaweedFS, 再创建 Report 记录引用 Artifact。
        校验不通过时返回缺失项供 Agent 修正。"
    }

func (s *SaveReportSkill) Parameters() json.RawMessage {
    return toJSONSchema(map[string]any{
        "type": "object",
        "properties": map[string]any{
            "report_type": map[string]any{
                "type": "string",
                "enum": []string{
                    "financial_analysis", "sales_analysis",
                    "general_analysis", "weekly_report", "user_behavior",
                },
                "description": "报告类型",
            },
            "title":      map[string]any{"type": "string", "description": "报告标题"},
            "content":     map[string]any{"type": "string", "description": "报告全文 (Markdown
            格式)"},
            "task_id":     map[string]any{"type": "string", "description": "关联的Agent任务
            ID"},
            "retry_count": map[string]any{"type": "integer", "description": "当前重试次数 (0-
            based)"},
        },
        "required": []string{"report_type", "title", "content"},
    })
}

func (s *SaveReportSkill) Execute(ctx context.Context, sc SkillContext, params
map[string]any) (any, error) {
    reportType := params["report_type"].(string)
    content     := params["content"].(string)
    title       := params["title"].(string)
    retryCount  := getInt(params, "retry_count")

    // Step 1: 格式校验
    result := s.validator.Validate(reportType, content)
    s.audit.LogValidation(ctx, AuditValidationLog{
        TaskID: sc.TaskID, ReportType: reportType,

```

```

        Passed: result.Passed, RetryCount: retryCount,
        Failures: result.Failures, Action: "validate",
    })
    if !result.Passed {
        maxRetries := s.validator.MaxRetries(reportType)
        return map[string]any{
            "saved": false, "retry_count": retryCount + 1,
            "max_retries": maxRetries, "feedback": result.FeedbackMsg, "failures":
result.Failures,
            }, nil
        }

// Step 2: 创建 Artifact (共用 ArtifactLogic, 幂等 upsert)
artifact, err := s.artifactLogic.CreateArtifact(ctx, sc, logic.CreateArtifactInput{
    Name:      fmt.Sprintf("%s.md", title),
    Content:   []byte(content),
    MimeType:  "text/markdown",
})
if err != nil {
    return nil, fmt.Errorf("save_report: create artifact failed: %w", err)
}

// Step 3: 创建 Report - 幂等写入 (task_id + report_type 唯一)
report := &AnalysisReport{
    ID:          id.New("rpt"),
    UserID:      sc.UserID,
    SessionID:   sc.SessionID,
    TaskID:      sc.TaskID,
    ReportType:  reportType,
    Title:       title,
    Content:     content,
    ArtifactID:  artifact.ID,      // ← 引用 Artifact
    CreatedAt:   time.Now(),
    UpdatedAt:   time.Now(),
}

filter := bson.M{"task_id": sc.TaskID, "report_type": reportType}
update := bson.M{"$setOnInsert": report}
opts := options.Update().SetUpsert(true)

```

```

result, err := s.mongo.UpdateOne(ctx, "analysis_reports", filter, update, opts)

if err != nil {
    s.artifactLogic.DeleteArtifact(ctx, artifact.ID) // best-effort rollback
    return nil, fmt.Errorf("save_report: db upsert failed: %w", err)
}

s.audit.LogValidation(ctx, AuditValidationLog{TaskID: sc.TaskID, ReportType:
reportType, Passed: true, Action: "saved"})

return map[string]any{
    "saved":      true,
    "report_id":  report.ID,
    "artifact_id": artifact.ID,
    "is_new":     result.UpsertedCount > 0,
}, nil
}

```

Agent 使用示例 (Agent 自行管理重试循环):

```

Agent decides to save report → calls save_analysis_report
├─ [FAIL] Skill returns feedback with missing sections
│   Agent reads feedback, revises report
│   Agent calls save_analysis_report again (retry_count=1)
│   ...
│   Up to max_retries (configured in report_rules)
│   If still failing after max_retries:
│       Agent annotates report with warnings and saves as draft
└─ [PASS] Skill saves to MongoDB, returns result_id
        Agent proceeds to next step

```

规则配置 (保持不变, 存储在 MongoDB `report_rules` 集合):

```

{
  "_id": "rule_financial_analysis",
  "report_type": "financial_analysis",
  "description": "财务分析报告格式规范",
  "max_retries": 3,
  "enabled": true,
  "rules": [
    {

```

```

    "name": "has_summary",
    "type": "section_required",
    "match_mode": "md_ast",
    "sections": ["摘要", "执行摘要"],
    "min_level": 2,
    "feedback": "请补充「摘要」章节，概述本次分析的核心发现"
  },
  {
    "name": "has_data_source",
    "type": "section_required",
    "match_mode": "md_ast",
    "sections": ["数据来源", "数据说明"],
    "min_level": 2,
    "feedback": "请补充「数据来源」章节，说明分析所使用的数据范围和时间"
  },
  {
    "name": "has_methodology",
    "type": "section_required",
    "match_mode": "md_ast",
    "sections": ["分析方法"],
    "min_level": 2,
    "feedback": "请补充「分析方法」章节，说明采用的统计方法及其选择理由"
  },
  {
    "name": "has_key_metrics",
    "type": "section_required",
    "match_mode": "md_ast",
    "sections": ["关键指标", "核心指标"],
    "min_level": 2,
    "feedback": "请补充「关键指标」章节，列出核心财务指标及其变化"
  },
  {
    "name": "has_comparison",
    "type": "element_required",
    "match_mode": "md_ast",
    "sections": [],
    "min_level": 0,
    "keywords": ["同比", "环比", "对比", "较上"],
    "feedback": "请补充同比/环比对比数据，说明各项指标的变化趋势"
  }

```

```
    },
    {
      "name": "has_conclusion",
      "type": "section_required",
      "match_mode": "md_ast",
      "sections": ["结论", "总结", "建议"],
      "min_level": 2,
      "feedback": "请补充「结论与建议」章节，给出分析结论和可操作的建议"
    },
    {
      "name": "min_word_count",
      "type": "format_constraint",
      "threshold": 200,
      "unit": "characters",
      "feedback": "报告内容过短（少于200字），请丰富分析内容和数据支撑"
    }
  ]
}
```

预置报告类型与规则:

报告类型	关联分析类型	必需章节	最小字数	最大重试
financial_analysis	财务分析	摘要、数据来源、分析方法、关键指标、同比环比、结论	200	3
sales_analysis	销售分析	摘要、时间范围、核心指标、趋势分析、TOP/N、结论	150	3
general_analysis	SQL统计、高级分析	摘要、数据描述、分析过程、关键发现、结论	100	3
weekly_report	经营周报	本周概览、核心指标达成率、异常标注、对比上周、下周关注	200	3
user_behavior	用户分析	摘要、数据范围、用户分层、关键行为指标、洞察建议	150	3

与审计/安全层的关系: save_analysis_report Skill 的校验逻辑独立于安全审查中间件。安全审查层仅负责输入/输出的敏感内容和越权检查（Section 8.1），不介入报告格式校验。报告格式的合规性由 Skill 在保存时自主保证。

4.7 SQL 执行器 (含 AST 语法树解析)

功能: SQL Executor Skill 负责将 Agent 生成的 SQL 语句安全地执行到企业数据源。核心安全机制：在 SQL 执行前通过 AST 语法树解析，识别并拒绝所有写入/修改类操作。

4.7.1 AST 解析流程



4.7.2 AST 节点白名单策略

采用白名单 + 黑名单双重策略:

允许的操作 (白名单): | 操作类型 | SQL 语句 | AST Node Type | |-----|-----|-----|
 ----|| 数据查询 | `SELECT` | `*ast.SelectStmt` || 公共表达式 | `WITH ... AS` | `*ast.With` || 子查询 |
 subquery | `*ast.SubqueryExpr` || 表连接 | `JOIN` | `*ast.Join` || 集合操作 | `UNION`, `INTERSECT`,
`EXCEPT` | `*ast.SetOprStmt` || 聚合函数 | `GROUP BY`, `HAVING` | embedded in `SelectStmt` || 排序分页 |
`ORDER BY`, `LIMIT`, `OFFSET` | embedded in `SelectStmt` || 数据描述 | `DESCRIBE`, `SHOW`, `EXPLAIN` |
`*ast.ShowStmt` / `*ast.ExplainStmt` |

禁止的操作 (黑名单 — 立即拒绝): | 操作类型 | SQL 语句 | AST Node Type | 严重级别 | |-----|-----|
 -----|-----|-----|:---:| | 数据插入 | `INSERT` | `*ast.InsertStmt` | HIGH || 数据更新 |
`UPDATE` | `*ast.UpdateStmt` | CRITICAL || 数据删除 | `DELETE` | `*ast.DeleteStmt` | CRITICAL || 表结
 构 变 更 | `DROP`, `ALTER`, `TRUNCATE`, `RENAME` | `*ast.DropStmt`, `*ast.AlterTableStmt`,
`*ast.TruncateStmt`, `*ast.RenameTableStmt` | CRITICAL || 数据库操作 | `CREATE DATABASE`, `DROP`
`DATABASE` | `*ast.CreateDatabaseStmt`, `*ast.DropDatabaseStmt` | CRITICAL || 表 创 建 | `CREATE`
`TABLE` | `*ast.CreateTableStmt` | HIGH || 索 引 操 作 | `CREATE INDEX`, `DROP INDEX` |
`*ast.CreateIndexStmt`, `*ast.DropIndexStmt` | HIGH || 权 限 操 作 | `GRANT`, `REVOKE` |
`*ast.GrantStmt`, `*ast.RevokeStmt` | CRITICAL || 事 务 控 制 | `BEGIN`, `COMMIT`, `ROLLBACK` |
`*ast.BeginStmt`, `*ast.CommitStmt` | HIGH || 变 量 赋 值 | `SET @var = ...` | `*ast.SetStmt` |
MEDIUM || 存 储 过 程 | `CALL`, `EXEC` | `*ast.CallStmt` | HIGH || 数 据 加 载 | `LOAD DATA` |
`*ast.LoadDataStmt` | CRITICAL || 表 锁 操 作 | `LOCK TABLES`, `UNLOCK TABLES` |
`*ast.LockTablesStmt`, `*ast.UnlockTablesStmt` | HIGH |

4.7.3 核心实现

```
// SQL Executor Skill — 包含 AST 安全校验
type SQLExecutorSkill struct {
    dbConn    *sql.DB           // 只读连接的数据库
    audit     *AuditService
    security  *SecurityService
}

// SQLParseResult 保存解析结果
type SQLParseResult struct {
    SQL        string           // 原始 SQL
    Parsed     bool            // 是否成功解析
```

```

    ASTType      string          // AST 根节点类型

    DBOperation  string          // 数据库操作类型: SELECT | INSERT | UPDATE | ...

    IsReadOnly   bool            // 是否只读操作

    Forbidden    bool            // 是否被禁止

    Violations   []Violation     // 违规详情

    Tables       []string        // 涉及的表名
}

type Violation struct {
    Rule      string // 触发的规则名称
    NodeType  string // 违规的 AST 节点类型
    Severity  string // LOW | MEDIUM | HIGH | CRITICAL
    Message   string // 违规说明
}

// ParseAndValidate 是 SQL Executor 的入口
func (s *SQLExecutorSkill) ParseAndValidate(ctx context.Context, sql string)
(*SQLParseResult, error) {
    // Step 1: 基础正则预检 (快速拦截明显攻击)
    if s.hasDangerousPatterns(sql) {
        return nil, ErrDangerousSQL
    }

    // Step 2: AST 语法树解析
    // 使用 pingcap/tidb/parser (兼容 MySQL) 或 xwb1989/sqlparser
    stmts, err := s.parser.Parse(sql)
    if err != nil {
        // 解析失败 - 拒绝执行
        s.audit.LogSQLRejection(ctx, sql, "PARSE_ERROR", err.Error())
        return nil, fmt.Errorf("SQL 语法解析失败: %w", err)
    }

    // Step 3: AST 遍历 - 检测禁止操作
    result := &SQLParseResult{SQL: sql, Parsed: true}
    for _, stmt := range stmts {
        violations := s.walkASTForForbidden(stmt)
        result.Violations = append(result.Violations, violations...)

        switch stmt.(type) {

```

```

        case *ast.SelectStmt:
            result.ASTType = "SELECT"
            result.IsReadOnly = true
        case *ast.InsertStmt:
            result.ASTType = "INSERT"
            result.Forbidden = true
        case *ast.UpdateStmt:
            result.ASTType = "UPDATE"
            result.Forbidden = true
        case *ast.DeleteStmt:
            result.ASTType = "DELETE"
            result.Forbidden = true
        case *ast.DropStmt:
            result.ASTType = "DROP"
            result.Forbidden = true
        // ... 其他禁止类型
    }
}

// Step 4: 拒绝写入/修改操作
if result.Forbidden {
    s.audit.LogSQLRejection(ctx, sql, result.ASTType, "WRITE_OPERATION_BLOCKED")
    s.security.Alert(ctx, SecurityAlert{
        Type:      "SQL_WRITE_BLOCKED",
        Severity: "CRITICAL",
        SQL:       sql,
        ASTType:   result.ASTType,
    })
    return result, fmt.Errorf("禁止的 SQL 操作类型: %s。仅允许 SELECT/DESCRIBE/SHOW/EXPLAIN 等只读操作", result.ASTType)
}

return result, nil
}

// walkASTForForbidden 递归遍历 AST 树查找禁止的节点类型
func (s *SQLExecutorSkill) walkASTForForbidden(node ast.Node) []Violation {
    var violations []Violation
    if node == nil { return nil }

```

```
// 黑名单检查
switch n := node.(type) {
case *ast.InsertStmt:
    violations = append(violations, Violation{
        Rule: "no_write", NodeType: "INSERT", Severity: "HIGH",
        Message: "不允许 INSERT 操作",
    })
case *ast.UpdateStmt:
    violations = append(violations, Violation{
        Rule: "no_write", NodeType: "UPDATE", Severity: "CRITICAL",
        Message: "不允许 UPDATE 操作",
    })
case *ast.DeleteStmt:
    violations = append(violations, Violation{
        Rule: "no_write", NodeType: "DELETE", Severity: "CRITICAL",
        Message: "不允许 DELETE 操作",
    })
case *ast.DropStmt:
    violations = append(violations, Violation{
        Rule: "no_ddl", NodeType: "DROP", Severity: "CRITICAL",
        Message: "不允许 DROP 操作",
    })
case *ast.TruncateStmt:
    violations = append(violations, Violation{
        Rule: "no_ddl", NodeType: "TRUNCATE", Severity: "CRITICAL",
        Message: "不允许 TRUNCATE 操作",
    })
case *ast.AlterTableStmt:
    violations = append(violations, Violation{
        Rule: "no_ddl", NodeType: "ALTER TABLE", Severity: "CRITICAL",
        Message: "不允许 ALTER TABLE 操作",
    })
case *ast.CreateTableStmt:
    violations = append(violations, Violation{
        Rule: "no_ddl", NodeType: "CREATE TABLE", Severity: "HIGH",
        Message: "不允许 CREATE TABLE 操作",
    })
case *ast.SetStmt:
```

```

        violations = append(violations, Violation{
            Rule: "no_variable", NodeType: "SET", Severity: "MEDIUM",
            Message: "不允许 SET 变量操作",
        })
    // ... 更多禁止节点类型
default:
    // 允许的节点类型, 不做处理
}

// 递归检查子节点
node.Accept(func(child ast.Node) error {
    if child != node {
        violations = append(violations, s.walkASTForbidden(child)...)
    }
    return nil
})

return violations
}

// hasDangerousPatterns 快速正则预检
func (s *SQLExecutorSkill) hasDangerousPatterns(sql string) bool {
    patterns := []string{
        `(?(i)\bDROP\s+(TABLE|DATABASE|INDEX|VIEW)\b`,
        `(?(i)\bTRUNCATE\s+(TABLE\s+)?`,
        `(?(i)\bALTER\s+(TABLE|DATABASE)\b`,
        `(?(i);\s*(INSERT|UPDATE|DELETE|DROP|ALTER|TRUNCATE)`, // 换行注入
        `(?(i)'\s*OR\s*'1'='1`, // SQL 注入
        `(?(i)\bUNION\s+SELECT\b.*\bFROM\b`, // UNION 注入 (仅在一个 SQL 中
        出现非预期的 UNION SELECT)
    }
    for _, p := range patterns {
        if matched, _ := regexp.MatchString(p, sql); matched {
            return true
        }
    }
    return false
}

```

```
// Execute 仅当 SQL 通过 AST 校验后才执行
func (s *SQLExecutorSkill) Execute(ctx context.Context, params map[string]any) (any, error)
{
    sql := params["sql"].(string)

    // AST 解析 + 安全校验
    result, err := s.ParseAndValidate(ctx, sql)
    if err != nil {
        return nil, err
    }

    // 执行只读 SQL
    rows, err := s.dbConn.QueryContext(ctx, sql)
    if err != nil {
        s.audit.LogSQLExecution(ctx, sql, "ERROR", err.Error())
        return nil, err
    }
    defer rows.Close()

    // 序列化结果
    data := s.serializeRows(rows)

    s.audit.LogSQLExecution(ctx, sql, "SUCCESS", fmt.Sprintf("%d rows", len(data)))
    return data, nil
}
```

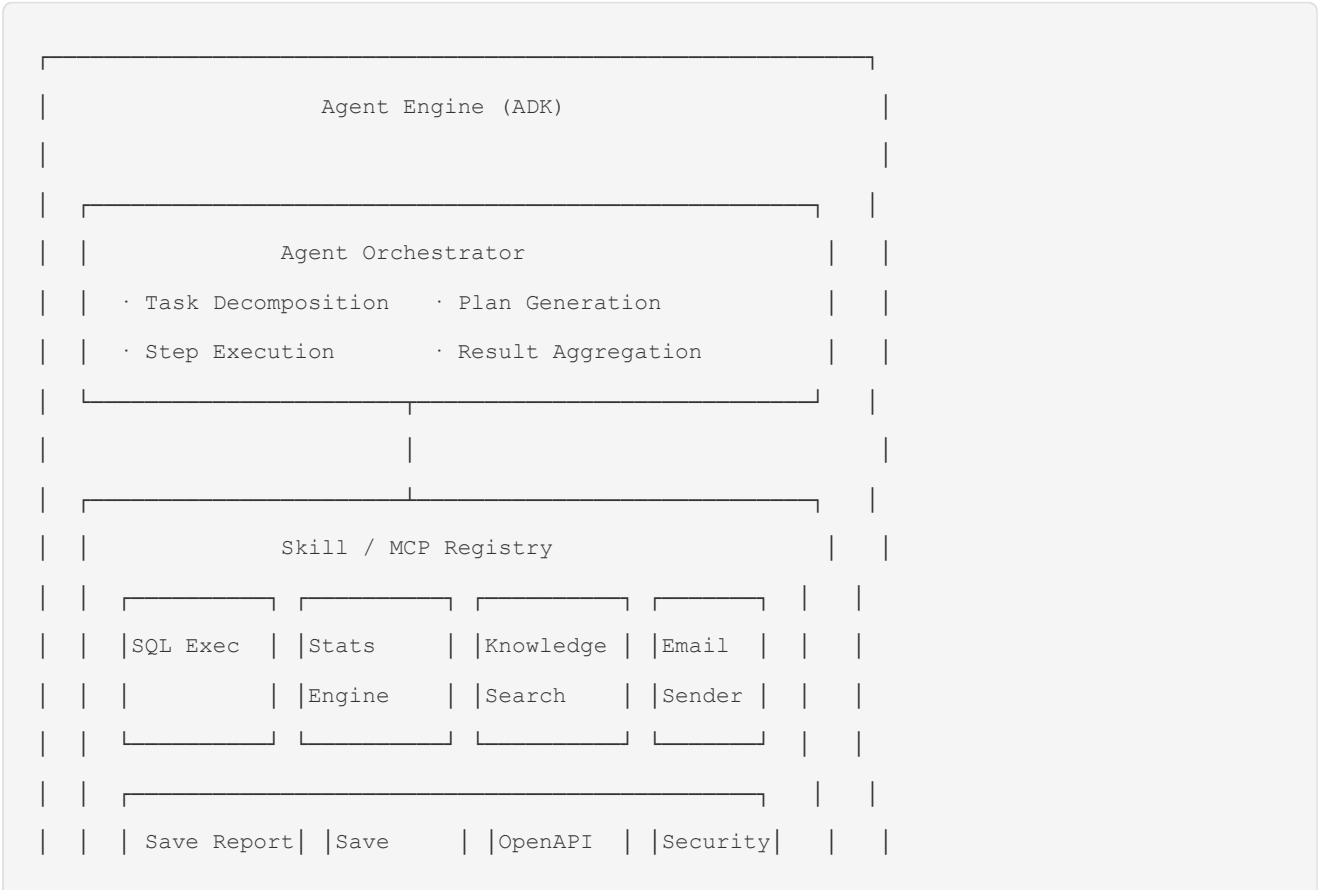
4.7.4 推荐 AST 解析库

库	适用场景	优势
<code>github.com/pingcap/tidb/parser</code>	MySQL 兼容	TiDB 出品，生产级品质，支持 MySQL 5.7/8.0 语法
<code>github.com/xwb1989/sqlparser</code>	通用 SQL	轻量级，Google Vitess 分支，支持 MySQL 语法
<code>github.com/auxten/postgresql-parser</code>	PostgreSQL	CockroachDB 出品，支持 PostgreSQL 语法
<code>github.com/dolthub/go-mysql-server</code>	MySQL 兼容	完整的 MySQL 语法支持，包含分析引擎

MVP 选型: `pingcap/tidb/parser` — MySQL 兼容性好，AST 节点类型完备，经过大规模生产验证。

6. Agent 系统设计

5.1 ADK Agent 引擎架构





5.2 Agent 系统提示词设计

<agent_system_prompt>

你是企业数据分析助手，具备以下能力：

****核心能力**：**

1. 数据查询与分析：通过 SQL 执行器查询企业数据库，进行统计分析
2. 高级分析：回归分析、聚类分析、主成分分析、时间序列预测
3. 财务分析：财务比率计算、同比环比分析、预算偏差分析
4. 结果解读：结合企业知识库和行业数据，对分析结果进行深度解读

****工具使用规则**：**

- SQL 查询前必须先确认数据源和表结构
- 涉及多表关联查询时，先验证关联条件是否正确
- 分析结果必须通过 `save\_analysis\_report` Skill 落库保存（该 Skill 会自动校验报告格式合规性）
- 若 `save\_analysis\_report` 返回校验失败，根据反馈逐一修正报告后重新调用，直到通过或达到重试上限
- 生成的图表、数据文件等产出物通过 `save\_artifact` Skill 保存为 Artifact
- 查询用户敏感数据前需要额外确认

****安全规则**：**

- ****SQL 执行器自动拦截写入操作****：系统通过 AST 语法树解析，自动识别并拒绝 INSERT/UPDATE/DELETE/DROP/ALTER/TRUNCATE 等 DML 和 DDL 语句。Agent 无需手动判断，SQL Executor Skill 在执行前完成安全校验
- 不得查询其他用户的数据（除非有授权）
- 不得将内部数据通过邮件发送到外部邮箱
- 分析结果中的敏感信息需要脱敏处理

****交互规则**：**

- 复杂分析任务建议使用异步模式，避免等待
- 提供清晰的分析步骤说明
- 结果以结构化方式呈现（表格、图表描述）
- 对于不确定的分析结论，标注置信度

****限制**：**

- 不对外透露系统内部实现细节
- 不执行未授权的 API 调用
- 不修改系统配置

****报告格式要求**：**

- 生成分析报告时，必须按照系统指定的报告类型模板组织内容
- 财务分析报告必须包含：摘要、数据来源与范围、分析方法、关键财务指标、同比/环比数据、结论与建议
- 销售分析报告必须包含：摘要、时间范围、核心指标、趋势分析、TOP/N分析、结论
- 通用分析报告必须包含：摘要、数据描述、分析过程、关键发现、结论

- 若系统反馈格式校验不通过，根据反馈信息逐一修正缺失的章节/要素后重新生成

</agent_system_prompt>

5.3 Skill/MCP 插件机制

Skill 注册接口:

```
// Skill 定义
type Skill interface {
    Name() string
    Description() string
    Parameters() json.RawMessage // JSON Schema
    Execute(ctx context.Context, params map[string]any) (any, error)
    Permissions() []string        // 所需权限
    RateLimit() *RateLimit        // 频率限制
}

// Skill 注册中心
type SkillRegistry struct {
    skills map[string]Skill
    mu      sync.RWMutex
}

func (r *SkillRegistry) Register(s Skill) error {
    r.mu.Lock()
    defer r.mu.Unlock()
    if _, exists := r.skills[s.Name()]; exists {
        return fmt.Errorf("skill %s already registered", s.Name())
    }
    r.skills[s.Name()] = s
    return nil
}

func (r *SkillRegistry) GetAdkTools() []adk.Tool {
    // 将所有 Skill 转换为 ADK Tool 格式
    // 自动生成 Function Declaration
}
```

Skill 目录结构:

```
skills/
├─ sql-executor/
│   ├── skill.go          # Skill 实现
│   ├── config.yaml       # Skill 配置 (MCP 连接信息等)
│   └─ SKILL.md           # Skill 说明文档
├─ stats-engine/
│   ├── skill.go
│   ├── regression.go     # 回归分析
│   ├── clustering.go     # 聚类分析
│   ├── pca.go            # 主成分分析
│   ├── time_series.go    # 时间序列
│   └─ config.yaml
├─ email-sender/
│   ├── skill.go
│   └─ config.yaml
├─ openapi-converter/
│   ├── skill.go
│   └─ config.yaml
├─ knowledge-search/
│   ├── skill.go
│   └─ config.yaml
├─ save-analysis-report/
│   ├── skill.go
│   └─ config.yaml
├─ save-artifact/
│   ├── skill.go
│   └─ config.yaml
└─ security-audit/
    ├── skill.go
    └─ config.yaml
└─ workspace-read/
    ├── skill.go
    └─ config.yaml
└─ workspace-write/
    ├── skill.go
    └─ config.yaml
└─ workspace-exec/
```

```
├─ skill.go
└─ config.yaml
```

Skill 自动加载:

```
// 启动时自动扫描 skills/ 目录

func (r *SkillRegistry) AutoLoad(skillsDir string) error {
    entries, _ := os.ReadDir(skillsDir)
    for _, entry := range entries {
        if !entry.IsDir() { continue }
        skillPath := filepath.Join(skillsDir, entry.Name())
        skill, err := LoadSkill(skillPath) // 读取 config.yaml + 初始化
        if err != nil {
            log.Warnf("Failed to load skill %s: %v", entry.Name(), err)
            continue
        }
        r.Register(skill)
        log.Infof("Skill loaded: %s", skill.Name())
    }
    return nil
}
```

5.4 Skill 与 Session 绑定机制

设计目标： Skill 执行时必须自动继承当前 Session 上下文，无需 Agent 或 Caller 手动传递 `session_id`。确保 workspace 操作、Artifact 产出、报告保存**强制关联**当前 Session，杜绝跨 Session 操作和数据归属混乱。

5.4.1 SkillContext — 注入式 Session 上下文

```
// internal/skill/context.go

// SkillContext 封装 Skill 执行的运行时上下文
// 由 Agent Engine 在执行 Skill 前自动注入，禁止 Skill 自行修改

type SkillContext struct {
    SessionID string    `json:"session_id"` // 当前 Session ID (自动注入)
    UserID     string    `json:"user_id"`    // 当前用户 ID
    TaskID     string    `json:"task_id"`    // 当前任务 ID (可选, Agent 模式)
    TraceID    string    `json:"trace_id"`   // 请求追踪 ID
}
```

```

Role      string    `json:"role"`      // 用户角色（用于权限校验）

// 私有字段，仅 framework 可访问

workspaceRoot string // /workspaces/<session_id>/

}

```

5.4.2 升级后的 Skill 接口

```

// Skill（升级版）- Execute 第一个参数强制为 SkillContext
type Skill interface {
    Name() string
    Description() string
    Parameters() json.RawMessage

    // 核心变更: SkillContext 替代零散的 session_id/user_id 参数
    Execute(ctx context.Context, sc SkillContext, params map[string]any) (any, error)

    Permissions() []string
    RateLimit() *RateLimit
}

```

Agent Engine 注入流程:

```

// internal/agent/engine.go

func (e *AgentEngine) invokeSkill(ctx context.Context, toolCall adk.ToolCall) (any, error) {
    skill := e.registry.Get(toolCall.Name)

    // 自动构建 SkillContext（从 gin.Context / Agent 运行时提取）
    sc := SkillContext{
        SessionID:    e.currentSession.ID,      // 自动注入
        UserID:       e.currentSession.UserID,
        TaskID:       e.currentTask.ID,         // Agent 模式有值, Chat 模式为空
        TraceID:      e.traceID,               // 从 X-Trace-Id header 继承
        Role:         e.currentUser.Role,
        workspaceRoot: e.workspaceManager.Root(e.currentSession.ID),
    }
}

```

```
logger.Debug("[AgentEngine] invoking skill",
    zap.String("component", "agent_engine"),
    zap.String("trace_id", sc.TraceID),
    zap.String("session_id", sc.SessionID),
    zap.String("skill_name", toolCall.Name),
)

return skill.Execute(ctx, sc, toolCall.Arguments)
}
```

5.4.3 Session 绑定强制规则

Skill	绑定规则	实现方式
<code>workspace_read</code>	路径强制限定 <code>/workspaces/<sc.SessionID>/</code>	<code>resolveSafePath</code> 使用 <code>sc.SessionID</code> ，不接受外部传入
<code>workspace_write</code>	同上	同上
<code>workspace_exec</code>	同上 + 环境变量 <code>HOME</code> / <code>TMPDIR</code> 限定	同上
<code>save_artifact</code>	<code>artifact.session_id</code> 自动设为 <code>sc.SessionID</code>	MongoDB <code>session_id</code> 字段由 framework 填入，Skill 无法覆盖
<code>save_analysis_report</code>	<code>report.session_id</code> 自动设为 <code>sc.SessionID</code>	同上
<code>sql_executor</code>	记录 SQL 日志时自动关联 <code>sc.SessionID</code> + <code>sc.TaskID</code>	审计字段自动注入

关键约束：

1. Skill 不得接受外部 `session_id` 参数 — 由 SkillContext 注入，Skill 实现中不暴露 `session_id` 给 LLM/Agent 填值。
2. 数据归属强制写入 — `save_artifact` 和 `save_analysis_report` 的 MongoDB 文档中，`session_id` 字段由 Skill framework 在 Execute 内部设置，Skill 代码传入的 `params["session_id"]` 被忽略。
3. 跨 Session 操作拒绝 — workspace Skill 的 `resolveSafePath` 以 `sc.SessionID` 为锚，即使 Skill 代码内部尝试访问其他 session 目录，也会被路径沙箱拦截。

5.4.4 改造示例：save_artifact

```
// internal/skill/save_artifact.go

type SaveArtifactSkill struct {
    logic *logic.ArtifactLogic // ← 共用 Logic 层, 内部封装 SeaweedFS + MongoDB 操作
}

type ArtifactRecord struct {
    ID          string    `bson:"_id"`
    TaskID      string    `bson:"task_id"`
    SessionID   string    `bson:"session_id"` // ← 由 framework 注入, Skill 不可覆盖
    UserID      string    `bson:"user_id"`    // ← 同上
    Name        string    `bson:"name"`
    MimeType     string    `bson:"mime_type"`  // ← 替代 type 字段 (由文件内容推断)
    SizeBytes   int64     `bson:"size_bytes"`
    StoragePath string    `bson:"storage_path"`
    Persistent  bool      `bson:"persistent"`
    CreatedAt   time.Time `bson:"created_at"`
    UpdatedAt   time.Time `bson:"updated_at"`
}

func (s *SaveArtifactSkill) Execute(ctx context.Context, sc SkillContext, params
map[string]any) (any, error) {
    name, _ := params["name"].(string)
    content, _ := params["content"].([]byte)
    mimeType, _ := params["mime_type"].(string)

    // 通过 Logic 层幂等创建 (共用 save_report 底层逻辑)
    artifact, err := s.logic.CreateArtifact(ctx, sc, logic.CreateArtifactInput{
        Name:      name,
        Content:   content,
        MimeType:  mimeType,
    })
    if err != nil {
        return nil, fmt.Errorf("save_artifact: %w", err)
    }

    logger.Info("[SaveArtifact] created",
```



```

        zap.String("component", "skill"),
        zap.String("session_id", sc.SessionID),
        zap.String("artifact_id", artifact.ID),
    )

    return map[string]any{"artifact_id": artifact.ID, "path": artifact.StoragePath}, nil
}

```

5.4.5 改造示例：save_analysis_report

```

func (s *SaveReportSkill) Execute(ctx context.Context, sc SkillContext, params
map[string]any) (any, error) {
    reportType, _ := params["report_type"].(string)
    content, _ := params["content"].(string)

    // 格式校验
    validation := s.validator.Validate(reportType, content)
    if !validation.Passed {
        return map[string]any{"status": "validation_failed", "errors": validation.Errors,
"retry": true}, nil
    }

    // 通过 Logic 层创建 Artifact + Report (幂等, 失败时自动回滚)
    report, err := s.reportLogic.CreateReport(ctx, sc, logic.CreateReportInput{
        ReportType: reportType,
        Content:     content,
    })
    if err != nil {
        return nil, err
    }

    return map[string]any{"status": "saved", "report_id": report.ID, "artifact_id":
report.ArtifactID}, nil
}

```

5.4.6 约束总览



5.5 OpenAPI → MCP 转换器

转换流程:

```

OpenAPI Spec (JSON/YAML)
  → Parse via kin-openapi
  → Extract: endpoints, methods, parameters, schemas
  → Generate MCP Tool Definition:
    - Tool Name: operationId
    - Tool Description: summary/description
    - Parameters: JSON Schema from OpenAPI schema
  → Create Dynamic Skill (内存中)
  → Submit for Admin Review
  → [Approved] → Register to SkillRegistry
  → [Rejected] → Delete from pending list

```

核心实现:

```

type OpenAPIConverter struct {
    parser    *openapi3.Loader
    registry *SkillRegistry
}

type PendingAPIConversion struct {
    ID            string    `bson:"_id"`
    SpecContent string    `bson:"spec_content"`
    Tools         []ToolDef `bson:"tools"`
    SubmittedBy string    `bson:"submitted_by"`
    ReviewedBy string    `bson:"reviewed_by"`
    Status        string    `bson:"status"` // pending | approved | rejected
    CreatedAt     time.Time `bson:"created_at"`
}

func (c *OpenAPIConverter) Convert(specContent []byte) ([]ToolDef, error) {
    doc, _ := openapi3.NewLoader().LoadFromData(specContent)
    var tools []ToolDef
    for path, pathItem := range doc.Paths.Map() {
        for method, op := range pathItem.Operations() {
            tool := ToolDef{
                Name:      op.OperationID,

```

```
        Description: op.Summary,
        Method:      method,
        Path:        path,
        Parameters:  buildJSONSchema(op.Parameters),
        BaseURL:     doc.Servers[0].URL,
    }
    tools = append(tools, tool)
}
}
return tools, nil
}
```

7. 数据模型设计

7.0 数据表统一规范

所有 MongoDB Collection 遵循以下字段约定：

标准字段

字段	类型	必填	说明
<code>_id</code>	<code>string</code> (UUID v4)	✓	主键，36 字符 UUID，由应用层生成（非 MongoDB ObjectId）
<code>user_id</code>	<code>string</code>	视情况	数据归属用户 ID，记录/审计类表必须
<code>session_id</code>	<code>string</code>	视情况	数据归属 Session ID，Session 生命周期内的操作产物必须
<code>task_id</code>	<code>string</code>	视情况	关联 Agent 任务 ID，异步任务产物必须
<code>created_at</code>	<code>ISODate</code>	✓	创建时间，服务端写入
<code>updated_at</code>	<code>ISODate</code>	✓	最后更新时间，每次修改自动更新
<code>deleted_at</code>	<code>ISODate</code>	✗	软删除时间， <code>null</code> 表示未删除（仅需要软删除的表使用）

UUID 主键生成

```
// internal/pkg/id/id.go

package id

import "github.com/google/uuid"

// 前缀可读类型 + UUID v4
func New(prefix string) string {
    return prefix + "_" + uuid.New().String()
}

// 示例: id.New("task") → "task_3f7a2b1c-..."
//       id.New("rpt")  → "rpt_a1b2c3d4-..."
```

索引设计原则

1. 按查询 case 建索引 — 先梳理实际查询场景，再建对应索引，避免"先建再说"。
2. 唯一索引优先 — 天然唯一的字段（如 `order_no`、`email`）用唯一索引防脏数据。
3. 复合索引覆盖查询 — 常见查询条件组建成复合索引，避免多个单字段索引重复。
4. ESR 规则 — 复合索引字段顺序：Equality → Sort → Range。
5. 避免功能重复索引 — `{a:1, b:1}` 已覆盖仅按 `a` 查询的场景，无需额外建 `{a:1}`。
6. TTL 索引清理临时数据 — Session、临时 Artifact 等用 TTL 索引自动清理。
7. 索引数量控制在 5 个以内 — 每表索引数不超过 5 个，写入性能与查询性能平衡。

7.1 MongoDB 集合设计（完整字段）

MongoDB 作为业务主存储：所有业务实体数据统一存储在 MongoDB 中。SeaweedFS 仅存储二进制大文件（Session 临时文件、Artifact 文件内容），Milvus 仅存储向量索引，Redis 仅做缓存和消息队列。

Collection	用途	关键字段	索引
users	用户信息	_id, username, password_hash, roles[], status, created_at, updated_at	{username:1} unique, {status:1}
roles	角色定义	_id, name, permissions[], created_at	{name:1} unique
sessions	用户会话	_id, user_id, type, expires_at, workspace_id, created_at, updated_at	{user_id:1, created_at:-1}, {expires_at:1} TTL
chat_messages	对话消息	_id, session_id, user_id, role, content, created_at	{session_id:1, created_at:1}
agent_tasks	Agent 任务	_id, user_id, session_id, status, input, progress, result, created_at, updated_at	{user_id:1, created_at:-1}, {status:1, created_at:1}
scheduled_tasks	定时任务	_id, user_id, cron_expr, agent_input, status, created_at, updated_at	{status:1}, {user_id:1}
analysis_reports	分析报告	_id, user_id, session_id, task_id, report_type, title, content, artifact_id, created_at, updated_at	{task_id:1}, {session_id:1, created_at:-1}, {user_id:1, created_at:-1}
aggregation_layers	聚合层数据	_id, name, dimensions[], metrics[], source, data, created_at, updated_at	{name:1} unique
knowledge_docs	知识库文档元数据	_id, user_id, title, type, tags[], permissions, versions[], created_at, updated_at	{user_id:1, created_at:-1}, {tags:1}
knowledge_doc_contents	知识库文档内容	_id, doc_id, version, gridfs_file_id, filename, size_bytes, created_at	{doc_id:1, version:-1}

Collection	用途	关键字段	索引
	(GridFS 引用)		
<code>prompts</code>	快捷提示词	<code>_id</code> , <code>category</code> , <code>text</code> , <code>roles[]</code> , <code>sort_order</code> , <code>created_at</code>	<code>{category:1, sort_order:1}</code>
<code>audit_logs</code>	审计日志	<code>_id</code> , <code>user_id</code> , <code>session_id</code> , <code>action</code> , <code>ip</code> , <code>detail</code> , <code>created_at</code>	<code>{user_id:1, created_at:-1}</code> , <code>{created_at:-1}</code>
<code>security_alerts</code>	安全告警	<code>_id</code> , <code>type</code> , <code>user_id</code> , <code>content</code> , <code>rule</code> , <code>created_at</code>	<code>{created_at:-1}</code> , <code>{type:1, created_at:-1}</code>
<code>pending_api_conversions</code>	API 转换审核	<code>_id</code> , <code>user_id</code> , <code>spec_content</code> , <code>tools[]</code> , <code>status</code> , <code>created_at</code> , <code>updated_at</code>	<code>{status:1}</code> , <code>{user_id:1}</code>
<code>model_configs</code>	模型配置	<code>_id</code> , <code>provider</code> , <code>base_url</code> , <code>api_key_vault_id</code> , <code>model_name</code> , <code>temperature</code> , <code>top_p</code> , <code>max_context_tokens</code> , <code>max_output_tokens</code> , <code>is_active</code> , <code>created_at</code> , <code>updated_at</code>	<code>{is_active:1}</code>
<code>vault_secrets</code>	Vault 加密密钥	<code>_id</code> , <code>key_type</code> , <code>encrypted_value</code> , <code>created_by</code> , <code>created_at</code> , <code>updated_at</code>	<code>{key_type:1}</code>
<code>skill_configs</code>	Skill 配置	<code>_id</code> , <code>skill_name</code> , <code>config_yaml</code> , <code>enabled</code> , <code>rate_limit</code> , <code>created_at</code> , <code>updated_at</code>	<code>{skill_name:1}</code> unique
<code>report_rules</code>	报告格式校验规则	<code>_id</code> , <code>report_type</code> , <code>max_retries</code> , <code>enabled</code> , <code>rules[]</code> , <code>created_at</code> , <code>updated_at</code>	<code>{report_type:1}</code> unique

Collection	用途	关键字段	索引
<code>report_validation_logs</code>	报告校验 审计日志	<code>_id</code> , <code>task_id</code> , <code>report_type</code> , <code>result</code> , <code>retry_attempt</code> , <code>failures[]</code> , <code>action</code> , <code>created_at</code>	<code>{task_id:1}</code> , <code>{created_at:-1}</code>
<code>artifacts</code>	Artifact 元数据	<code>_id</code> , <code>user_id</code> , <code>session_id</code> , <code>task_id</code> , <code>name</code> , <code>mime_type</code> , <code>size_bytes</code> , <code>storage_path</code> , <code>persistent</code> , <code>created_at</code> , <code>updated_at</code>	<code>{task_id:1}</code> , <code>{session_id:1}</code> , <code>created_at:-1}</code> , <code>{persistent:1}</code> , <code>created_at:-1}</code>
<code>token_consumptions</code>	Token 消 耗记录	<code>_id</code> , <code>session_id</code> , <code>task_id</code> , <code>model_name</code> , <code>input_tokens</code> , <code>output_tokens</code> , <code>cost_cents</code> , <code>created_at</code>	<code>{created_at:-1}</code> , <code>{model_name:1}</code> , <code>created_at:-1}</code>
<code>output_stats</code>	产出统计 (聚合)	<code>_id</code> , <code>date</code> , <code>report_count</code> , <code>chart_count</code> , <code>download_count</code> , <code>export_count</code> , <code>created_at</code> , <code>updated_at</code>	<code>{date:1}</code> unique
<code>roi_metrics</code>	ROI 指标 (聚合)	<code>_id</code> , <code>date</code> , <code>total_cost_cents</code> , <code>total_tasks</code> , <code>equivalent_man_hours_saved</code> , <code>created_at</code>	<code>{date:1}</code> unique
<code>kb_index_tasks</code>	知识库索 引任务	<code>_id</code> , <code>doc_id</code> , <code>status</code> , <code>chunk_count</code> , <code>model_name</code> , <code>created_at</code> , <code>updated_at</code>	<code>{doc_id:1}</code> , <code>created_at:-1}</code> , <code>{status:1}</code>
<code>kb_chunks</code>	知识库文 档分片	<code>_id</code> , <code>doc_id</code> , <code>index_task_id</code> , <code>chunk_index</code> , <code>content</code> , <code>embedding</code> , <code>created_at</code>	<code>{doc_id:1}</code> , <code>chunk_index:1}</code> , <code>{embedding:</code> <code>\\"vector\\"}</code>
<code>prompt_enhancements</code>	增强提示 词记录 (无状 态, 仅记 录)	<code>_id</code> , <code>original_text</code> , <code>enhanced_texts[]</code> , <code>model_name</code> , <code>created_at</code>	<code>{created_at:-1}</code>

Collection	用途	关键字段	索引
<code>im_bindings</code>	IM 用户绑定（飞书 open_id ↔ user_id）	<code>_id</code> , <code>platform</code> , <code>open_id</code> , <code>user_id</code> , <code>tenant_key</code> , <code>nickname</code> , <code>avatar_url</code> , <code>created_at</code> , <code>updated_at</code>	<code>{platform:1, open_id:1, tenant_key:1}</code> unique, <code>{user_id:1}</code>
<code>im_bind_tokens</code>	一次性绑定 Token	<code>_id</code> , <code>platform</code> , <code>open_id</code> , <code>tenant_key</code> , <code>expires_at</code> , <code>created_at</code>	<code>{expires_at:1}</code> TTL
<code>im_templates</code>	IM 消息模板 (V1.1)	<code>_id</code> , <code>platform</code> , <code>template_type</code> , <code>name</code> , <code>content</code> , <code>created_at</code> , <code>updated_at</code>	<code>{platform:1, template_type:1}</code>

关键设计变更: 1. `kb_index_tasks` 使用 Agent 任务框架异步执行（复用 `agent_tasks` 架构），索引完成后更新 `knowledge_docs.index_status` 2. `kb_chunks.embedding` 由当前配置的 LLM 模型生成（非专用 embedding 模型），通过向量索引实现语义检索 3. `kb_chunks.doc_id` 与 `knowledge_docs._id` 强绑定，查询时按 `doc_id` 隔离，避免跨文档串数据 4. `prompt_enhancements` 为无状态记录表，不关联 `session_id` 或 `user_id`，仅用于增强结果缓存和日志审计 5. **KB 文档统一使用 GridFS 存储:** 所有知识库文档上传后存 MongoDB GridFS（无需区分大小，GridFS 自动分片 255KB chunk），`gridfs_file_id` 关联 `knowledge_doc_contents`。索引任务和详情页均通过 `GridFSDownloadStream`（实现 `io.ReadSeeker`）流式读取，不占内存。小文件（<16MB）同样走 GridFS，避免两套路径。

7.2 分析报告数据模型（analysis_reports）

报告独立建表，通过 `artifact_id` 单向引用 `artifacts`，不反向在 `artifacts` 表加 `type` 字段：

```
// analysis_reports collection
{
  "_id": "rpt_alb2c3d4-e5f6-7890-abcd-ef1234567890",
  "user_id": "user_001",
  "session_id": "session_abcl23",
  "task_id": "task_001",
  "report_type": "financial_analysis",
  "title": "2026 Q2 财务分析报告",
```

```

"content": "# 2026 Q2 财务分析报告\n\n# 概述\n...",
"artifact_id": "art_xly2z3-...",
"metadata": {
  "data_sources": ["enterprise_demo.orders", "enterprise_demo.products"],
  "time_range": {"start": "2026-04-01", "end": "2026-06-30"},
  "validation_retries": 1
},
"created_at": "2026-07-01T10:30:00Z",
"updated_at": "2026-07-01T10:30:00Z"
}

```

```

// artifacts collection (对应报告的那个 artifact 记录)
{
  "_id": "art_xly2z3-w4v5-6789-abcd-ef1234567890",
  "user_id": "user_001",
  "session_id": "session_abc123",
  "task_id": "task_001",
  "name": "2026 Q2 财务分析报告.md",
  "mime_type": "text/markdown",
  "size_bytes": 15200,
  "storage_path": "artifacts/task_001/art_xly2z3_report.md",
  "persistent": false,
  "created_at": "2026-07-01T10:30:00Z",
  "updated_at": "2026-07-01T10:30:00Z"
}

```

设计说明： `artifact.type` 字段已移除。判断一个 artifact 是否为“报告”，通过查询 `analysis_reports` 表中是否存在 `artifact_id` 引用即可。Artifact 表保持通用性，不承载业务语义字段。

Report 下载流程 — 复用 Artifact 下载：

```

GET /api/v1/reports/:report_id/download
→ 查询 analysis_reports 获取 artifact_id
→ 查询 artifacts 获取 storage_path
→ SeaweedFS presigned URL → 302 重定向
→ 与 GET /api/v1/artifacts/:id/download 共用同一底层实现

```

6.3 聚合层数据模型

```
// aggregation_layers collection
{
  "_id": "agg_sales_by_region_product",
  "name": "按地区和产品线的销售聚合",
  "dimensions": ["region", "product_line"],
  "metrics": [
    {"name": "total_revenue", "agg_func": "sum", "field": "revenue"},
    {"name": "avg_price", "agg_func": "avg", "field": "unit_price"},
    {"name": "order_count", "agg_func": "count", "field": "order_id"}
  ],
  "source": {
    "type": "raw", // raw | aggregation_layer
    "data_source": "sales_db.orders",
    "parent_agg_id": null // 如果基于其他聚合层, 此处填写父层ID
  },
  "data": [
    {"region": "华东", "product_line": "电子产品", "total_revenue": 5000000, ...},
    ...
  ],
  "refresh_schedule": "0 2 * * *", // 每日凌晨2点刷新
  "created_at": "2026-07-01T00:00:00Z",
  "updated_at": "2026-07-01T02:00:00Z",
  "created_by": "user_001"
}
```

6.4 Milvus 向量集合设计

Collection	用途	向量维度	元数据字段
knowledge_chunks	知识库文档 分块向量	1536 (OpenAI) / 768 (其他)	doc_id, chunk_index, title, tags[], permissions[]
analysis_history	历史分析结果 向量	1536	task_id, user_id, analysis_type, result_id
query_cache	查询语义缓存	1536	query_hash, result_id, hit_count, last_hit_at

8. API 设计

7.1 API 路由结构

Base URL: /api/v1

认证

POST	/auth/login	# 登录
POST	/auth/logout	# 登出
POST	/auth/refresh	# 刷新 Token
GET	/auth/me	# 当前用户信息

Chat

POST	/chat/query	# 发送消息（支持 SSE 流式）
GET	/chat/sessions	# 我的会话列表（历史，按时间倒序）
GET	/chat/sessions/:id	# 会话详情（含历史消息）
DELETE	/chat/sessions/:id	# 删除会话
PUT	/chat/sessions/:id/archive	# 归档会话
GET	/chat/sessions/search	# 搜索历史会话（?q=xxx）
GET	/chat/prompts	# 获取快捷提示词

Prompt Enhancement（无状态，不依赖 Session）

POST	/enhance	# 增强提示词（请求体：{text, context?}）
------	----------	-------------------------------

返回：{original, enhanced: [{text, reason}]}, 不创建 session_id

Agent

POST	/agent/tasks	# 创建 Agent 任务
GET	/agent/tasks	# 任务列表（?page=&limit=&sort=&status=） - 支持分页/排序/筛选
GET	/agent/tasks/:id	# 任务详情（含进度 + Artifact 列表）
POST	/agent/tasks/:id/cancel	# 取消任务
GET	/agent/tasks/:id/result	# 任务结果

Scheduler

POST	/schedules	# 创建定时任务
GET	/schedules	# 定时任务列表
PUT	/schedules/:id	# 更新
DELETE	/schedules/:id	# 删除

```

POST    /schedules/:id/pause    # 暂停
POST    /schedules/:id/resume   # 恢复

# Analysis Results → Reports
GET      /reports                # 报告列表 (?task_id=xxx&report_type=xxx)
GET      /reports/:id           # 报告详情
GET      /reports/:id/download   # 下载报告 (复用 Artifact 下载链路)

# Aggregation
POST     /aggregations          # 创建聚合定义
GET      /aggregations          # 聚合列表
GET      /aggregations/:id      # 聚合详情
DELETE   /aggregations/:id      # 删除聚合
POST     /aggregations/:id/refresh # 手动刷新

# Knowledge Base
POST     /knowledge/docs/batch   # 批量上传 (multipart/form-data, 多个文件)
POST     /knowledge/docs         # 上传单个文档
GET      /knowledge/docs         # 文档列表 (?page=&limit=&sort=&filter=) - 支持分页/排序/筛选
GET      /knowledge/docs/:id     # 文档详情
DELETE   /knowledge/docs/:id     # 删除 (含级联清理 kb_chunks)
GET      /knowledge/search       # 搜索 (?q=xxx&type=semantic|fulltext)
GET      /knowledge/docs/:id/index-status # 索引状态 (pending/indexing/done/failed)

# Admin (需要 Admin 角色)
GET      /admin/dashboard/stats   # 看板统计 (调用量/成功率/耗时/Token/产出/ROI)
GET      /admin/dashboard/token-stat # Token 消耗统计 (按模型/日期聚合)
GET      /admin/dashboard/output-stat # 产出统计 (报告/图表/下载/导出)
GET      /admin/dashboard/roi     # AI Agent ROI 指标
POST     /admin/users             # 创建用户
GET      /admin/users             # 用户列表
PUT      /admin/users/:id         # 更新用户
DELETE   /admin/users/:id         # 删除用户
POST     /admin/roles             # 创建角色
PUT      /admin/roles/:id         # 更新角色
PUT      /admin/models            # 模型配置
GET      /admin/models            # 模型配置列表
GET      /admin/audit-logs        # 审计日志
GET      /admin/security-alerts   # 安全告警

```

```

# API Conversion (需要 Knowledge Admin)
POST    /admin/api-convert/batch # 批量上传 OpenAPI 文件
POST    /admin/api-convert      # 上传单个 OpenAPI
GET     /admin/api-convert      # 转换列表 (?page=&limit=&status=)
POST    /admin/api-convert/:id/approve # 审核通过
POST    /admin/api-convert/:id/reject  # 审核拒绝

# Vault (管理密钥)
POST    /vault/secrets          # 保存密钥 (加密存储)
GET     /vault/secrets/:id/reveal # 解密查看 (需二次验证)
DELETE  /vault/secrets/:id      # 删除密钥

# Artifact (Task Detail)
POST    /tasks/:id/artifacts/download # 批量打包下载 (ZIP)

```

7.3 消息渲染格式规范 (Message Render Schema)

Chat 模式下 AI 回复可能包含多种内容类型，前端需根据 `message\_type` 字段选择对应的渲染组件：

`message_type`	渲染组件	说明
`markdown`	Markdown 渲染器	标准自然语言文本，支持加粗/斜体/列表/链接
`tool_call`	工具调用卡片 (折叠)	显示工具图标、名称、执行耗时、输入参数摘要、输出结果
`sql_query`	SQL 代码卡片	语法高亮渲染，带「复制」和「查看执行计划」按钮
`data_table`	数据表格渲染器	斑马纹表格，支持轻量级排序和「导出 CSV」按钮
`chart`	内嵌图表渲染器	直接渲染图表，含放大/下载/引用工具栏
`progress`	进度指示器	旋转动画 + 状态文本 (查询中.../计算中.../索引中...)

7.4 知识库 LLM 索引流水线 (KB Indexing Pipeline)

知识库文档索引不依赖专用向量化模型，复用当前配置的 LLM 完成分片和 Embedding：

上传文档 → 保存文件 → 创建索引 Agent 任务(异步) ↓ LLM 分析语义段落边界 → 拆分 Chunks (≤2000 tokens) ↓ 每个 Chunk → LLM 生成 Embedding 向量 ↓ 写入 kb_chunks (doc_id 强绑定) + Milvus ↓ 更新 knowledge_docs.index_status = indexed

关键约束：`kb\_chunks.doc\_id` 与知识库文档 ID 强绑定，删除文档时级联清理，Semantic Search 按 `doc\_id` 隔离避免串数据。

```
# Mock LLM Service (仅 dev/test 环境, 需要 Admin 角色)
POST    /mock/v1/chat/completions      # OpenAI-compatible Chat Completions (支持
stream=true SSE)
POST    /mock/v1/chat/completions/stream # 同上 (显式 stream 路径, 便于测试)
POST    /mock/responses                # 向队列写入预设 mock 响应 (管理接口)
DELETE  /mock/responses/:key           # 清空指定 key 的队列
GET     /mock/responses/:key           # 查看指定 key 队列内容 (不消耗)
GET     /mock/responses                # 列出所有 mock key 及队列长度
DELETE  /mock/responses                # 清空所有 mock 数据
```

7.2 关键 API 示例

Chat 查询 (SSE 流式) :

```
// Request: POST /api/v1/chat/query
{
  "session_id": "session_chat_001", // 可选, 不传则创建新会话
  "message": "昨天华东区销售额多少?",
  "stream": true
}

// Response (SSE):
data: {"type": "thinking", "content": "正在分析查询意图..."}
data: {"type": "tool_call", "tool": "sql_executor", "status": "running"}
data: {"type": "tool_result", "tool": "sql_executor", "result": {...}}
data: {"type": "text", "content": "昨天华东区销售额为 1,250 万元..."}
data: {"type": "done"}
```

Agent 任务创建:

```
// Request: POST /api/v1/agent/tasks
{
  "query": "对过去3年全部销售数据做完整的回归分析和预测",
  "data_sources": ["sales_db"],
  "analysis_types": ["regression", "time_series"],
  "mode": "async", // sync | async
  "notification": {
    "email": true,
```

```
    "in_app": true
  }
}

// Response:
{
  "task_id": "task_001",
  "status": "queued",
  "estimated_duration": "约 2-5 分钟",
  "ws_channel": "/ws/tasks/task_001"  // 实时进度 WebSocket
}
```

9. Mock LLM Service（测试用）

9.1 设计目标

Mock Service 在开发 / 测试环境中完整替代真实 LLM（OpenAI / Azure / Anthropic 等），解决以下问题：

问题	Mock Service 解法
真实 LLM 调用费用高	全部请求由本地 Mock 响应，零成本
网络不稳定导致测试抖动	离线运行，延迟可控
测试需要精确控制返回内容	预先向队列注入期望响应
Agent 流程复杂，难以复现特定路径	可构造任意 LLM 输出驱动 Agent 走指定分支

仅在 APP_ENV=development 或 APP_ENV=test 时挂载 Mock 路由，生产环境自动禁用。

9.2 匹配策略：上下文末尾消息 + Redis List 队列

匹配键（Lookup Key）生成

当 Mock Service 收到 POST /mock/v1/chat/completions 请求时：

- 取请求体 messages[] 数组的最后一条消息（messages[len-1]）。

2. 取其 `content` 字段（字符串或 content parts 拼接），做 **SHA-256** 哈希，截取前 16 字节的 hex → `lookupKey`。
3. 以 `mock:resp:<lookupKey>` 为 Redis List Key 执行 **LPOP**。
4. 如果队列为空 → fallback 到通配符 key `mock:resp:*` 做 **LPOP**（按插入时间最早的队列）。
5. 如果仍然没有数据 → 返回内置默认响应（可配置）。

```
messages[last].content → SHA256 → hex[:16] → "mock:resp:<hex>"
```

```
↓
Redis LPOP
```

```
└ 有数据 → 返回该 mock 响应
```

```
└ 无数据 → LPOP "mock:resp:*"
↓
```

```
└ 有数据 → 返回
```

```
└ 无数据 → 内置默认响应
```

设计说明：

- 使用 List（队列）而非 String，同一 key 可以预置多轮不同响应，模拟连续对话或重试场景。
- **LPOP** 消耗式读取，保证每次调用消费一条；测试后队列自然清空。
- 通配符 fallback 让“不关心内容匹配”的测试只需注入一条通用响应。

9.3 接口规范

9.3.1 Chat Completions (OpenAI-compatible)

```
POST /mock/v1/chat/completions
```

```
Content-Type: application/json
```

```
Authorization: Bearer <any-string> # Mock 不校验 key, 仅验证 Bearer 格式
```

```
Request Body (完全兼容 OpenAI Chat Completions v1) :
```

```
{
  "model": "gpt-4o",           // 忽略, Mock 不区分模型
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "请分析上季度销售数据"} // ← 匹配此条
  ],
  "stream": true | false,
  "temperature": 0.7,         // 忽略
}
```

```
"max_tokens": 2000          // 忽略
}
```

非流式响应 (`stream: false`):

```
{
  "id": "chatcmpl-mock-<uuid>",
  "object": "chat.completion",
  "created": 1720000000,
  "model": "mock-llm",
  "choices": [{
    "index": 0,
    "message": {
      "role": "assistant",
      "content": "<预置内容或默认响应>"
    },
    "finish_reason": "stop"
  }],
  "usage": {
    "prompt_tokens": 0,
    "completion_tokens": 0,
    "total_tokens": 0
  }
}
```

流式响应 (`stream: true` , SSE 格式):

```
Content-Type: text/event-stream
Cache-Control: no-cache

data: {"id":"chatcmpl-mock-xxx","object":"chat.completion.chunk","choices":[{"delta":
{"role":"assistant"},"index":0}]}

data: {"id":"chatcmpl-mock-xxx","object":"chat.completion.chunk","choices":[{"delta":
{"content":"上季度"},"index":0}]}

data: {"id":"chatcmpl-mock-xxx","object":"chat.completion.chunk","choices":[{"delta":
{"content":"销售额总计..."},"index":0}]}

data: {"id":"chatcmpl-mock-xxx","object":"chat.completion.chunk","choices":[{"delta":
```

```
{}, "index": 0, "finish_reason": "stop"]}]}
```

```
data: [DONE]
```

预置 mock 内容时若包含 `\n` 分隔的多行，Mock Service 自动按行拆分为多个 SSE chunk，模拟真实流式输出；每个 chunk 之间插入 `5ms` 延迟（可通过 `MOCK_CHUNK_DELAY_MS` 环境变量覆盖）。

9.3.2 Mock 数据管理接口（Admin 专用）

写入预设响应

```
POST /mock/responses

Authorization: Bearer <admin-token>
Content-Type: application/json

{
  "match": "请分析上季度销售数据",    // 原始字符串（服务端计算 SHA-256 key）
  "response": "上季度总销售额 1.25 亿元，同比增长 12.3%，...",
  "stream_chunks": null,              // 可选：显式指定 SSE 分块列表（覆盖自动拆行）
  "ttl_seconds": 3600                  // 可选，0 = 永不过期
}

Response:
{
  "key": "mock:resp:alb2c3d4e5f6a7b8",
  "queue_length": 1,
  "match_preview": "请分析上季度销售数据"
}
```

批量写入（同一个 match 对应多轮响应）：

```
POST /mock/responses

{
  "match": "请分析上季度销售数据",
  "responses": [                      // 使用 responses（复数）即可写入多条
    "第一次回答：...",
    "第二次回答：..."
  ]
}
```

查看队列（不消耗）

```
GET /mock/responses/:key

// key 为 SHA-256 hex[:16], 或直接传原始文本（服务端转换）

Response:

{
  "key": "mock:resp:alb2c3d4e5f6a7b8",
  "match_preview": "请分析上季度销售数据",
  "queue_length": 2,
  "items": [
    "第一次回答: ...",
    "第二次回答: ..."
  ]
}
```

列出所有 Mock Key

```
GET /mock/responses

Response:

{
  "total_keys": 3,
  "keys": [
    {
      "key": "mock:resp:alb2c3d4e5f6a7b8",
      "preview": "请分析上季度销售数据",
      "queue_length": 2
    },
    {
      "key": "mock:resp:*",
      "preview": "(通配符 fallback)",
      "queue_length": 1
    }
  ]
}
```

清空指定 key

```
DELETE /mock/responses/:key

Response: {"deleted": true, "key": "mock:resp:alb2c3d4e5f6a7b8"}
```

清空所有

```
DELETE /mock/responses
```

```
Response: {"deleted_keys": 3}
```

9.4 Go 实现骨架

```
// internal/mockllm/service.go
package mockllm

import (
    "context"
    "crypto/sha256"
    "encoding/hex"
    "encoding/json"
    "fmt"
    "strings"
    "time"

    "github.com/redis/go-redis/v9"
    "go.uber.org/zap"
)

const (
    keyPrefix      = "mock:resp:"
    wildcardKey    = "mock:resp:*"
    defaultReply   = "这是一条 Mock LLM 默认响应。请先通过管理接口注入测试数据。"
    chunkDelayMs   = 5
)

// MockLLMService 兼容 OpenAI Chat Completions 协议的 Mock 服务
type MockLLMService struct {
    rdb      *redis.Client
    logger   *zap.Logger
}

// LookupKey 根据 messages 最后一条 content 生成 Redis key
func (s *MockLLMService) LookupKey(messages []ChatMessage) string {
    if len(messages) == 0 {
        return wildcardKey
    }
}
```

```

    }

    last := messages[len(messages)-1].Content
    sum := sha256.Sum256([]byte(last))
    return keyPrefix + hex.EncodeToString(sum[:])[0:16]
}

// PopResponse 从队列弹出一条预设响应
func (s *MockLLMService) PopResponse(ctx context.Context, key string) (string, error) {
    s.logger.Debug("[Mock] PopResponse attempt",
        zap.String("key", key),
    )

    // 优先精确 key
    val, err := s.rdb.LPop(ctx, key).Result()
    if err == nil {
        s.logger.Debug("[Mock] hit exact key", zap.String("key", key),
            zap.String("preview", truncate(val, 80)))
        return val, nil
    }

    // fallback 通配符
    val, err = s.rdb.LPop(ctx, wildcardKey).Result()
    if err == nil {
        s.logger.Debug("[Mock] hit wildcard fallback", zap.String("preview", truncate(val,
80)))
        return val, nil
    }

    // 默认响应
    s.logger.Warn("[Mock] queue empty, using default response", zap.String("key", key))
    return defaultReply, nil
}

// Push 写入预设响应（管理接口调用）
func (s *MockLLMService) Push(ctx context.Context, match string, responses []string, ttl
time.Duration) (string, error) {
    sum := sha256.Sum256([]byte(match))
    key := keyPrefix + hex.EncodeToString(sum[:])[0:16]

```

```

    vals := make([]interface{}, len(responses))

    for i, r := range responses {
        vals[i] = r
    }

    if err := s.rdb.RPush(ctx, key, vals...).Err(); err != nil {
        return "", fmt.Errorf("push mock response: %w", err)
    }

    if ttl > 0 {
        s.rdb.Expire(ctx, key, ttl)
    }

    s.logger.Info("[Mock] pushed responses",
        zap.String("key", key),
        zap.String("match_preview", truncate(match, 60)),
        zap.Int("count", len(responses)),
    )

    return key, nil
}

func truncate(s string, n int) string {
    if len([]rune(s)) <= n { return s }

    return string([]rune(s)[:n]) + "..."
}

```

```

// internal/mockllm/handler.go
package mockllm

import (
    "bufio"
    "fmt"
    "net/http"
    "strings"
    "time"

    "github.com/gin-gonic/gin"
    "go.uber.org/zap"
)

// ChatCompletionsHandler 处理 POST /mock/v1/chat/completions

```

```

func (s *MockLLMService) ChatCompletionsHandler(c *gin.Context) {
    var req ChatCompletionRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    key := s.LookupKey(req.Messages)
    s.logger.Debug("[Mock] ChatCompletions request",
        zap.String("model", req.Model),
        zap.Bool("stream", req.Stream),
        zap.Int("messages_count", len(req.Messages)),
        zap.String("last_content_preview", truncate(lastContent(req.Messages), 60)),
        zap.String("lookup_key", key),
    )

    content, _ := s.PopResponse(c.Request.Context(), key)

    if req.Stream {
        s.writeSSE(c, content)
    } else {
        s.writeCompletion(c, content)
    }
}

// writeSSE 按行拆分 content, 逐 chunk 输出 SSE
func (s *MockLLMService) writeSSE(c *gin.Context, content string) {
    id := fmt.Sprintf("chatcmpl-mock-%d", time.Now().UnixNano())
    delay := chunkDelayMs * time.Millisecond

    c.Header("Content-Type", "text/event-stream")
    c.Header("Cache-Control", "no-cache")
    c.Header("X-Accel-Buffering", "no")
    c.Status(http.StatusOK)

    flusher, _ := c.Writer.(http.Flusher)

    // 首包: role
    writeChunk(c.Writer, flusher, id, `{"role":"assistant"}`)
}

```



```

time.Sleep(delay)

// 按行拆分正文
scanner := bufio.NewScanner(strings.NewReader(content))
first := true
for scanner.Scan() {
    line := scanner.Text()
    if !first {
        writeChunk(c.Writer, flusher, id, `{"content":"\n"}`)
        time.Sleep(delay)
    }
    // 按字符拆分, 模拟真实 token 流 (每次最多 8 字符)
    runes := []rune(line)
    for i := 0; i < len(runes); i += 8 {
        end := i + 8
        if end > len(runes) { end = len(runes) }
        chunk := string(runes[i:end])
        escaped, _ := json.Marshal(chunk)
        writeChunk(c.Writer, flusher, id, fmt.Sprintf(`{"content":%s}`, escaped))
        time.Sleep(delay)
    }
    first = false
}

// 结束包
fmt.Fprintf(c.Writer, "data:
{"id\":"%s","\nobject\":"chat.completion.chunk","\nchoices\":[{"delta\":"
{"index\":"0","\nfinish_reason\":"stop"}]}\n\n", id)
fmt.Fprintf(c.Writer, "data: [DONE]\n\n")
if flusher != nil { flusher.Flush() }

s.logger.Debug("[Mock] SSE stream completed", zap.String("id", id))
}

func writeChunk(w http.ResponseWriter, f http.Flusher, id, delta string) {
    fmt.Fprintf(w, "data: {"id\":"%s","\nobject\":"chat.completion.chunk","\nchoices\":"
[{"delta\":"%s","\nindex\":"0"}]}\n\n", id, delta)
    if f != nil { f.Flush() }
}

```

```
// internal/mockllm/routes.go - 路由注册 (仅 dev/test 环境)

func RegisterMockRoutes(r *gin.Engine, svc *MockLLMService) {
    // OpenAI-compatible 接口

    mock := r.Group("/mock")

    mock.POST("/v1/chat/completions", svc.ChatCompletionsHandler)

    // 管理接口 (需要 Admin 角色)

    admin := mock.Group("/responses", middleware.RequireRole("admin"))

    admin.POST("",          svc.PushHandler)
    admin.GET("",           svc.ListHandler)
    admin.GET("/:key",      svc.GetQueueHandler)
    admin.DELETE("",        svc.ClearAllHandler)
    admin.DELETE("/:key",   svc.ClearKeyHandler)
}

// cmd/server/main.go - 条件挂载
if cfg.AppEnv == "development" || cfg.AppEnv == "test" {
    mockSvc := mockllm.New(redisClient, logger)
    mockllm.RegisterMockRoutes(router, mockSvc)
    logger.Info("Mock LLM Service enabled", zap.String("env", cfg.AppEnv))
}
```

LLM Router 集成：在 `internal/llm/router.go` 中，当环境变量 `LLM_BASE_URL` 指向本地 Mock（如 `http://localhost:8080/mock/v1`）时，Agent Engine 完全透明地使用 Mock——无需改动 Agent 逻辑代码。

9.5 Docker Compose 集成

Mock Service 内嵌在主 `api` 容器中，无需额外容器。通过环境变量控制：

```
# docker-compose.yml (api service 环境变量)

environment:
    APP_ENV: ${APP_ENV:-development}          # development | test | production
    MOCK_CHUNK_DELAY_MS: ${MOCK_CHUNK_DELAY_MS:-5}
    MOCK_DEFAULT_REPLY: ${MOCK_DEFAULT_REPLY:-"Mock LLM: 请注入测试数据"}
```

```
# .env 追加
# =====
# Mock LLM Service (仅 dev/test 生效)
# =====
APP_ENV=development
MOCK_CHUNK_DELAY_MS=5
MOCK_DEFAULT_REPLY=Mock LLM default response. Inject test data via /mock/responses API.
```

10. Debug 日志规范

10.1 日志库选型

使用 **uber-go/zap** (结构化高性能日志库, MIT License):

```
// internal/logger/logger.go
package logger

import (
    "os"
    "go.uber.org/zap"
    "go.uber.org/zap/zapcore"
)

func New(level, format string) *zap.Logger {
    var lvl zapcore.Level
    _ = lvl.UnmarshalText([]byte(level)) // "debug" | "info" | "warn" | "error"

    encoderCfg := zap.NewProductionEncoderConfig()
    encoderCfg.TimeKey = "ts"
    encoderCfg.EncodeTime = zapcore.ISO8601TimeEncoder

    var enc zapcore.Encoder
    if format == "console" {
        enc = zapcore.NewConsoleEncoder(encoderCfg)
    } else {
        enc = zapcore.NewJSONEncoder(encoderCfg)
    }
}
```

```
core := zapcore.NewCore(enc, zapcore.AddSync(os.Stdout), lvl)

return zap.New(core, zap.AddCaller(), zap.AddCallerSkip(0))

}
```

环境变量控制：

```
LOG_LEVEL=debug      # debug | info | warn | error (默认 info)
LOG_FORMAT=console   # console (开发友好) | json (生产结构化)
```

10.2 日志字段规范

每条日志必须包含以下**标准字段**（通过中间件自动注入）：

字段	类型	说明
ts	string	ISO-8601 时间戳
level	string	debug / info / warn / error
caller	string	源文件:行号
msg	string	日志主体描述
trace_id	string	请求追踪 ID (UUID, X-Trace-Id header)
user_id	string	当前用户 ID (认证后注入)
session_id	string	会话 ID (适用于 Chat/Agent)
task_id	string	Agent 任务 ID (适用于异步流程)
latency_ms	int	Handler 耗时 (仅请求日志)
component	string	所属组件 (见下表)

component 标准值：

值	说明
handler	HTTP 处理层
agent_service	Agent 执行核心
agent_engine	ADK Agent Engine
skill	Skill 调用
sql_executor	SQL 执行器
worker	异步 Worker
scheduler	定时调度器
mock_llm	Mock LLM Service
llm_router	LLM 路由器
vector_store	Milvus 向量操作
artifact	Artifact 存储
knowledge	知识库
auth	认证鉴权

10.3 关键路径 Debug 日志清单

HTTP 请求层（Middleware）

```
// middleware/logging.go
func LoggingMiddleware(logger *zap.Logger) gin.HandlerFunc {
    return func(c *gin.Context) {
        start := time.Now()
        traceID := uuid.New().String()
        c.Set("trace_id", traceID)
        c.Header("X-Trace-Id", traceID)

        logger.Debug("HTTP request received",
            zap.String("component", "handler"),
            zap.String("trace_id", traceID),
```

```

        zap.String("method", c.Request.Method),
        zap.String("path", c.FullPath()),
        zap.String("client_ip", c.ClientIP()),
        zap.String("user_agent", c.Request.UserAgent()),
        zap.Int64("content_length", c.Request.ContentLength),
    )

    c.Next()

    logger.Info("HTTP request completed",
        zap.String("component", "handler"),
        zap.String("trace_id", traceID),
        zap.String("method", c.Request.Method),
        zap.String("path", c.FullPath()),
        zap.Int("status", c.Writer.Status()),
        zap.Int64("latency_ms", time.Since(start).Milliseconds()),
        zap.Int("response_size", c.Writer.Size()),
    )
}
}

```

Agent Service — 任务全生命周期

```

// Debug 日志点 (internal/service/agent_service.go)

// 1. 任务创建
logger.Debug("[AgentService] task created",
    zap.String("component", "agent_service"),
    zap.String("trace_id", traceID),
    zap.String("task_id", task.ID),
    zap.String("user_id", task.UserID),
    zap.String("mode", task.Mode),           // sync | async
    zap.String("query_preview", truncate(task.Query, 80)),
    zap.Strings("data_sources", task.DataSources),
)

// 2. 入队
logger.Debug("[AgentService] task enqueued to Redis Stream",
    zap.String("component", "agent_service"),

```

```
zap.String("task_id", task.ID),
zap.String("stream_key", "agent:tasks"),
zap.String("msg_id", streamMsgID),
)

// 3. Agent Engine 调用前
logger.Debug("[AgentService] invoking Agent Engine",
zap.String("component", "agent_service"),
zap.String("task_id", task.ID),
zap.String("session_id", sessionID),
zap.String("llm_model", cfg.LLMModel),
zap.Int("context_messages", len(messages)),
)

// 4. Skill 调用
logger.Debug("[AgentService] skill call",
zap.String("component", "skill"),
zap.String("task_id", task.ID),
zap.String("skill_name", skillName),
zap.Any("input_preview", truncateAny(input, 200)),
)

// 5. Skill 返回
logger.Debug("[AgentService] skill result",
zap.String("component", "skill"),
zap.String("task_id", task.ID),
zap.String("skill_name", skillName),
zap.Int64("duration_ms", durationMs),
zap.Bool("success", err == nil),
zap.Any("output_preview", truncateAny(output, 200)),
)

// 6. 任务完成
logger.Info("[AgentService] task completed",
zap.String("component", "agent_service"),
zap.String("task_id", task.ID),
zap.String("status", "completed"),
zap.Int64("total_duration_ms", totalMs),
zap.Int("artifact_count", len(artifacts)),
```

```
)

// 7. 任务失败
logger.Error("[AgentService] task failed",
    zap.String("component", "agent_service"),
    zap.String("task_id", task.ID),
    zap.Error(err),
    zap.Int("retry_count", retryCount),
)
```

SQL 执行器

```
// Debug 日志点 (internal/skill/sql_executor.go)

// 1. AST 解析
logger.Debug("[SQLExecutor] AST parse",
    zap.String("component", "sql_executor"),
    zap.String("task_id", taskID),
    zap.String("raw_sql_preview", truncate(rawSQL, 200)),
    zap.String("stmt_type", stmtType),    // SELECT / DESCRIBE 等
    zap.Bool("allowed", allowed),
)

// 2. 执行前 (包含完整 SQL, DEBUG 级别)
logger.Debug("[SQLExecutor] executing query",
    zap.String("component", "sql_executor"),
    zap.String("task_id", taskID),
    zap.String("data_source", dsName),
    zap.String("sql", rawSQL),            // 完整 SQL, 仅 DEBUG 级别暴露
    zap.Any("params", params),
)

// 3. 执行后
logger.Debug("[SQLExecutor] query result",
    zap.String("component", "sql_executor"),
    zap.String("task_id", taskID),
    zap.Int("row_count", rowCount),
    zap.Int64("query_ms", queryMs),
)
```



```
        zap.Int("col_count", colCount),  
    )  
}
```

Worker

```
// Debug 日志点 (internal/worker/agent_worker.go)  
  
logger.Debug("[Worker] dequeued message",  
    zap.String("component", "worker"),  
    zap.String("stream_msg_id", msgID),  
    zap.String("task_type", taskType),    // "agent" | "scheduled"  
    zap.String("task_id", taskID),  
)  
  
// scheduled → agent 转换  
logger.Debug("[Worker] converting ScheduledTask to AgentTask",  
    zap.String("component", "worker"),  
    zap.String("scheduled_task_id", scheduledID),  
    zap.String("new_agent_task_id", agentTaskID),  
)  
  
logger.Debug("[Worker] re-enqueued as AgentTask",  
    zap.String("component", "worker"),  
    zap.String("agent_task_id", agentTaskID),  
    zap.String("stream_msg_id", newMsgID),  
)
```

LLM Router

```
logger.Debug("[LLMRouter] sending request to LLM",  
    zap.String("component", "llm_router"),  
    zap.String("task_id", taskID),  
    zap.String("base_url", cfg.LLMBaseURL),  
    zap.String("model", model),  
    zap.Bool("stream", stream),  
    zap.Int("messages_count", len(messages)),  
    zap.String("last_msg_preview", truncate(lastMsg, 80)),  
)
```

```
logger.Debug("[LLMRouter] LLM response received",
    zap.String("component", "llm_router"),
    zap.String("task_id", taskID),
    zap.Int64("latency_ms", latencyMs),
    zap.Int("completion_tokens", tokens),
    zap.String("finish_reason", finishReason),
)
```

Mock LLM Service

```
logger.Debug("[Mock] incoming request",
    zap.String("component", "mock_llm"),
    zap.String("model", req.Model),
    zap.Bool("stream", req.Stream),
    zap.Int("msg_count", len(req.Messages)),
    zap.String("last_content", truncate(lastContent(req.Messages), 80)),
    zap.String("lookup_key", key),
)

logger.Debug("[Mock] queue status before pop",
    zap.String("component", "mock_llm"),
    zap.String("key", key),
    zap.Int64("queue_length", queueLen),
)
```

10.4 日志级别使用规范

级别	用途	示例
DEBUG	开发调试，生产默认关闭。包含：完整 SQL、LLM 请求/响应 preview、每步 skill 调用入参出参、Mock 队列状态	<code>[Mock] hit exact key</code> 、 <code>[SQLExecutor] executing query</code>
INFO	关键业务事件，生产默认开启	任务创建/完成、用户登录/登出、定时任务触发
WARN	可恢复异常，需关注但不阻断业务	Mock 队列为空 fallback、重试超时、SQL 行数超限
ERROR	需立即处理的错误，触发告警	任务执行失败、数据库连接断开、Vault 读取失败

 **安全注意：** `DEBUG` 级别日志可能包含完整 SQL 语句和 LLM prompt/response 内容，**生产环境必须设置 `LOG_LEVEL=info`**，避免敏感数据泄漏到日志系统。

10.5 日志采样与聚合（生产建议）

```
# 生产环境日志配置建议 (docker-compose / k8s)

LOG_LEVEL: info

LOG_FORMAT: json           # 结构化 JSON，方便 ELK/Loki 解析

# 采样（高频路径避免日志爆炸）

LOG_SAMPLING_INITIAL: 100   # 每秒前 100 条完整记录

LOG_SAMPLING_THEREAFTER: 10 # 之后每 10 条记录 1 条
```

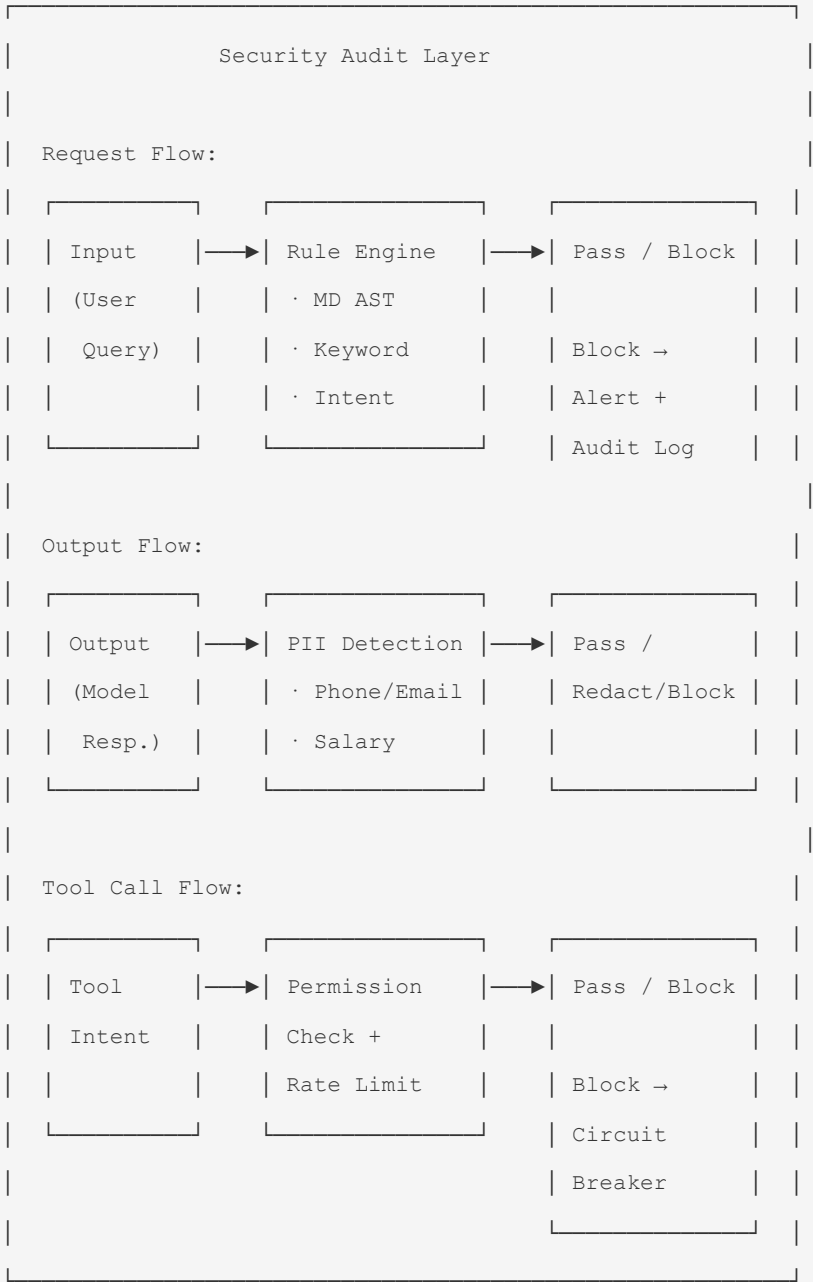
采样实现（zap sampler）：

```
if cfg.LogSamplingInitial > 0 {
    core = zapcore.NewSamplerWithOptions(
        core,
        time.Second,
        cfg.LogSamplingInitial,
        cfg.LogSamplingThereafter,
```

```
)  
  
}
```

11. 安全架构

8.1 安全审查层



安全规则配置:

```
# security_rules.yaml

input_rules:
  - name: "sql_injection"
    patterns:
      - "(?i) (drop\\s+table|delete\\s+from|truncate|alter\\s+table) "
    action: "block"
    severity: "critical"

  - name: "prompt_injection"
    patterns:
      - "(?i) (ignore\\s+(all\\s+)?(previous|above)\\s+(instructions|rules)) "
      - "(?i) (system\\s*prompt) "
    action: "block"
    severity: "high"

  - name: "unauthorized_access"
    patterns:
      - "(?i) (other\\s+users?('|\\s+)?data|salary|personal\\s+info) "
    action: "warn"
    severity: "medium"

output_rules:
  - name: "pii_detection"
    patterns:
      - "\\b1[3-9]\\d{9}\\b" # 手机号
      - "\\b\\d{17}[\\dXx]\\b" # 身份证
    action: "redact"
    severity: "high"

tool_rules:
  - name: "email_whitelist"
    check: "validate_email_domain"
    action: "block"
    severity: "critical"

  - name: "rate_limit"
    check: "check_rate_limit"
```

```
action: "throttle"

severity: "medium"
```

熔断器 (Circuit Breaker):

```
type CircuitBreaker struct {
    failureThreshold int          // 连续失败次数阈值
    successThreshold int          // 恢复所需成功次数
    timeout          time.Duration // 熔断后等待时间
    state            State         // closed | open | half-open
    failures         int
    successes        int
    lastFailure      time.Time
    mu               sync.Mutex
}

func (cb *CircuitBreaker) Execute(fn func() error) error {
    cb.mu.Lock()

    if cb.state == StateOpen {
        if time.Since(cb.lastFailure) < cb.timeout {
            cb.mu.Unlock()
            return ErrCircuitOpen
        }
        cb.state = StateHalfOpen
    }
    cb.mu.Unlock()

    err := fn()

    cb.mu.Lock()
    defer cb.mu.Unlock()
    if err != nil {
        cb.failures++
        cb.lastFailure = time.Now()
        if cb.failures >= cb.failureThreshold {
            cb.state = StateOpen
            // 记录审计日志
            audit.LogSecurityAlert("circuit_breaker_open", ...)
        }
    }
}
```

```

        return err
    }

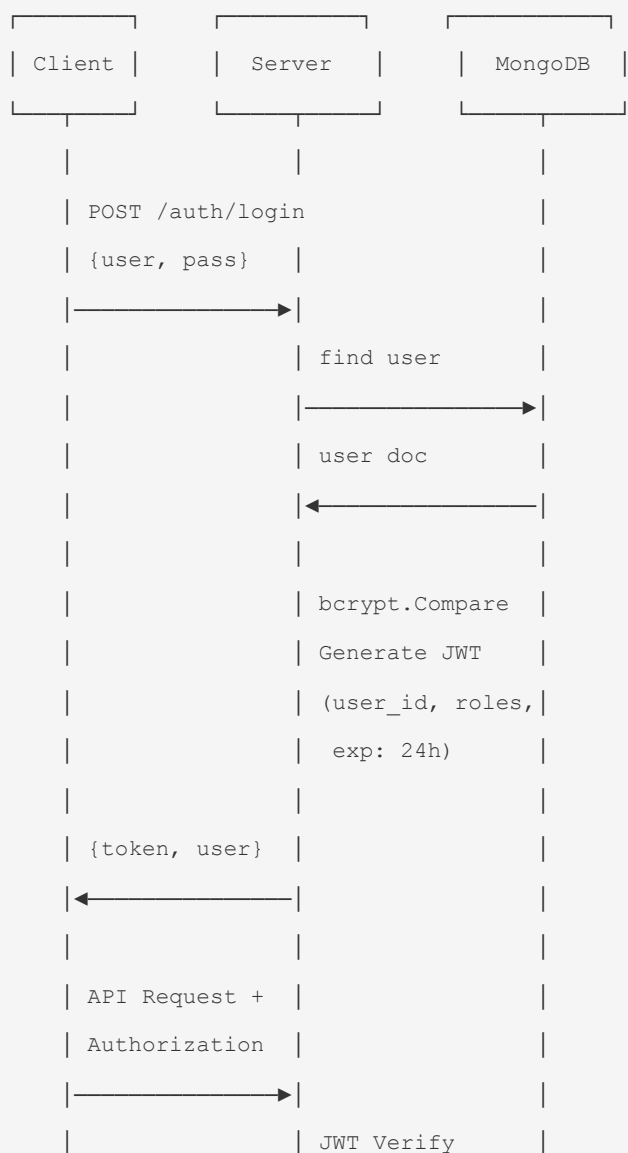
    // success
    cb.successes++

    if cb.state == StateHalfOpen && cb.successes >= cb.successThreshold {
        cb.state = StateClosed
        cb.failures = 0
        cb.successes = 0
    }

    return nil
}

```

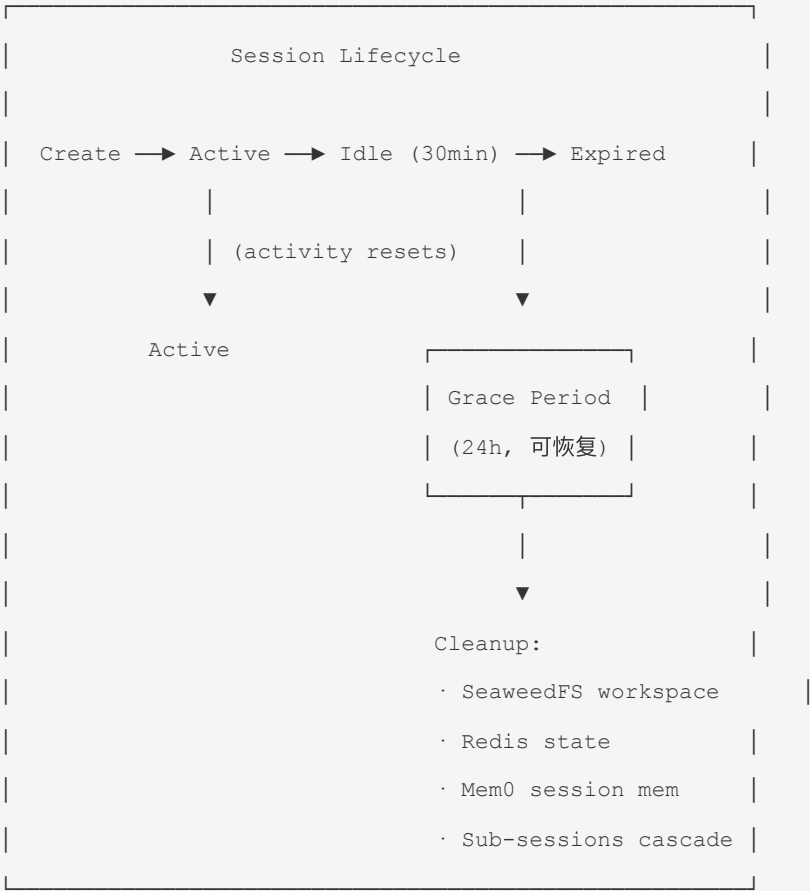
8.2 认证流程



	RBAC Check	

12. 会话与工作区管理

9.1 Session 生命周期



Session 管理器:

```
type SessionManager struct {
    mongo      *MongoClient
    redis      *RedisClient
    seaweedfs  *SeaweedFSClient
    mem0       *Mem0Client
}

func (sm *SessionManager) CreateSession(userID, sessionType string) (*Session, error) {
```



```

workspaceID := uuid.New().String()

// 1. 创建 SeaweedFS workspace bucket
sm.seaweedfs.CreateBucket(ctx, workspaceID)

// 2. 初始化 Mem0 session memory
sm.mem0.CreateSession(ctx, userID, workspaceID)

// 3. 保存 Session 记录到 MongoDB
session := &Session{
    ID:          uuid.New().String(),
    UserID:      userID,
    Type:        sessionType,
    WorkspaceID: workspaceID,
    CreatedAt:   time.Now(),
    ExpiresAt:   time.Now().Add(30 * time.Minute),
}
sm.mongo.Insert("sessions", session)

return session, nil
}

func (sm *SessionManager) CleanupExpiredSessions() {
    sessions := sm.mongo.Find("sessions", bson.M{
        "expires_at": bson.M{"$lt": time.Now().Add(-24 * time.Hour)},
    })
    for _, s := range sessions {
        // 级联清理
        sm.cleanupSession(s)
        // 清理子 session
        subSessions := sm.mongo.Find("sessions", bson.M{
            "parent_session_id": s.ID,
        })
        for _, sub := range subSessions {
            sm.cleanupSession(sub)
        }
    }
}

```

```
func (sm *SessionManager) cleanupSession(s *Session) {
    sm.seaweedfs.DeleteBucket(ctx, s.WorkspaceID) // 清理工作区文件
    sm.mem0.DeleteSession(ctx, s.ID)             // 清理 Mem0 会话记忆
    sm.redis.Delete(ctx, "session:"+s.ID)        // 清理 Redis 缓存
    sm.mongo.Delete("sessions", bson.M{"_id": s.ID}) // 删除记录
}
```

9.2 子 Agent Session 管理

```
// A2A 子 Agent 创建
func (sm *SessionManager) CreateSubSession(parentSessionID string, task AgentInput)
(*Session, error) {
    parent, _ := sm.GetSession(parentSessionID)

    // 继承父 session 的部分上下文
    subSession, _ := sm.CreateSession(parent.UserID, "sub_agent")
    subSession.ParentSessionID = parentSessionID
    subSession.WorkspaceID = parent.WorkspaceID + "/sub_" + subSession.ID

    sm.mongo.Update("sessions", bson.M{"_id": subSession.ID}, subSession)

    return subSession, nil
}
```

12.3 本地文件系统工作区 (Workspace)

每个 Agent Session 拥有一个**独立的本地文件系统工作区**，生命周期与 Session 完全一致（创建时分配，销毁时级联清理）。工作区用于存放 Agent 执行过程中需要操作的文件：

- 临时脚本（Shell、SQL 片段等）
- 生成的图表、图片
- 生成的 PPT/Word/PDF 文档
- 数据分析中间结果文件
- 其他需要文件系统交互的临时产物

与 SeaweedFS Workspace 的关系：本地 Workspace（`/workspaces/<session_id>/`）用于运行时文件操作和脚本执行；SeaweedFS Workspace（`workspaces/` bucket）用于持久化存储跨步骤共享的数据文件。两者互补：本地 Workspace 满足 POSIX 文件系统需求（脚本执行、工具链），SeaweedFS 满足持久化和跨副本共享需求。

目录结构

```

/workspaces/                                # 宿主机挂载点（可配置 WORKSPACE_ROOT）
├── <session_id_a>/                          # Session A 的工作区
│   ├── scripts/                            # 临时脚本
│   │   └── analysis.sh
│   ├── output/                            # 生成产出
│   │   ├── chart.png
│   │   └── report.docx
│   └── data_export.xlsx
├── tmp/                                    # 临时文件
├── <session_id_b>/                          # Session B 的工作区（完全隔离）
├── ...
└── _cleanup/                              # 待清理目录（异步 GC）

```

安全隔离原则（三层防护）

Layer 1 - Docker 层：容器以 appuser（非 root）运行

```

├── /workspaces/ → rw-x（唯一可读、写、执行的用户目录）
├── /app/         → r-x（只读，无写权限；server 需要 x 启动）
├── /usr/bin 等   → r-x（系统工具如 bash 需要 x，但属 root，不可写）
└── 其他系统路径 → ---（无任何权限）

```

Layer 2 - Skill 网关层：禁止任何代码直接操作文件系统

```

├── 所有文件读写 → workspace_read / workspace_write Skill
└── 所有脚本执行 → workspace_exec Skill（含安全审计）

```

Layer 3 - 运行时审计层：

```

├── 路径沙箱：resolved_path 必须在 /workspaces/<session_id>/ 内
├── 脚本白名单：禁止 rm -rf、curl/wget、nc、chmod 777 等
└── 资源限制：单文件 ≤ 100MB，总目录 ≤ WORKSPACE_MAX_SIZE_MB

```

12.3.1 文件读写 Skill（workspace_read / workspace_write）

文件操作必须通过 Skill 接口，禁止 Handler/Service/Agent Engine 直接调用 `os.Open` / `os.WriteFile`。

`workspace_read` Skill:

```
// internal/skill/workspace_read.go

type WorkspaceReadSkill struct {
    maxFileSizeMB int64    // 默认: 50
    // workspaceRoot 从 SkillContext 获取, 不存储为字段
}

// Execute 使用 SkillContext - session_id 自动注入, 参数中不可见
func (s *WorkspaceReadSkill) Execute(ctx context.Context, sc SkillContext, params
map[string]any) (any, error) {
    path, _ := params["path"].(string)

    logger.Debug("[WorkspaceRead] executing",
        zap.String("component", "skill"),
        zap.String("skill_name", "workspace_read"),
        zap.String("session_id", sc.SessionID),
        zap.String("path", path),
    )

    // 路径安全解析 - session 从 SkillContext 获取, 不接受外部传入
    safePath, err := s.resolveSafePath(sc.SessionID, path)
    if err != nil {
        return nil, fmt.Errorf("workspace_read: path not allowed: %w", err)
    }

    // 大小检查
    fi, err := os.Stat(safePath)
    if err != nil {
        return nil, fmt.Errorf("workspace_read: stat failed: %w", err)
    }
    if fi.Size() > s.maxFileSizeMB*1024*1024 {
        return nil, fmt.Errorf("workspace_read: file too large (%d MB > %d MB max)",
fi.Size()/1024/1024, s.maxFileSizeMB)
    }

    // 读取
    content, err := os.ReadFile(safePath)
    if err != nil {
        return nil, fmt.Errorf("workspace_read: read failed: %w", err)
    }
}
```

```

    }

    return &WorkspaceReadOutput{
        Path:      path,
        Content:    content,
        ContentType: mime.TypeByExtension(filepath.Ext(safePath)),
        Size:       fi.Size(),
    }, nil
}

// resolveSafePath 确保解析后路径在 session workspace 内 - sessionID 来自 SkillContext
func (s *WorkspaceReadSkill) resolveSafePath(sessionID, relPath string) (string, error) {
    // 禁止绝对路径
    if filepath.IsAbs(relPath) {
        return "", fmt.Errorf("absolute path not allowed: %s", relPath)
    }
    // 清理并拼接
    clean := filepath.Clean(relPath)
    full := filepath.Join("/workspaces", sessionID, clean)

    // 二次校验: 解析真实路径后必须仍在 /workspaces/<session_id>/ 内
    realPath, err := filepath.EvalSymlinks(full)
    if err != nil {
        return "", fmt.Errorf("cannot resolve path: %w", err)
    }
    prefix := filepath.Join("/workspaces", sessionID) + "/"
    if !strings.HasPrefix(realPath, prefix) {
        return "", fmt.Errorf("path escape attempt blocked: %s → %s", relPath, realPath)
    }
    return realPath, nil
}

```

workspace_write Skill:

```

// internal/skill/workspace_write.go
type WorkspaceWriteSkill struct {
    maxFileSizeMB int64 // 默认: 100
}

```

```

func (s *WorkspaceWriteSkill) Execute(ctx context.Context, sc SkillContext, params
map[string]any) (any, error) {
    path, _ := params["path"].(string)
    content, _ := params["content"].([]byte)
    overwrite, _ := params["overwrite"].(bool)

    logger.Debug("[WorkspaceWrite] writing file",
        zap.String("component", "skill"),
        zap.String("skill_name", "workspace_write"),
        zap.String("session_id", sc.SessionID),
        zap.String("path", path),
        zap.Int("size_bytes", len(content)),
    )

    safePath, err := resolveSafePath(sc.SessionID, path)
    if err != nil {
        return nil, fmt.Errorf("workspace_write: %w", err)
    }

    if len(content) > int(s.maxFileSizeMB*1024*1024) {
        return nil, fmt.Errorf("workspace_write: content too large (%d MB > %d MB max)",
len(content)/1024/1024, s.maxFileSizeMB)
    }

    // 确保父目录存在
    os.MkdirAll(filepath.Dir(safePath), 0o750)

    flag := os.O_WRONLY | os.O_CREATE | os.O_TRUNC
    if !overwrite {
        flag |= os.O_EXCL
    }

    f, err := os.OpenFile(safePath, flag, 0o640)
    if err != nil {
        return nil, fmt.Errorf("workspace_write: open failed: %w", err)
    }

    defer f.Close()

    if _, err := f.Write(content); err != nil {
        return nil, fmt.Errorf("workspace_write: write failed: %w", err)
    }
}

```

```
return map[string]any{"path": path, "size_bytes": len(content)}, nil
}
```

12.3.2 脚本执行 Skill (workspace_exec) — 含安全审计

脚本执行通过 `workspace_exec` Skill 统一入口，执行前必须通过安全审计。

安全审计规则（强制，不可绕过）：

审计维度	规则	拒绝原因
路径逃逸检测	<code>resolved_script_path</code> 必须在 <code>/workspaces/<session_id>/</code> 内	防止读写其他 session 或系统文件
网络请求禁止	拦截 <code>curl</code> 、 <code>wget</code> 、 <code>nc</code> 、 <code>socat</code> 、 <code>telnet</code> 、 <code>ssh</code> 、 <code>python</code> <code>-m http</code> 、 <code>go run</code>	防止脚本访问外部网络
危险命令拦截	<code>rm -rf /</code> 、 <code>chmod 777</code> 、 <code>chown</code> 、 <code>mount</code> 、 <code>mkfs</code> 、 <code>dd</code> <code>if=</code> 、 <code>> /dev/sda</code>	防止破坏容器或宿主机
进程 fork 限制	禁止 <code>fork()</code> 炸弹特征 (<code>() { : :;& };;</code>)	防止资源耗尽
超时限制	单次脚本执行 ≤ 30 秒	防止长时间阻塞 Worker
输出大小限制	<code>stdout+stderr</code> ≤ 10 MB	防止内存溢出

```
// internal/skill/workspace_exec.go

type WorkspaceExecSkill struct {
    timeout          time.Duration // 默认 30s
    maxOutputBytes  int64         // 默认 10 MB
    audit            *ExecAuditor
    // workspaceRoot 从 SkillContext 获取，不存储
}

func (s *WorkspaceExecSkill) Execute(ctx context.Context, sc SkillContext, params
map[string]any) (any, error) {
    script, _ := params["script"].(string)
    interpreter, _ := params["interpreter"].(string)
    workDir, _ := params["work_dir"].(string)
    if workDir == "" { workDir = "." }
}
```

```

logger.Debug("[WorkspaceExec] audit+exec",
    zap.String("component", "skill"),
    zap.String("skill_name", "workspace_exec"),
    zap.String("session_id", sc.SessionID),
    zap.String("interpreter", interpreter),
    zap.Int("script_bytes", len(script)),
)

// Step 1: 安全审计
if err := s.audit.Audit(script); err != nil {
    logger.Warn("[WorkspaceExec] audit blocked",
        zap.String("session_id", sc.SessionID),
        zap.Error(err),
    )
    return nil, fmt.Errorf("workspace_exec: security audit failed: %w", err)
}

// Step 2: 写脚本到 workspace (sc.SessionID 来自 SkillContext, 不接受 params 传入)
scriptPath := filepath.Join("/workspaces", sc.SessionID, "scripts",
fmt.Sprintf("exec_%d.sh", time.Now().UnixNano()))
safePath, err := resolveSafePath(sc.SessionID, scriptPath)
if err != nil {
    return nil, err
}
if err := os.WriteFile(safePath, []byte(script), 0o700); err != nil {
    return nil, fmt.Errorf("workspace_exec: write script failed: %w", err)
}

// Step 3: 构建命令 (限制可用解释器)
var cmd *exec.Cmd
workDirFull := filepath.Join("/workspaces", sc.SessionID, workDir)
switch interpreter {
case "bash":
    cmd = exec.CommandContext(ctx, "bash", safePath)
case "sh":
    cmd = exec.CommandContext(ctx, "sh", safePath)
default:
    return nil, fmt.Errorf("workspace_exec: unsupported interpreter: %s", interpreter)
}

```



```
}

cmd.Dir = workDirFull
cmd.Env = []string{
    "HOME=/workspaces/" + sc.SessionID,
    "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
    "TMPDIR=/workspaces/" + sc.SessionID + "/tmp",
}

// Step 4: 超时执行
timeoutCtx, cancel := context.WithTimeout(ctx, s.timeout)
defer cancel()

var stdout, stderr bytes.Buffer
cmd.Stdout = io.MultiWriter(&stdout, limitWriter{max: s.maxOutputBytes})
cmd.Stderr = io.MultiWriter(&stderr, limitWriter{max: s.maxOutputBytes})

start := time.Now()
err = cmd.Run()
elapsed := time.Since(start)

output := &ExecOutput{
    ExitCode: 0,
    Stdout:   stdout.String(),
    Stderr:   stderr.String(),
    Duration: elapsed,
}

if err != nil {
    if exitErr, ok := err.(*exec.ExitError); ok {
        output.ExitCode = exitErr.ExitCode()
    } else {
        return nil, fmt.Errorf("workspace_exec: execution error: %w", err)
    }
}

logger.Debug("[WorkspaceExec] completed",
    zap.String("component", "skill"),
    zap.String("session_id", sc.SessionID),
    zap.Int("exit_code", output.ExitCode),
    zap.Int64("duration_ms", elapsed.Milliseconds()),
```

```

    )

    return output, nil
}

type ExecOutput struct {
    ExitCode int          `json:"exit_code"`
    Stdout   string              `json:"stdout"`
    Stderr   string              `json:"stderr"`
    Duration time.Duration `json:"duration"`
}

```

12.3.3 Workspace 生命周期管理

```

// internal/session/workspace.go

type WorkspaceManager struct {
    root      string    // /workspaces
    maxSizeMB int64
}

// Create 在 Session 创建时调用
func (wm *WorkspaceManager) Create(ctx context.Context, sessionID string) (string, error) {
    wsPath := filepath.Join(wm.root, sessionID)
    dirs := []string{
        filepath.Join(wsPath, "scripts"),
        filepath.Join(wsPath, "output"),
        filepath.Join(wsPath, "tmp"),
    }
    for _, d := range dirs {
        if err := os.MkdirAll(d, 0o750); err != nil {
            return "", fmt.Errorf("create workspace dirs: %w", err)
        }
    }
    logger.Info("[Workspace] created", zap.String("session_id", sessionID),
        zap.String("path", wsPath))
    return wsPath, nil
}

// Destroy 在 Session 过期/清理时调用

```

```

func (wm *WorkspaceManager) Destroy(ctx context.Context, sessionID string) error {
    wsPath := filepath.Join(wm.root, sessionID)
    logger.Info("[Workspace] destroying", zap.String("session_id", sessionID))

    // 移到 _cleanup 目录异步删除（避免阻塞）
    cleanupDir := filepath.Join(wm.root, "_cleanup", sessionID)
    if err := os.Rename(wsPath, cleanupDir); err != nil {
        return err
    }

    go func() {
        os.RemoveAll(cleanupDir)
        logger.Debug("[Workspace] cleanup completed", zap.String("session_id", sessionID))
    }()

    return nil
}

// EnforceSizeLimit 周期性执行，清理超限 workspace
func (wm *WorkspaceManager) EnforceSizeLimit(ctx context.Context) {
    sessions, _ := os.ReadDir(wm.root)
    for _, s := range sessions {
        if !s.IsDir() || strings.HasPrefix(s.Name(), "_") {
            continue
        }

        size, _ := dirSize(filepath.Join(wm.root, s.Name()))
        if size > wm.maxSizeMB*1024*1024 {
            logger.Warn("[Workspace] size limit exceeded, cleaning up",
                zap.String("session_id", s.Name()),
                zap.Int64("size_mb", size/1024/1024),
            )
            wm.Destroy(ctx, s.Name())
        }
    }
}

```

13. Artifact 管理 (基于 SeaweedFS)

10.1 概述

Artifact 是 Agent 在执行分析任务过程中生成的各类产出物：图表、导出数据文件、报告附件、中间结果等。所有 Artifact 统一存储在 SeaweedFS 中，通过 MongoDB 管理元数据。

设计原则：

- **与 Session Workspace 分离：**Artifact 有独立的 SeaweedFS volume，不与 Session 临时文件混存
- **生命周期独立：**Artifact 的清理策略不同于 Session workspace：默认跟随 Session 清理，但用户可标记为「持久化」长期保留
- **Agent 透明：**Agent 通过 Skill 自动将产出物保存为 Artifact，无需感知底层存储

10.2 SeaweedFS 存储设计

```
SeaweedFS Buckets:
├─ workspaces/           # Session 临时工作区 (已有)
│   ├─ ws_a1/
│   ├─ ws_a2/
│   └─ ...
└─
└─ artifacts/           # Artifact 持久存储 (新增)
    ├─ {task_id}/
    │   ├─ chart_revenue_trend.png
    │   ├─ export_top10.csv
    │   └─ report_q2_analysis.pdf
    └─ ...
```

存储路径规则： `artifacts/{task_id}/{artifact_id}.{ext}`

- `task_id`：关联的 Agent 任务 ID（保证同一任务的所有 Artifact 在同一目录下）
- `artifact_id`：Artifact 唯一标识（UUID）
- `ext`：文件扩展名

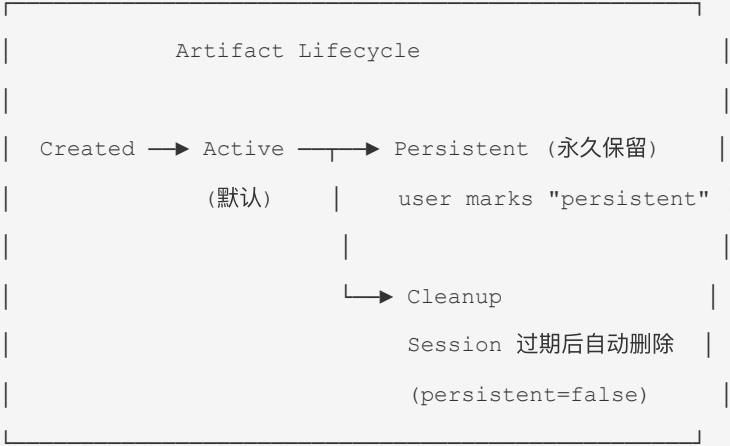
10.3 数据模型

MongoDB `artifacts` 集合：

```
{
  "_id": "artifact_001",
  "task_id": "task_001",
  "session_id": "session_agent_001",
```

```
"user_id": "user_001",
"name": "华东区销售额趋势图",
"type": "chart",           // chart | export | report | interim | screenshot
"mime_type": "image/png",
"size_bytes": 245760,
"storage_path": "artifacts/task_001/artifact_001.png",
"persistent": false,      // 用户是否标记为持久化（不受 Session 清理）
"metadata": {
  "chart_type": "line",
  "dimensions": "1920x1080",
  "generated_by": "stats_engine_skill",
  "generation_duration_ms": 3200
},
"created_at": "2026-07-01T10:30:00Z"
}
```

10.4 Artifact 生命周期管理



清理策略:

条件	行为
<code>persistent = false</code> + Session 过期	跟随 Session 清理，删除 SeaweedFS 文件和 MongoDB 记录
<code>persistent = true</code>	不受 Session 清理影响，永久保留
手动删除	用户主动删除 Artifact（需确认），同时清理 SeaweedFS 和 MongoDB
批量清理	管理员可按时间范围/任务批量清理非持久化 Artifact

Session 清理联动:

```
func (sm *SessionManager) cleanupArtifacts(sessionID string) {
    // 删除该 session 下所有非持久化的 artifact
    artifacts := sm.mongo.Find("artifacts", bson.M{
        "session_id": sessionID,
        "persistent": false,
    })
    for _, art := range artifacts {
        sm.seaweedfs.DeleteObject(ctx, "artifacts", art.StoragePath)
        sm.mongo.Delete("artifacts", bson.M{"_id": art.ID})
    }
}
```

13.5 Artifact Skill (`save_artifact`)

Agent 通过调用 `save_artifact` Skill 将生成的图表、数据文件等保存为 Artifact。底层复用 `ArtifactLogic` (§4.3 Logic 层) 实现，与 `save_report` Skill 共用同一 SeaweedFS 上传 + MongoDB 写入逻辑：

```
// internal/skill/save_artifact.go

type SaveArtifactSkill struct {
    logic *logicC.ArtifactLogic // ← 共用 Logic 层
}

func (s *SaveArtifactSkill) Name() string { return "save_artifact" }

func (s *SaveArtifactSkill) Execute(ctx context.Context, sc SkillContext, params
```

```
map[string]any) (any, error) {  
    name, _ := params["name"].(string)  
    content, _ := params["content"].([]byte) // 原始字节  
    mimeType, _ := params["mime_type"].(string)  
  
    // 幂等创建 (通过 ArtifactLogic)  
    artifact, err := s.logic.CreateArtifact(ctx, sc, logic.CreateArtifactInput{  
        Name:      name,  
        Content:   content,  
        MimeType:  mimeType,  
    })  
    if err != nil {  
        return nil, err  
    }  
  
    return map[string]any{  
        "artifact_id": artifact.ID,  
        "download_url": fmt.Sprintf("/api/v1/artifacts/%s/download", artifact.ID),  
        "is_new":      true, // ArtifactLogic 内部已做幂等处理  
    }, nil  
}
```

13.6 Artifact API

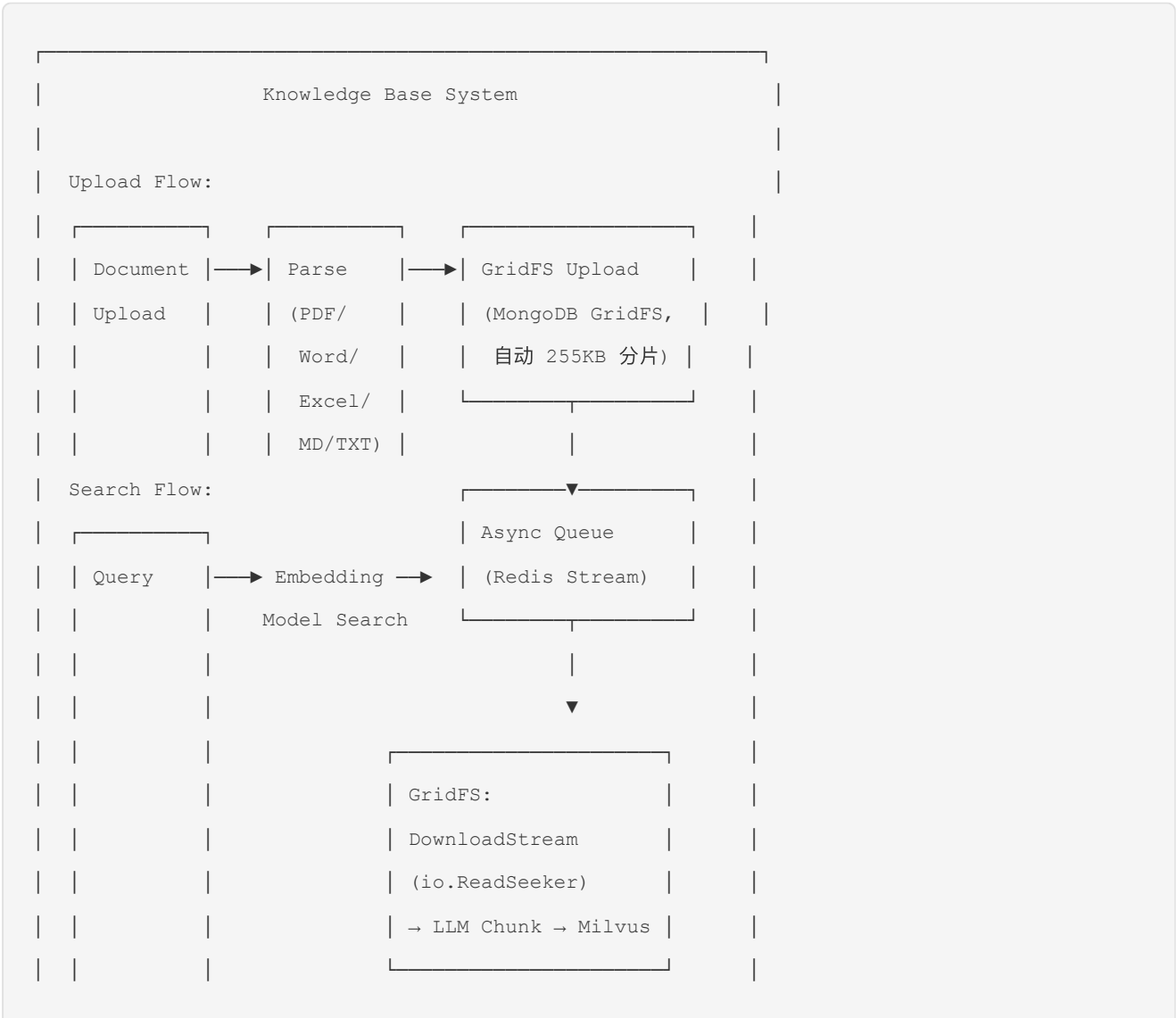
Method	Path	Description
GET	/api/v1/artifacts?task_id=xxx	获取 Artifact 列表 (按任务筛选)
GET	/api/v1/artifacts/:id	获取 Artifact 元数据
GET	/api/v1/artifacts/:id/download	下载 Artifact 原始文件 (SeaweedFS presigned URL)
GET	/api/v1/artifacts/:id/preview	获取 Artifact 预览 (图片直接返回, CSV 返回前 N 行)
POST	/api/v1/artifacts/:id/persist	标记为持久化
POST	/api/v1/artifacts/:id/unpersist	取消持久化标记
DELETE	/api/v1/artifacts/:id	删除 Artifact (同时清理 SeaweedFS)

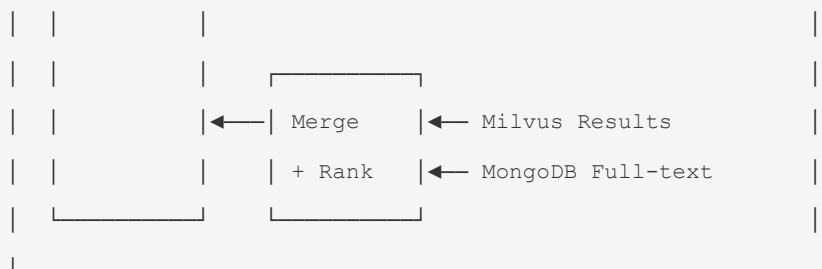
10.7 预览支持

文件类型	预览方式	限制
PNG / JPEG / SVG / GIF	直接返回文件流	单文件 ≤ 10MB
CSV	返回前 100 行 JSON	单文件 ≤ 50MB
PDF	返回文件流（浏览器内置预览）	单文件 ≤ 20MB
其他类型	显示文件信息，提供下载链接	-

14. 知识库设计

11.1 知识库架构





文档处理流程:

```

type KnowledgeService struct {
    mongo    *MongoClient
    gridfs   *gridfs.Bucket // mongo.GridFSBucket
    milvus   *MilvusClient
    redis    *RedisClient
}

func (s *KnowledgeService) UploadDocument(userID string, file io.Reader, filename string,
metadata DocMetadata) (*Document, error) {
    // 1. 解析文档
    content, err := s.parseDocument(file, filename)
    if err != nil { return nil, err }
    contentBytes := []byte(content)

    doc := &Document{
        ID:      uuid.New().String(),
        Title:   metadata.Title,
        Type:    metadata.Type,
        Tags:    metadata.Tags,
        UploadBy: userID,
        Status:  "processing",
    }

    // 2. 统一存入 GridFS (小文件和大文件同一路径, GridFS 自动 255KB 分片)
    uploadOpts := options.GridFSUpload().SetMetadata(bson.M{
        "doc_id": doc.ID,
        "filename": filename,
    })

    fileId, err := s.gridfs.UploadFromStream(filename, bytes.NewReader(contentBytes),
uploadOpts)

```

```

    if err != nil { return nil, fmt.Errorf("gridfs upload: %w", err) }

    // 3. 保存元数据
    s.mongo.Insert("knowledge_doc_contents", bson.M{
        "_id":          uuid.New().String(),
        "doc_id":        doc.ID,
        "gridfs_file_id": fileID,
        "filename":      filename,
        "size_bytes":    len(contentBytes),
        "created_at":    time.Now(),
    })
    s.mongo.Insert("knowledge_docs", doc)

    // 4. 异步索引
    s.redis.XAdd(ctx, "knowledge:index:queue", map[string]any{
        "doc_id": doc.ID,
    })
    return doc, nil
}

```

```

func (s *KnowledgeService) IndexWorker() { for { msgs := s.redis.XReadGroup(ctx,
"knowledge:index:queue", "indexer") for _, msg := range msgs { docID := msg.Values["doc_id"]
s.indexDocument(docID.(string)) s.redis.XAck(ctx, "knowledge:index:queue", "indexer", msg.ID) } } }

```

```

func (s *KnowledgeService) indexDocument(docID string) { doc :=
s.mongo.FindOne("knowledge_docs", bson.M{"_id": docID}) docContent :=
s.mongo.FindOne("knowledge_doc_contents", bson.M{"doc_id": docID})

```

```

// 统一从 GridFS 读取, DownloadStream 实现 io.ReadSeeker, LLM 分片器直接 seek
stream, err := s.gridfs.OpenDownloadStream(docContent.GridFSFileID)
if err != nil { /* handle */ }
defer stream.Close()

// 1. LLM 语义分片 (直接操作 io.ReadSeeker, 不占内存)
chunks := s.llmChunker.Split(stream, docContent.SizeBytes)

// 2. 向量化 + 写入 Milvus
for i, chunk := range chunks {
    embedding := s.embedModel.Embed(chunk)
    s.milvus.Insert("knowledge_chunks", MilvusEntity{

```

```

        Vector:      embedding,
        DocID:       docID,
        ChunkIndex: i,
        Title:       doc.Title,
        Tags:        doc.Tags,
    })
}

// 3. 更新状态
s.mongo.Update("knowledge_docs", bson.M{"_id": docID}, bson.M{
    "$set": bson.M{"status": "indexed", "chunk_count": len(chunks)},
})

```

```

}

```

11.2 混合搜索策略

```

```go
func (s *KnowledgeService) Search(query string, userID string, opts SearchOptions)
([]SearchResult, error) {
 var results []SearchResult

 switch opts.Type {
 case "semantic":
 // Milvus 向量搜索
 embedding := s.embedModel.Embed(query)
 milvusResults := s.milvus.Search("knowledge_chunks", embedding, opts.TopK)
 results = append(results, s.enrichResults(milvusResults)...)

 case "fulltext":
 // MongoDB 全文搜索
 mongoResults := s.mongo.Find("knowledge_doc_contents", bson.M{
 "$text": bson.M{"$search": query},
 })
 results = append(results, s.enrichResults(mongoResults)...)

 case "hybrid":
 // 混合搜索 + 重排序
 semResults := s.Search(query, userID, SearchOptions{Type: "semantic", TopK:

```

```
opts.TopK))

 fullResults := s.Search(query, userID, SearchOptions{Type: "fulltext", TopK:
opts.TopK}))

 results = s.mergeAndRerank(semResults, fullResults)

}

// 权限过滤
results = s.filterByPermission(results, userID)

return results, nil
}
```

---

## 15. 审计系统设计

### 12.1 审计日志模型

```
{
 "_id": "audit_20260701_001",
 "timestamp": "2026-07-01T10:30:00Z",
 "user_id": "user_001",
 "username": "zhangsan",
 "ip_address": "192.168.1.100",
 "session_id": "session_chat_001",
 "action_type": "agent_task_create", // 操作类型枚举
 "action_detail": {
 "task_id": "task_001",
 "query": "对过去3年销售数据做回归分析",
 "data_sources": ["sales_db"],
 "mode": "async"
 },
 "resource_type": "agent_task",
 "resource_id": "task_001",
 "result": "success",
 "error": null,
 "duration_ms": 120
}
```

## 12.2 Agent 操作审计

```
{
 "_id": "audit_20260701_002",
 "timestamp": "2026-07-01T10:30:05Z",
 "trigger_user_id": "user_001",
 "agent_id": "agent_001",
 "task_id": "task_001",
 "session_id": "session_agent_001",
 "ip_address": "192.168.1.100",
 "action_type": "skill_call",
 "action_detail": {
 "skill_name": "sql_executor",
 "params_summary": "SELECT revenue FROM sales WHERE region='华东'",
 "duration_ms": 320,
 "result": "success",
 "rows_affected": 150
 },
 "result": "success"
}
```

## 12.3 日志类数据表 TTL 自动过期

所有日志类 Collection 使用 MongoDB TTL 索引自动过期，无需手动清理任务：

```
// 审计日志 - 90 天
db.audit_logs.createIndex(
 { "created_at": 1 },
 { expireAfterSeconds: 7776000, name: "ttl_created_at" }
)

// 会话记录 - 30 天（已过期/已完成的会话）
db.sessions.createIndex(
 { "expires_at": 1 },
 { expireAfterSeconds: 0, name: "ttl_expires_at" }
)

// 请求日志 - 30 天
db.request_logs.createIndex(
```

```
 { "created_at": 1 },
 { expireAfterSeconds: 2592000, name: "ttl_created_at" }
)

// 通知记录 - 7 天
db.notifications.createIndex(
 { "created_at": 1 },
 { expireAfterSeconds: 604800, name: "ttl_created_at" }
)

// Token 消耗日志 - 90 天 (长期统计从 Redis Stats 聚合)
db.token_consumptions.createIndex(
 { "created_at": 1 },
 { expireAfterSeconds: 7776000, name: "ttl_created_at" }
)
```

**TTL 策略:** MongoDB TTL 索引在后台每 60 秒扫描一次，过期文档自动删除。不需要额外的定时清理 Worker。部分查询依赖 `created_at` 的集合需额外创建非 TTL 的查询索引。

---

## 13. 统计监控 (Redis Stats Counter)

---

### 13.1 设计原则

统计指标使用 Redis 作为实时计数器（必须开启 AOF + RDB 持久化），Scheduler 定时任务直接写入 Redis，不经过消息队列，仅记录日志。提供 Dashboard API 聚合查询。

13.2 统计指标

指标	Redis Key	说明
Agent 调用次数	<code>stats:agent_calls:{date}</code>	按日聚合，区分 sync/async
模型调用次数	<code>stats:model_calls:{date}:{model_name}</code>	按模型+日期聚合
Session 创建次数	<code>stats:sessions:{date}</code>	按日聚合，区分 chat/agent
Task 创建次数	<code>stats:tasks:{date}</code>	按日聚合，区分 sync/async/scheduled
Token 消耗量	<code>stats:tokens:{date}:{model_name}</code>	按模型+日期聚合 input/output token
AI 总成本	<code>stats:cost:{date}</code>	按日聚合，单位分 (cents)

13.3 Redis 写入策略

```
// Scheduler 定时任务 (每 5 分钟)，直接写入 Redis
func (s *StatsCollector) Collect(ctx context.Context) error {
 today := time.Now().Format("2006-01-02")

 // 原子递增 Redis 计数器
 pipe := s.redis.Pipeline()

 // Agent 调用次数
 pipe.IncrBy(ctx, fmt.Sprintf("stats:agent_calls:%s", today), s.agentCallCount())
 pipe.IncrBy(ctx, fmt.Sprintf("stats:agent_calls:%s:sync", today), s.agentSyncCount())
 pipe.IncrBy(ctx, fmt.Sprintf("stats:agent_calls:%s:async", today), s.agentAsyncCount())

 // Session/Task
 pipe.IncrBy(ctx, fmt.Sprintf("stats:sessions:%s", today), s.sessionCount())
 pipe.IncrBy(ctx, fmt.Sprintf("stats:tasks:%s", today), s.taskCount())

 // Token 消耗 (按模型拆分)
 for model, tokens := range s.tokenUsage() {
 pipe.IncrBy(ctx, fmt.Sprintf("stats:tokens:%s:%s:input", today, model),
tokens.Input)
```

```

 pipe.IncrBy(ctx, fmt.Sprintf("stats:tokens:%s:%s:output", today, model),
tokens.Output)

 pipe.IncrByFloat(ctx, fmt.Sprintf("stats:cost:%s", today), tokens.CostCents)
 }

 _, err := pipe.Exec(ctx)
 return err
}

```

### 13.4 投入产出比 (ROI) 计算

```

// ROI = 等效节省人时 / AI 总成本
// 等效节省人时 = Agent调用次数 × 平均单次节省时间(30min) / 60
// AI 总成本 = sum(stats:cost:{date}) / 100 → 元

func (s *StatsCollector) CalcROI(ctx context.Context, days int) (*ROIResult, error) {
 // 从 Redis 读取近 N 天的聚合数据
 dates := s.lastNDates(days)

 var totalCalls int64
 var totalCost float64

 for _, date := range dates {
 calls, _ := s.redis.Get(ctx, fmt.Sprintf("stats:agent_calls:%s", date)).Int64()
 cost, _ := s.redis.Get(ctx, fmt.Sprintf("stats:cost:%s", date)).Float64()
 totalCalls += calls
 totalCost += cost
 }

 savedHours := float64(totalCalls) * 0.5 // 每次节省 30 分钟
 totalCostYuan := totalCost / 100 // 分 → 元

 return &ROIResult{
 TotalCalls: totalCalls,
 SavedHours: savedHours,
 TotalCostYuan: totalCostYuan,
 ROI: savedHours / totalCostYuan * 100, // 百分比
 }
}

```



```
 }, nil
}
```

13.5 Redis 持久化配置

```
redis.conf - 必须同时开启 AOF + RDB

save 900 1

save 300 10

save 60 10000

appendonly yes

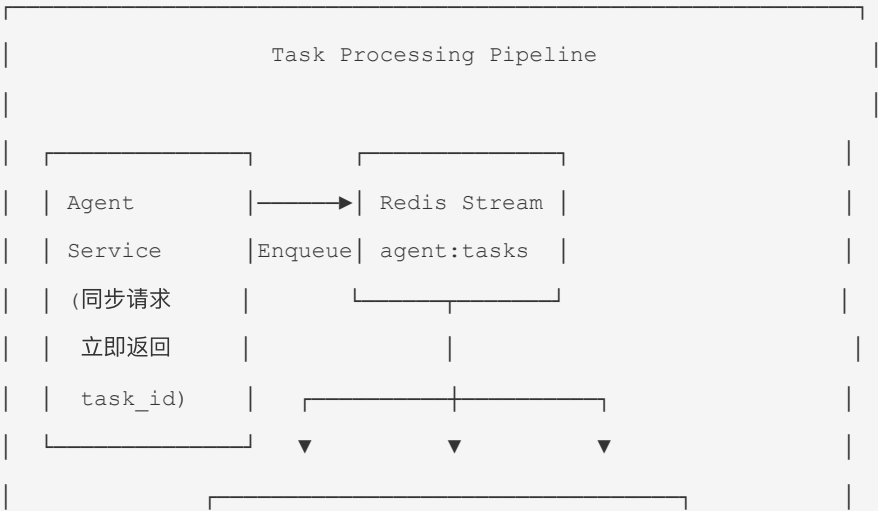
appendfsync everysec
```

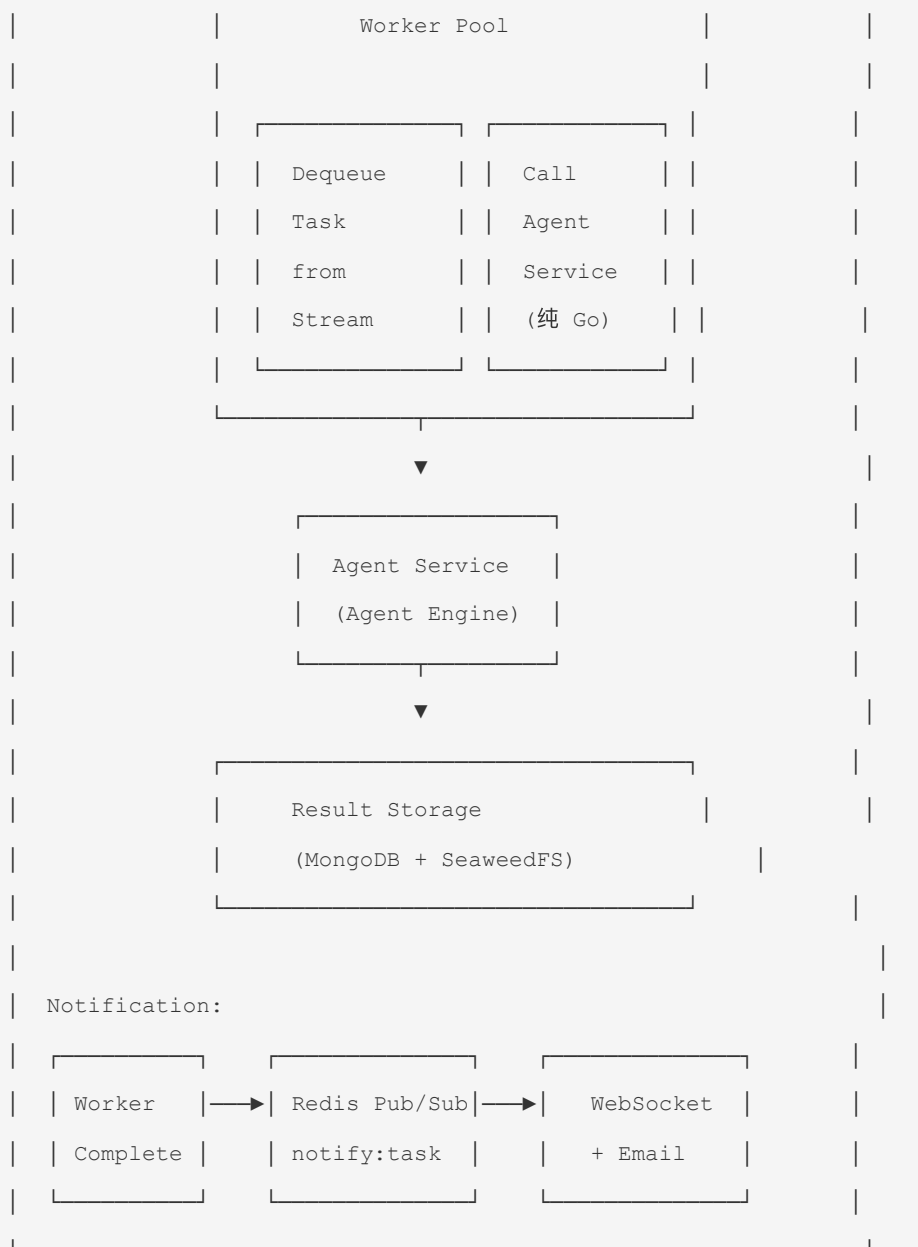
**数据可靠性:** AOF + RDB 双持久化确保统计数据不会因 Redis 重启丢失。Scheduler 写入采用 Pipeline 批量操作，5 分钟窗口内的少量数据丢失可接受。

16. 异步任务与消息队列

13.1 Redis Stream 作为轻量消息队列

**Worker 职责定义:** Worker 是异步任务消费者，运行在 Go 服务进程的独立 goroutine 中。核心职责：从 Redis Stream 消费异步 Agent 任务和定时任务，回调 Agent Service 执行 Agent Engine 核心逻辑。Worker 纯 Go 实现，不依赖 Python/R 等外部运行时。





## Worker 实现:

```

// Worker 职责 (纯 Go, 不依赖外部脚本运行时) :
// 从 Redis Stream 消费异步 Agent 任务 → 回调 Agent Service 执行核心逻辑

type AgentWorker struct {
 redis *RedisClient
 agentSvc *AgentService // 回调 Agent Service
 notifier *Notifier
}

func (w *AgentWorker) Run(ctx context.Context, workerID string) {
 for {

```

```

// 从 Redis Stream 消费任务

msgs, err := w.redis.XReadGroup(ctx, &redis.XReadGroupArgs{
 Group: "agent_workers",
 Consumer: workerID,
 Streams: []string{"agent:tasks", ">"},
 Count: 1,
 Block: 5 * time.Second,
})

if err != nil || len(msgs) == 0 {
 continue
}

for _, msg := range msgs[0].Messages {
 taskID := msg.Values["task_id"].(string)
 w.processTask(ctx, taskID)
 w.redis.XAck(ctx, "agent:tasks", "agent_workers", msg.ID)
}
}

func (w *AgentWorker) processTask(ctx context.Context, taskID string) {
 task := w.loadTask(taskID)

 // 创建可取消的 context
 taskCtx, cancel := context.WithCancel(ctx)
 w.registerCancelHandler(taskID, cancel)
 defer cancel()

 // 更新状态
 w.updateTaskStatus(taskID, "running")

 // 回调 Agent Service 执行核心逻辑
 // Worker 不持有 Agent Engine, 完全复用 Agent Service
 result, err := w.agentSvc.ExecuteTask(taskCtx, task)

 if taskCtx.Err() == context.Canceled {
 w.updateTaskStatus(taskID, "cancelled")
 return
 }
}

```

```
 if err != nil {
 w.updateTaskStatus(taskID, "failed")
 w.notifier.NotifyFailure(task.UserID, taskID, err)
 return
 }

 // 保存结果
 w.saveResult(taskID, result)
 w.updateTaskStatus(taskID, "completed")
 w.notifier.NotifyCompletion(task.UserID, taskID)
}
```

## 17. Hermes Service（自由探索模式）

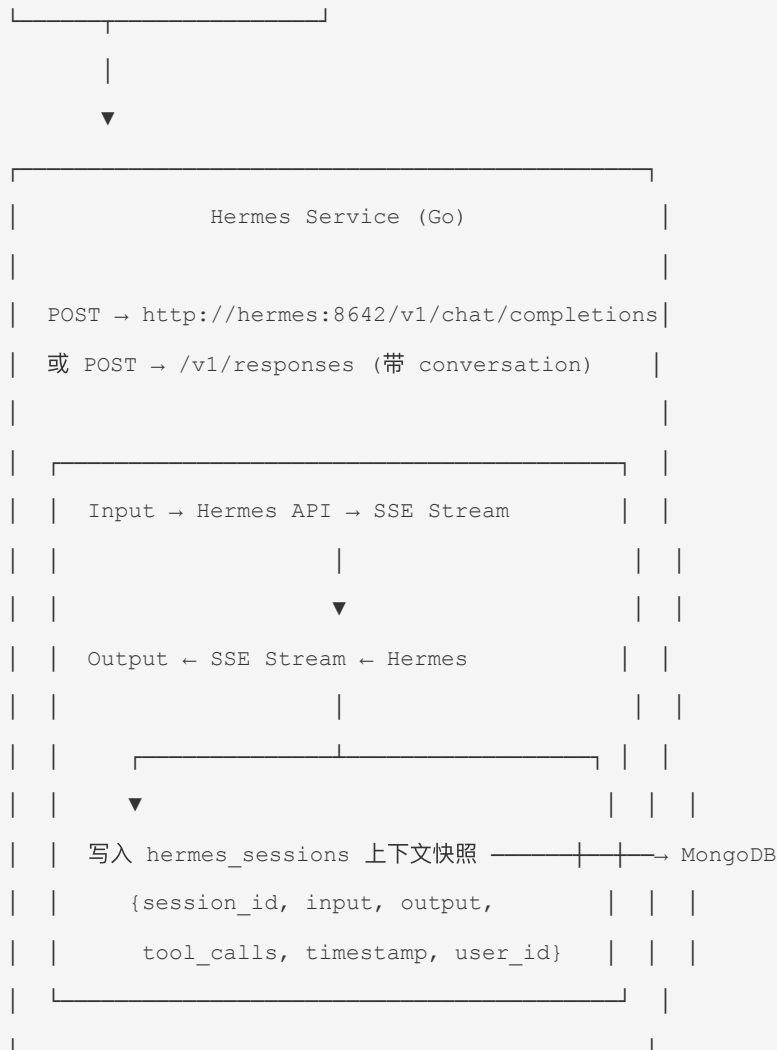
### 17.1 设计原则

Hermes Service 是 Data Agent 系统中的一个**轻量独立服务**，专用于"自由探索模式"。它遵循以下原则：

原则	说明
接口隔离	与 Agent Service 完全解耦，互不依赖
转发模式	不做任何 LLM 逻辑处理，仅做请求转发和输入/输出记录
Session 双轨	Hermes Session 由 Hermes 端管理；Data Agent 仅保存轻量上下文快照
共享 MongoDB	仅共享 MongoDB 连接用于 <code>hermes_sessions</code> 集合写入
独立部署	Docker Compose 中独立容器，可独立扩缩

### 17.2 架构示意





### 17.3 MongoDB `hermes_sessions` 集合

```

{
 "_id": "ObjectId",
 "session_id": "hermes-session-abcl23",
 "user_id": "user_zhangsan",
 "conversation_name": "探索-2026-07-03-001",
 "messages": [
 {
 "role": "user",
 "content": "帮我分析以太坊最近的价格趋势",
 "timestamp": "2026-07-03T10:30:00Z"
 },
 {
 "role": "assistant",

```

```
 "content": "根据最近数据...",
 "tool_calls": ["web_search"],
 "timestamp": "2026-07-03T10:30:15Z"
 }
],
 "hermes_session_id": "hermes://20260703_103000_a1b2c3",
 "platform": "api-server",
 "started_at": "2026-07-03T10:30:00Z",
 "ended_at": null,
 "token_count_input": 1240,
 "token_count_output": 850,
 "status": "active"
}
```

索引设计:

```
hermes_sessions:
 - {user_id: 1, started_at: -1} # 用户按时间查询
 - {session_id: 1} # 唯一查找
 - {status: 1, started_at: -1} # 活跃会话筛选
```

17.4 API 端点

方法	路径	说明
POST	/api/hermes/chat	探索模式对话: 接收 {input, conversation?} → 转发 Hermes → SSE 流式返回
GET	/api/hermes/sessions	列出当前用户的 Hermes Session 记录 (从 MongoDB 读取)
GET	/api/hermes/sessions/{id}	获取指定 Session 的上下文快照
DELETE	/api/hermes/sessions/{id}	删除 Session 记录 (仅删 MongoDB 快照, 不删 Hermes 端)

17.5 Docker Compose 集成

```
docker-compose.prod.yml / docker-compose.test.yml 新增

services:
 # =====
```

```
Hermes Service

=====

hermes-service:

 build:

 context: ./hermes-service

 dockerfile: Dockerfile

 container_name: hermes-service

 ports:

 - "${HERMES_PORT:-8088}:8088"

 environment:

 HERMES_API_URL: ${HERMES_API_URL:-http://hermes:8642}

 HERMES_API_KEY: ${HERMES_API_KEY:-}

 MONGO_URI: mongodb://mongodb:27017

 MONGO_DB: dataagent

 LOG_LEVEL: info

 depends_on:

 mongodb:

 condition: service_healthy

 hermes:

 condition: service_healthy

 restart: unless-stopped

 healthcheck:

 test: ["CMD", "wget", "-q", "-O-", "http://localhost:8088/health"]

 interval: 30s

 timeout: 5s

 retries: 3

Hermes Agent (官方镜像/自部署)

hermes:

 image: ghcr.io/nousresearch/hermes-agent:latest

 container_name: hermes-agent

 ports:

 - "${HERMES_PORT_INTERNAL:-8642}:8642"

 environment:

 API_SERVER_ENABLED: "true"

 API_SERVER_PORT: "8642"

 API_SERVER_HOST: "0.0.0.0"

 API_SERVER_KEY: ${HERMES_API_KEY:-}

 LLM_PROVIDER: ${HERMES_LLM_PROVIDER:-openai}
```

```
LLM_API_KEY: ${HERMES_LLM_API_KEY:-}

LLM_MODEL: ${HERMES_LLM_MODEL:-gpt-4o}

volumes:

 - hermes_data:/root/.hermes

restart: unless-stopped

healthcheck:

 test: ["CMD", "wget", "-q", "-O-", "http://localhost:8642/health"]

 interval: 30s

 timeout: 5s

 retries: 3

volumes:

 hermes_data:
```

## 17.6 环境变量

```
.env.prod / .env.test 新增

Hermes Service
HERMES_PORT=8088
HERMES_API_URL=http://hermes:8642
HERMES_API_KEY=hermes-api-key-change-me

Hermes Agent
HERMES_PORT_INTERNAL=8642
HERMES_LLM_PROVIDER=openai
HERMES_LLM_API_KEY=sk-your-llm-key
HERMES_LLM_MODEL=gpt-4o
```

## 17.7 关键约束

1. **Session 不互通**: Hermes Session 与 Data Agent Session 完全隔离，不交叉引用。探索模式下无法访问 Data Agent 的工具/知识库/MCP。
2. **仅代理转发**: Hermes Service 不对 Hermes 的请求/响应做任何解释、转换或增强。它仅做透传 + 记录。
3. **数据最小化**: MongoDB 中仅存储输入/输出摘要 + 元数据，不存储完整工具结果。
4. **无状态服务**: Hermes Service 本身无状态，可水平扩展多副本。
5. **单向依赖**: hermes-service → hermes 容器，hermes 崩溃时 hermes-service 返回 503。



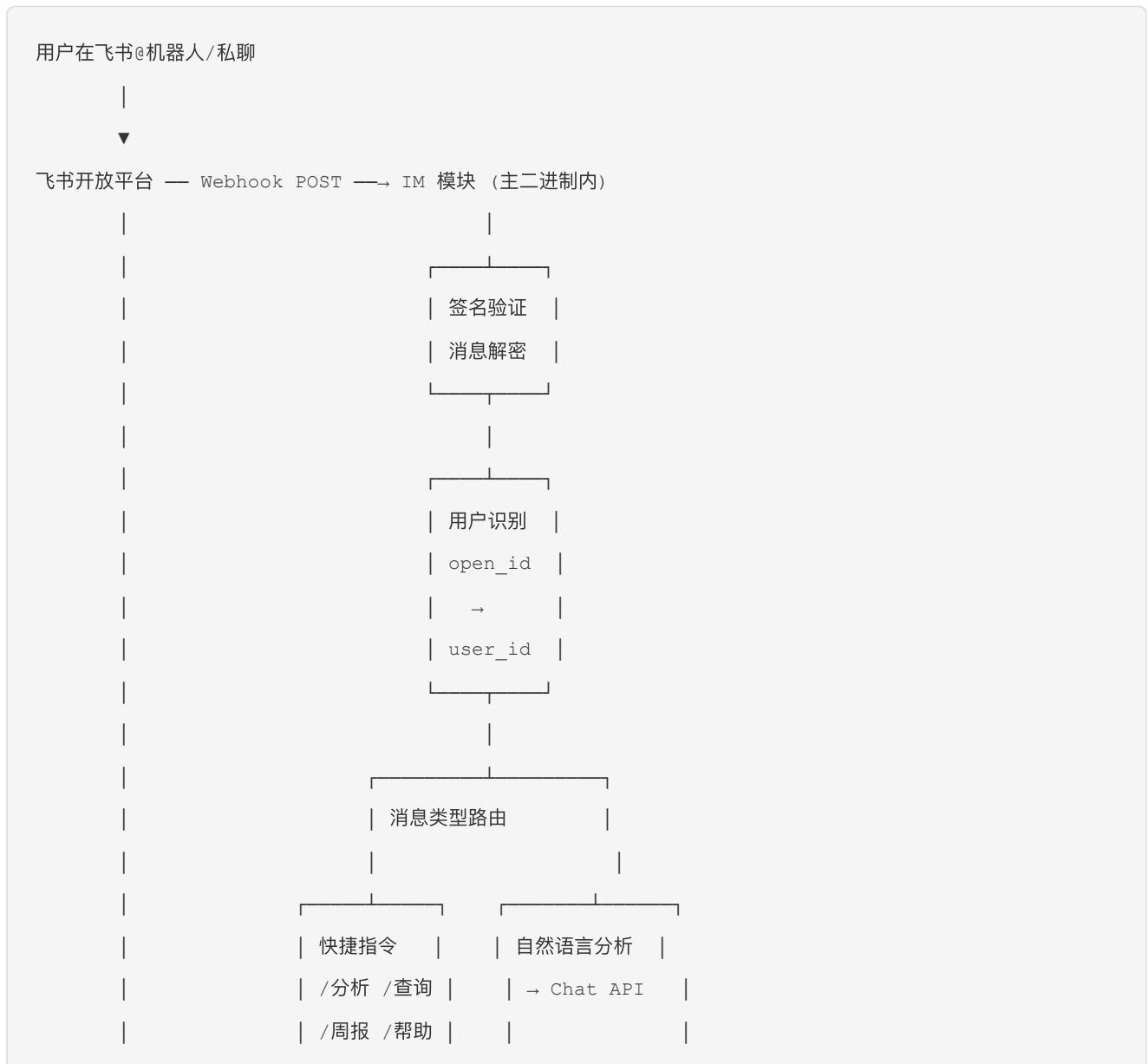
## 17.8 IM 集成架构 (IM 模块，集成在主二进制)

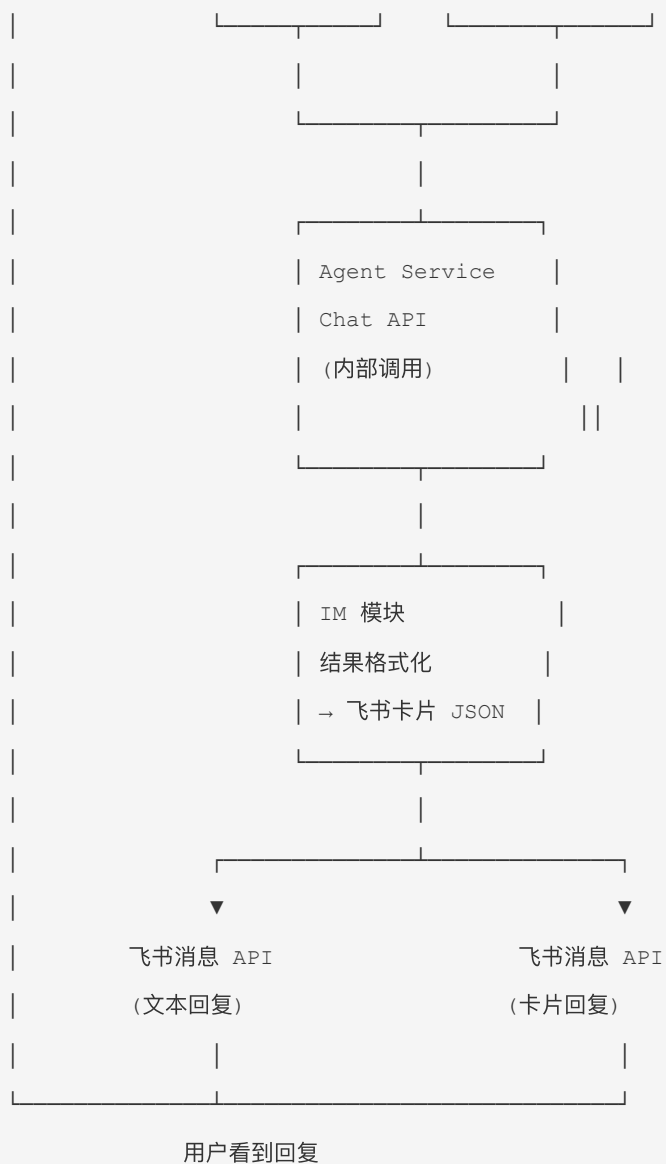
### 17.8.1 设计目标

IM 模块集成在 Data Agent 主二进制中 ( `internal/service/im/` ), 负责统一管理企业 IM 平台的消息收发。遵循以下原则:

- **集成部署:** IM 模块与 Agent Service 在同一进程中运行, 通过内部调用通信, 不单独部署
- **平台适配:** 每个 IM 平台 (飞书/钉钉/企微) 有独立的适配器, 共享消息路由和用户绑定逻辑
- **消息路由:** IM 消息 → Agent Service Chat API (内部调用) → IM 返回 (复用现有 Chat 能力)
- **MVP 策略:** 优先实现飞书适配器, 架构预留钉钉/企微扩展点

### 17.8.2 飞书集成流程





### 17.8.3 核心数据结构

用户绑定 (im\_bindings):

```

// im_bindings collection - MongoDB
{
 "_id": "imb_feishu_ou_abc123",
 "platform": "feishu", // feishu | dingtalk | wecom
 "open_id": "ou_abc123def456", // IM 平台的用户唯一标识
 "user_id": "user_001", // 系统用户 ID
 "tenant_key": "2ed263bf32cf1651", // 飞书企业标识 (多租户隔离)
 "nickname": "张三", // IM 昵称 (冗余, 方便展示)
 "avatar_url": "https://...", // IM 头像 (冗余)
 "created_at": "2026-07-01T10:00:00Z",
}

```

```
"updated_at": "2026-07-01T10:00:00Z"
}
// 唯一索引: {platform: 1, open_id: 1, tenant_key: 1}
```

### 飞书卡片消息模板:

```
// internal/im/feishu/card.go

// BuildAnalysisResultCard 分析结果 → 飞书卡片消息 JSON
func BuildAnalysisResultCard(result *chat.ChatResult) *lark.MsgContent {
 card := &lark.MsgContent{
 Config: &lark.CardConfig{WideScreenMode: true},
 Header: &lark.CardHeader{
 Title: &lark.Text{Tag: "plain_text", Content: "📊 数据分析结果"},
 Template: "blue",
 },
 Elements: []lark.CardElement{
 // 关键指标摘要
 buildMetricsElement(result.Metrics),
 // 数据表格
 buildTableElement(result.Data),
 // 图表链接 (跳转 Web 查看完整图表)
 buildChartLink(result.ChartURL),
 // 操作按钮
 buildActionButtons(result),
 },
 }
 return card
}
```

## 17.8.4 IM 模块结构

```
internal/service/im/
├── gateway.go # IM 模块主入口 (HTTP 路由注册, 集成在主二进制)
├── router.go # 消息路由器 (平台适配 → Agent Service 内部调用)
├── binding.go # 用户绑定管理 (CRUD + 缓存)
├── command.go # 快捷指令解析
├── formatter.go # 消息格式化 (Agent 响应 → IM 消息格式)
├── audit.go # IM 消息审计日志
```

```

|
├─ feishu/ # 飞书平台适配器
| ├─ adapter.go # 飞书适配器实现 (im.PlatformAdapter 接口)
| ├─ webhook.go # Webhook 接收 + 签名验证 + 消息解密
| ├─ message.go # 消息发送 (文本/卡片/图片)
| ├─ card.go # 卡片消息模板 (分析结果/通知/引导)
| └─ event.go # 事件订阅 (消息事件/用户事件)
|
├─ dingtalk/ # 钉钉平台适配器 (V1.1)
| └─ adapter.go
|
└─ wecom/ # 企业微信平台适配器 (V1.1)
 └─ adapter.go

```

### 平台适配器接口:

```

// internal/service/im/gateway.go

// PlatformAdapter 定义 IM 平台适配器接口
type PlatformAdapter interface {

 // Platform 返回平台标识
 Platform() string // "feishu" | "dingtalk" | "wecom"

 // HandleWebhook 处理 IM 平台的 Webhook 回调
 HandleWebhook(ctx context.Context, req *http.Request) (*IMMessage, error)

 // SendMessage 发送消息到 IM 平台
 SendMessage(ctx context.Context, openID string, content *MessageContent) error

 // GetUserInfo 获取 IM 用户信息 (用于绑定引导)
 GetUserInfo(ctx context.Context, openID string) (*IMUserInfo, error)

 // VerifySignature 验证 Webhook 签名
 VerifySignature(req *http.Request, body []byte) error
}

// IMMessage 标准化 IM 消息
type IMMessage struct {

 Platform string // feishu

```

```

 OpenID string // IM 用户 ID
 TenantKey string // 企业标识
 ChatID string // 群聊 ID (私聊为空)
 ChatType string // private | group
 Text string // 消息文本 (去除 @机器人 前缀)
 RawData json.RawMessage // 平台原始消息
 Timestamp time.Time

}

// MessageContent 标准化消息内容
type MessageContent struct {
 Text string // 纯文本
 Card *CardDefinition // 卡片消息 (可选)
 QuickReplies []string // 快捷回复按钮
}

```

## 17.8.5 飞书 SDK 集成 (go-lark)

```

// go-lark/lark SDK - 消息 Bot 初始化

import "github.com/go-lark/lark"

func NewFeishuAdapter(cfg *FeishuConfig) (*FeishuAdapter, error) {
 bot := lark.NewChatBot(
 cfg.AppID,
 cfg.AppSecret,
 lark.WithVerificationToken(cfg.VerificationToken),
 lark.WithEncryptKey(cfg.EncryptKey),
)

 // 注册消息处理器
 bot.OnTextMessage(func(ctx context.Context, msg *lark.TextMsg) error {
 return handleTextMessage(ctx, msg)
 })

 // 启动事件订阅服务
 go bot.StartEventSvr(cfg.EventPort)
}

```

```

 return &FeishuAdapter{bot: bot, cfg: cfg}, nil
}

// 发送文本消息
func (a *FeishuAdapter) SendText(ctx context.Context, openID, text string) error {
 msg := lark.NewMsgBuffer(lark.MsgText)
 msg.BindChatter(openID, lark.UserChatterType)
 msg.Text(text)
 return a.bot.Send(msg.Build())
}

// 发送卡片消息
func (a *FeishuAdapter) SendCard(ctx context.Context, openID string, card *CardDefinition)
error {
 msg := lark.NewMsgBuffer(lark.MsgInteractive)
 msg.BindChatter(openID, lark.UserChatterType)
 msg.Card(card.ToFeishuJSON())
 return a.bot.Send(msg.Build())
}

```

## 17.8.6 用户绑定流程

首次使用机器人

|

▼

用户 @机器人 或 私聊任意消息

|

▼

IM 模块查询 im\_bindings

|

├ 已绑定 → 识别 user\_id → 正常处理消息

|

└ 未绑定 → 返回绑定引导卡片

|

▼

卡片内容:

"请先绑定您的 Data Agent 账号"

[绑定账号] 按钮 → 生成一次性绑定 Token



绑定 Token 设计:

```
// 一次性绑定 Token, 5 分钟过期
type BindToken struct {
 Token string `bson:"_id"`
 Platform string `bson:"platform"`
 OpenID string `bson:"open_id"`
 TenantKey string `bson:"tenant_key"`
 ExpiresAt time.Time `bson:"expires_at"` // TTL 索引自动清理
 CreatedAt time.Time `bson:"created_at"`
}
```

17.8.7 快捷指令

指令	功能	映射
/分析 <query>	发起数据分析	→ Agent Service Chat API (mode: chat)
/查询 <query>	快速数据查询	→ Agent Service Chat API (mode: chat, cache_first)
/周报	生成本周经营周报	→ Agent Service Chat API (预设 prompt)
/帮助	查看可用指令和说明	→ 本地静态回复

17.8.8 集成说明

IM 模块（`internal/service/im/`）编译在主二进制内，无需独立 Docker Compose 容器。飞书 Webhook 端口复用主服务的 HTTP Server（通过 `GET/POST /api/v1/im/feishu/*` 路由注册）。

主服务 `docker-compose.yml` 中只需增加飞书环境变量：

```
docker-compose.yml agent-service 内新增
environment:
 FEISHU_APP_ID: ${FEISHU_APP_ID:-}
 FEISHU_APP_SECRET: ${FEISHU_APP_SECRET:-}
 FEISHU_VERIFICATION_TOKEN: ${FEISHU_VERIFICATION_TOKEN:-}
 FEISHU_ENCRYPT_KEY: ${FEISHU_ENCRYPT_KEY:-}
```

17.8.9 环境变量

```
飞书配置（主服务 .env 新增）
FEISHU_APP_ID=cli_XXXXXXXXXXXX
FEISHU_APP_SECRET=XXXXXXXXXXXXXXXXXXXXXXXXXXXX
FEISHU_VERIFICATION_TOKEN=XXXXXXXXXXXXXXXXXXXX
FEISHU_ENCRYPT_KEY=XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

17.8.10 关键约束

- 1. 消息转发不变形: IM 模块不修改用户消息内容，仅做 @机器人 前缀去除和平台差异处理。
- 2. 用户绑定必做: 未绑定用户的所有消息返回绑定引导，不进入 Agent 处理流程。
- 3. 审计日志记录: 所有 IM 消息（入/出）记录审计日志， `source: "feishu_bot"`，包含 open\_id、user\_id、消息内容摘要。
- 4. 复用 Chat 模式: IM 渠道复用现有 Agent Service Chat API，不创建独立的推理管道。
- 5. 平台适配器模式: 每个 IM 平台实现 `PlatformAdapter` 接口，核心路由逻辑不变。
- 6. 集成部署: IM 模块与主服务编译在同一二进制，共享 MongoDB/Redis 连接池，IM 会话上下文由 Agent Service 统一管理。

17.8.11 V1.1 扩展规划

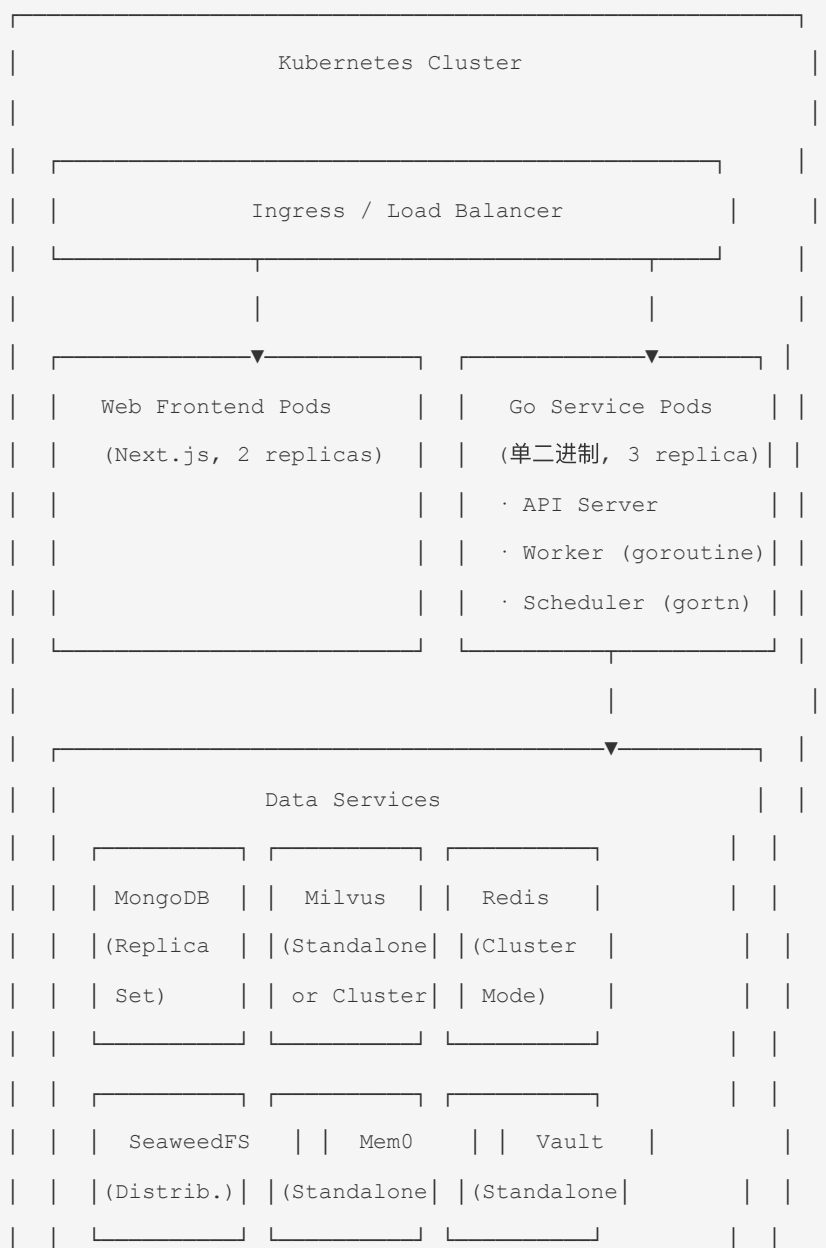
平台	新增组件	复用部分
钉钉	<code>dingtalk/adapters.go</code>	消息路由、用户绑定、结果格式化
企业微信	<code>wecom/adapters.go</code>	同上
交互式卡片	飞书/钉钉卡片 Action 处理	快捷指令解析
消息模板管理	MongoDB <code>im_templates</code> + Admin UI	-

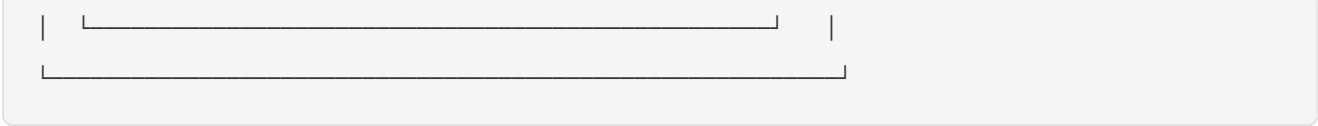


## 18. 部署架构

### 14.1 生产部署拓扑

**部署形态:** Service Layer 为单个 Go 二进制文件，API Server、Worker、Scheduler 均在同一进程中通过 goroutine 运行。通过 Kubernetes Deployment 水平扩展多个副本，各副本共享 Redis 协调任务分配。





**Vault 职责:** Go Service 启动时从 Vault 拉取 JWT Secret、MongoDB 密码、SeaweedFS Access Key、SMTP 密码、LLM API Key 等敏感配置。支持配置热更新（Vault lease renewal）。开发环境可用文件模式（`vault agent -mode=dev`）替代。

17.2 MVP 一键部署 (Docker Compose)

MVP 提供**两套** Docker Compose 配置，按场景区分：

文件	用途	包含服务
<code>docker-compose.prod.yml</code>	正式部署 / Demo	api, frontend, mongodb, milvus, seaweedfs, redis, vault, mem0
<code>docker-compose.test.yml</code>	本地开发 / 自动化测试	prod 全部 + LLM Mock（内置于 api 容器）+ MySQL（模拟客户数据库）+ <b>migration</b> （初始化企业测试数据）

两套配置可通过环境变量灵活切换 LLM 指向（真实 LLM 或 Mock），无需修改 compose 文件。

17.2.1 正式部署环境变量 (.env.prod)

正式部署所需的环境变量，复制为 `.env` 后使用：

```
=====
服务端口
=====
API_PORT=8080
FRONTEND_PORT=3000

=====
MongoDB
=====
MONGO_IMAGE=mongo:7.0
MONGO_PORT=27017
MONGO_ROOT_USER=admin
MONGO_ROOT_PASSWORD=changeme
MONGO_DATABASE=agent_platform

=====
Milvus (Standalone)
```

```
=====
MILVUS_IMAGE=milvusdb/milvus:v2.5.0
MILVUS_PORT=19530
MILVUS_METRICS_PORT=9091

=====
SeaweedFS (S3)
=====
SEAWEEDFS_IMAGE=chrislusf/seaweedfs:latest
SEAWEEDFS_S3_PORT=8333
SEAWEEDFS_ACCESS_KEY=seaweedfs-admin
SEAWEEDFS_SECRET_KEY=seaweedfs-admin

=====
Redis
=====
REDIS_IMAGE=redis:7.2-alpine
REDIS_PORT=6379
REDIS_PASSWORD=

=====
Vault (Dev Mode)
=====
VAULT_IMAGE=hashicorp/vault:1.14
VAULT_PORT=8200
VAULT_DEV_ROOT_TOKEN=dev-root-token

=====
Mem0
=====
MEM0_IMAGE=mem0ai/mem0:latest
MEM0_PORT=8000

=====
LLM
=====
LLM_PROVIDER=openai
LLM_MODEL=gpt-4o
LLM_BASE_URL=https://api.openai.com/v1
```

```
LLM_API_KEY=sk-your-key-here

LLM_MAX_TOKENS=8192

LLM_TEMPERATURE=0.1

=====

Embedding

=====

EMBEDDING_PROVIDER=openai

EMBEDDING_MODEL=text-embedding-3-small

EMBEDDING_DIM=1536

=====

JWT & Security

=====

JWT_SECRET=change-me-to-a-random-string

JWT_EXPIRE_HOURS=24

=====

Email (SMTP)

=====

SMTP_HOST=smtp.company.com

SMTP_PORT=587

SMTP_USER=noreply@company.com

SMTP_PASSWORD=

SMTP_ALLOWED_DOMAINS=company.com

=====

Report Validator

=====

REPORT_VALIDATOR_MAX_RETRIES=3
```

### 17.2.2 正式部署一键启动

```
1. 复制环境变量文件

cp .env.prod .env

2. 编辑 .env (至少修改 LLM_API_KEY、JWT_SECRET)

vim .env
```

```
3. 一键启动全部服务（使用正式 compose 文件）

docker compose -f docker-compose.prod.yml up -d

4. 查看日志

docker compose -f docker-compose.prod.yml logs -f api

5. 访问

前端： http://localhost:3000
管理后台： http://localhost:3000/admin
API： http://localhost:8080
```

### 17.2.3 docker-compose.prod.yml（正式部署）

```
version: '3.8'

services:

 # =====
 # Go API Service (含 Agent Service + Worker Pool + Scheduler)
 # =====
 api:
 build:
 context: .
 dockerfile: Dockerfile
 container_name: agent-api
 ports:
 - "${API_PORT:-8080}:8080"
 environment:
 # MongoDB
 MONGO_URI:
 mongodb://${MONGO_ROOT_USER}:${MONGO_ROOT_PASSWORD}@mongodb:${MONGO_PORT:-27017}/${MONGO_DATABASE:-agent_platform}?authSource=admin
 # Redis
 REDIS_ADDR: redis:${REDIS_PORT:-6379}
 REDIS_PASSWORD: ${REDIS_PASSWORD:-}
 # SeaweedFS (S3)
 SEAWEEDFS_ENDPOINT: seaweedfs:${SEAWEEDFS_S3_PORT:-8333}
 SEAWEEDFS_ACCESS_KEY: ${SEAWEEDFS_ACCESS_KEY:-seaweedfs-admin}
 SEAWEEDFS_SECRET_KEY: ${SEAWEEDFS_SECRET_KEY:-seaweedfs-admin}
 SEAWEEDFS_WORKSPACE_VOLUME: workspaces
```

```

SEAWEEEDFS_ARTIFACT_VOLUME: artifacts

Milvus
MILVUS_ADDR: milvus:${MILVUS_PORT:-19530}

Mem0
MEM0_ADDR: mem0:${MEM0_PORT:-8000}

Vault
VAULT_ADDR: http://vault:${VAULT_PORT:-8200}
VAULT_TOKEN: ${VAULT_DEV_ROOT_TOKEN:-dev-root-token}

LLM
LLM_PROVIDER: ${LLM_PROVIDER:-openai}
LLM_MODEL: ${LLM_MODEL:-gpt-4o}
LLM_BASE_URL: ${LLM_BASE_URL:-https://api.openai.com/v1}
LLM_API_KEY: ${LLM_API_KEY}
LLM_MAX_TOKENS: ${LLM_MAX_TOKENS:-8192}
LLM_TEMPERATURE: ${LLM_TEMPERATURE:-0.1}

Embedding
EMBEDDING_PROVIDER: ${EMBEDDING_PROVIDER:-openai}
EMBEDDING_MODEL: ${EMBEDDING_MODEL:-text-embedding-3-small}
EMBEDDING_DIM: ${EMBEDDING_DIM:-1536}

JWT
JWT_SECRET: ${JWT_SECRET}
JWT_EXPIRE_HOURS: ${JWT_EXPIRE_HOURS:-24}

Email
SMTP_HOST: ${SMTP_HOST}
SMTP_PORT: ${SMTP_PORT:-587}
SMTP_USER: ${SMTP_USER}
SMTP_PASSWORD: ${SMTP_PASSWORD:-}
SMTP_ALLOWED_DOMAINS: ${SMTP_ALLOWED_DOMAINS:-company.com}

Report Validator
REPORT_VALIDATOR_MAX_RETRIES: ${REPORT_VALIDATOR_MAX_RETRIES:-3}

Workspace
WORKSPACE_ROOT: ${WORKSPACE_ROOT:-/workspaces}
WORKSPACE_MAX_SIZE_MB: ${WORKSPACE_MAX_SIZE_MB:-500}

volumes:
 - workspace_data:/workspaces

depends_on:
 mongodb:
 condition: service_healthy
 redis:

```

```

 condition: service_healthy
seaweedfs:
 condition: service_healthy
milvus:
 condition: service_healthy
restart: unless-stopped
healthcheck:
 test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
 interval: 15s
 timeout: 5s
 retries: 3
 start_period: 30s

=====
Next.js Frontend (含用户端 + 管理后台)
=====
frontend:
 build:
 context: ./web
 dockerfile: Dockerfile
 container_name: agent-frontend
 ports:
 - "${FRONTEND_PORT:-3000}:3000"
 environment:
 NEXT_PUBLIC_API_URL: http://localhost:${API_PORT:-8080}
 API_INTERNAL_URL: http://api:8080
 depends_on:
 api:
 condition: service_healthy
 restart: unless-stopped

=====
MongoDB
=====
mongodb:
 image: ${MONGO_IMAGE:-mongo:7.0}
 container_name: agent-mongodb
 ports:
 - "${MONGO_PORT:-27017}:27017"

```

```

environment:

 MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USER:-admin}

 MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD:-changeme}

volumes:

 - mongo_data:/data/db

healthcheck:

 test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]

 interval: 10s

 timeout: 5s

 retries: 5

 start_period: 20s

restart: unless-stopped

=====
Milvus (Standalone - 无需 etcd/minio 依赖)
=====
milvus:

 image: ${MILVUS_IMAGE:-milvusdb/milvus:v2.5.0}

 container_name: agent-milvus

 ports:

 - "${MILVUS_PORT:-19530}:19530"

 - "${MILVUS_METRICS_PORT:-9091}:9091"

 environment:

 ETCD_USE_EMBED: "true"

 COMMON_STORAGETYPE: local

 volumes:

 - milvus_data:/var/lib/milvus

 healthcheck:

 test: ["CMD", "curl", "-f", "http://localhost:9091/healthz"]

 interval: 15s

 timeout: 10s

 retries: 5

 start_period: 40s

 restart: unless-stopped

=====
SeaweedFS (S3)
=====
seaweedfs:

```



```

 image: ${SEAWEEEDFS_IMAGE:-chrislusf/seaweedfs:latest}

 container_name: agent-seaweedfs

 ports:
 - "${SEAWEEEDFS_S3_PORT:-8333}:8333"

 command: server -s3 -s3.accessKey=${SEAWEEEDFS_ACCESS_KEY:-seaweedfs-admin} -
s3.secretKey=${SEAWEEEDFS_SECRET_KEY:-seaweedfs-admin}

 volumes:
 - seaweedfs_data:/data

 healthcheck:
 test: ["CMD", "curl", "-f", "http://localhost:8333/"]
 interval: 10s
 timeout: 5s
 retries: 5
 start_period: 10s
 restart: unless-stopped

=====
Redis
=====

redis:
 image: ${REDIS_IMAGE:-redis:7.2-alpine}

 container_name: agent-redis

 ports:
 - "${REDIS_PORT:-6379}:6379"

 command: ${REDIS_PASSWORD:+redis-server --requirepass ${REDIS_PASSWORD}}

 volumes:
 - redis_data:/data

 healthcheck:
 test: ["CMD", "redis-cli", "ping"]
 interval: 10s
 timeout: 3s
 retries: 5
 restart: unless-stopped

=====
Vault (Dev Mode - MVP 快速启动)
=====

vault:
 image: ${VAULT_IMAGE:-hashicorp/vault:1.14}

```

```

 container_name: agent-vault

 ports:
 - "${VAULT_PORT:-8200}:8200"

 cap_add:
 - IPC_LOCK

 environment:
 VAULT_DEV_ROOT_TOKEN_ID: ${VAULT_DEV_ROOT_TOKEN:-dev-root-token}
 VAULT_DEV_LISTEN_ADDRESS: 0.0.0.0:${VAULT_PORT:-8200}

 healthcheck:
 test: ["CMD", "vault", "status"]
 interval: 10s
 timeout: 5s
 retries: 5
 start_period: 10s
 restart: unless-stopped

=====
Mem0
=====
mem0:
 image: ${MEM0_IMAGE:-mem0ai/mem0:latest}
 container_name: agent-mem0
 ports:
 - "${MEM0_PORT:-8000}:8000"
 restart: unless-stopped

=====
Volumes (持久化存储)
=====
volumes:
 mongo_data:
 milvus_data:
 seaweedfs_data:
 redis_data:
 workspace_data:

```

## 17.2.4 Go Service Dockerfile（多阶段构建，安全加固）

```
===== Stage 1: Build =====
FROM golang:1.22-alpine AS builder

WORKDIR /app

安装依赖（利用 Docker 缓存层）
COPY go.mod go.sum ./
RUN go mod download

编译
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -o /app/server ./cmd/server

===== Stage 2: Runtime（安全加固） =====
FROM alpine:3.20

最小化依赖
RUN apk add --no-cache ca-certificates curl tzdata bash

创建非 root 用户
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

工作空间根目录 - appuser 唯一可读、写、执行的目录
RUN mkdir -p /workspaces && chown appuser:appgroup /workspaces && chmod 750 /workspaces

WORKDIR /app

复制二进制及配置（属 root:root）
COPY --from=builder /app/server .
COPY --from=builder /app/configs/ ./configs/
COPY --from=builder /app/skills/ ./skills/

限制 /app 目录：appuser 只能读和执行（755），不能写入
/app/server 需要 x 权限供容器启动执行；configs/skills 仅需 r
RUN chown -R root:root /app && chmod -R 755 /app

运行时以 appuser 执行 - 仅有 /workspaces 具备写+执行权限
```

```
其他路径 (/app、/usr 等) 仅有必要的读和系统级执行权限

USER appuser

EXPOSE 8080

HEALTHCHECK --interval=15s --timeout=5s --retries=3 \
 CMD curl -f http://localhost:8080/health || exit 1

ENTRYPOINT ["/server"]
```

### 17.2.5 正式部署启动后验证

```
检查所有服务健康状态

docker compose -f docker-compose.prod.yml ps

预期输出 - 全部 healthy

NAME STATUS
agent-api Up (healthy)
agent-frontend Up
agent-mongodb Up (healthy)
agent-milvus Up (healthy)
agent-seaweedfs Up (healthy)
agent-redis Up (healthy)
agent-vault Up (healthy)
agent-mem0 Up

API 健康检查

curl http://localhost:8080/health

#
{"status":"ok","mongodb":"connected","redis":"connected","milvus":"connected","seaweedfs":"connected"}

前端访问

open http://localhost:3000
```

## 17.3 测试部署 (docker-compose.test.yml)

测试部署在正式部署的基础上增加：

- **LLM Mock**：内置于 api 容器，通过 `LLM_BASE_URL=http://localhost:8080/mock/v1` 启用
- **MySQL**：模拟客户企业数据库，预置企业运营数据
- **Migration 服务**：启动时自动执行 SQL 初始化脚本，写入模拟企业数据

### 17.3.1 测试环境变量 (.env.test)

```
=====
基础配置 (继承 prod 环境变量)
=====
从 .env.prod 复制基础配置，追加/覆盖以下测试专用变量

=====
环境标识
=====
APP_ENV=test

=====
LLM → 指向内置 Mock
=====
LLM_BASE_URL=http://localhost:8080/mock/v1
LLM_API_KEY=mock-key
LLM_PROVIDER=openai
LLM_MODEL=mock-llm
LLM_MAX_TOKENS=4096
LLM_TEMPERATURE=0

=====
Embedding (仍需真实服务生成向量)
=====
EMBEDDING_PROVIDER=openai
EMBEDDING_MODEL=text-embedding-3-small
EMBEDDING_DIM=1536

=====
Mock LLM 参数
=====
MOCK_CHUNK_DELAY_MS=0
MOCK_DEFAULT_REPLY=Mock LLM: 请通过 POST /mock/responses 注入测试响应
```

```

=====
LOG - 测试环境开启 DEBUG
=====
LOG_LEVEL=debug
LOG_FORMAT=console

=====
MySQL (模拟客户企业数据库)
=====
MYSQL_IMAGE=mysql:8.0
MYSQL_PORT=3306
MYSQL_ROOT_PASSWORD=dev-root-password
MYSQL_DATABASE=enterprise_demo
MYSQL_USER=demo_user
MYSQL_PASSWORD=demo_password

=====
API Service 环境变量 (连接 MySQL)
=====
测试数据源连接 (SQL Executor 使用)
TEST_DATASOURCE_NAME=enterprise_demo
TEST_DATASOURCE_TYPE=mysql
TEST_DATASOURCE_DSN=demo_user:demo_password@tcp(mysql:3306)/enterprise_demo?
charset=utf8mb4&parseTime=true

=====
Workspace (本地目录挂载, 供 Skill 脚本执行)
=====
WORKSPACE_ROOT=/workspaces
WORKSPACE_MAX_SIZE_MB=500 # 单 session workspace 上限
WORKSPACE_CLEANUP_ON_STOP=true # compose down 时自动清理

```

### 17.3.2 docker-compose.test.yml (完整)

```

version: '3.8'

services:
 # =====

```

```

Go API Service (含 Agent Service + Worker Pool + Scheduler + Mock LLM)

=====
api:
 build:
 context: .
 dockerfile: Dockerfile
 container_name: agent-api
 ports:
 - "${API_PORT:-8080}:8080"
 environment:
 APP_ENV: ${APP_ENV:-test}
 LOG_LEVEL: ${LOG_LEVEL:-debug}
 LOG_FORMAT: ${LOG_FORMAT:-console}
 # MongoDB
 MONGO_URI:
mongodb://${MONGO_ROOT_USER}:${MONGO_ROOT_PASSWORD}@mongodb:${MONGO_PORT:-27017}/${MONGO_DATABASE:-agent_platform}?authSource=admin
 # Redis
 REDIS_ADDR: redis:${REDIS_PORT:-6379}
 REDIS_PASSWORD: ${REDIS_PASSWORD:-}
 # SeaweedFS (S3)
 SEAWEEDFS_ENDPOINT: seaweedfs:${SEAWEEDFS_S3_PORT:-8333}
 SEAWEEDFS_ACCESS_KEY: ${SEAWEEDFS_ACCESS_KEY:-seaweedfs-admin}
 SEAWEEDFS_SECRET_KEY: ${SEAWEEDFS_SECRET_KEY:-seaweedfs-admin}
 SEAWEEDFS_WORKSPACE_VOLUME: workspaces
 SEAWEEDFS_ARTIFACT_VOLUME: artifacts
 # Milvus
 MILVUS_ADDR: milvus:${MILVUS_PORT:-19530}
 # Mem0
 MEM0_ADDR: mem0:${MEM0_PORT:-8000}
 # Vault
 VAULT_ADDR: http://vault:${VAULT_PORT:-8200}
 VAULT_TOKEN: ${VAULT_DEV_ROOT_TOKEN:-dev-root-token}
 # LLM → 指向内置 Mock
 LLM_PROVIDER: ${LLM_PROVIDER:-openai}
 LLM_MODEL: ${LLM_MODEL:-mock-llm}
 LLM_BASE_URL: ${LLM_BASE_URL:-http://localhost:8080/mock/v1}
 LLM_API_KEY: ${LLM_API_KEY:-mock-key}
 LLM_MAX_TOKENS: ${LLM_MAX_TOKENS:-4096}

```

```

LLM_TEMPERATURE: ${LLM_TEMPERATURE:-0}

Embedding (仍需真实服务)
EMBEDDING_PROVIDER: ${EMBEDDING_PROVIDER:-openai}
EMBEDDING_MODEL: ${EMBEDDING_MODEL:-text-embedding-3-small}
EMBEDDING_DIM: ${EMBEDDING_DIM:-1536}

JWT
JWT_SECRET: ${JWT_SECRET:-test-secret-do-not-use-in-prod}
JWT_EXPIRE_HOURS: ${JWT_EXPIRE_HOURS:-999}

Workspace
WORKSPACE_ROOT: ${WORKSPACE_ROOT:-/workspaces}
WORKSPACE_MAX_SIZE_MB: ${WORKSPACE_MAX_SIZE_MB:-500}

Mock LLM
MOCK_CHUNK_DELAY_MS: ${MOCK_CHUNK_DELAY_MS:-0}
MOCK_DEFAULT_REPLY: ${MOCK_DEFAULT_REPLY:-Mock LLM: Inject test data via POST
/mock/responses}

SQL 测试数据源
TEST_DATASOURCE_NAME: ${TEST_DATASOURCE_NAME:-enterprise_demo}
TEST_DATASOURCE_TYPE: ${TEST_DATASOURCE_TYPE:-mysql}
TEST_DATASOURCE_DSN: ${TEST_DATASOURCE_DSN:-
demo_user:demo_password@tcp(mysql:3306)/enterprise_demo?charset=utf8mb4&parseTime=true}

volumes:
 # 挂载 workspace 目录到宿主机，便于测试时查看生成的文件
 - workspace_data:/workspaces

depends_on:
 mongodb:
 condition: service_healthy
 redis:
 condition: service_healthy
 seaweedfs:
 condition: service_healthy
 milvus:
 condition: service_healthy
 mysql:
 condition: service_healthy

restart: unless-stopped

healthcheck:
 test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
 interval: 15s
 timeout: 5s

```



```

 retries: 3

 start_period: 30s

=====
Next.js Frontend (含用户端 + 管理后台)
=====

frontend:

 build:

 context: ./web

 dockerfile: Dockerfile

 container_name: agent-frontend

 ports:

 - "${FRONTEND_PORT:-3000}:3000"

 environment:

 NEXT_PUBLIC_API_URL: http://localhost:${API_PORT:-8080}

 API_INTERNAL_URL: http://api:8080

 depends_on:

 api:

 condition: service_healthy

 restart: unless-stopped

=====
MongoDB
=====

mongodb:

 image: ${MONGO_IMAGE:-mongo:7.0}

 container_name: agent-mongodb

 ports:

 - "${MONGO_PORT:-27017}:27017"

 environment:

 MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USER:-admin}

 MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD:-changeme}

 volumes:

 - mongo_data:/data/db

 healthcheck:

 test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]

 interval: 10s

 timeout: 5s

 retries: 5

```

```

 start_period: 20s

 restart: unless-stopped

=====
Milvus (Standalone)
=====

milvus:

 image: ${MILVUS_IMAGE:-milvusdb/milvus:v2.5.0}

 container_name: agent-milvus

 ports:

 - "${MILVUS_PORT:-19530}:19530"

 - "${MILVUS_METRICS_PORT:-9091}:9091"

 environment:

 ETCD_USE_EMBED: "true"

 COMMON_STORAGETYPE: local

 volumes:

 - milvus_data:/var/lib/milvus

 healthcheck:

 test: ["CMD", "curl", "-f", "http://localhost:9091/healthz"]

 interval: 15s

 timeout: 10s

 retries: 5

 start_period: 40s

 restart: unless-stopped

=====
SeaweedFS (S3)
=====

seaweedfs:

 image: ${SEAWEEDFS_IMAGE:-chrislusf/seaweedfs:latest}

 container_name: agent-seaweedfs

 ports:

 - "${SEAWEEDFS_S3_PORT:-8333}:8333"

 command: server -s3 -s3.accessKey=${SEAWEEDFS_ACCESS_KEY:-seaweedfs-admin} -
s3.secretKey=${SEAWEEDFS_SECRET_KEY:-seaweedfs-admin}

 volumes:

 - seaweedfs_data:/data

 healthcheck:

 test: ["CMD", "curl", "-f", "http://localhost:8333/"]

```

```

 interval: 10s
 timeout: 5s
 retries: 5
 start_period: 10s
restart: unless-stopped

=====
Redis
=====

redis:
 image: ${REDIS_IMAGE:-redis:7.2-alpine}
 container_name: agent-redis
 ports:
 - "${REDIS_PORT:-6379}:6379"
 command: ${REDIS_PASSWORD:+redis-server --requirepass ${REDIS_PASSWORD}}
 volumes:
 - redis_data:/data
 healthcheck:
 test: ["CMD", "redis-cli", "ping"]
 interval: 10s
 timeout: 3s
 retries: 5
 restart: unless-stopped

=====
Vault (Dev Mode)
=====

vault:
 image: ${VAULT_IMAGE:-hashicorp/vault:1.14}
 container_name: agent-vault
 ports:
 - "${VAULT_PORT:-8200}:8200"
 cap_add:
 - IPC_LOCK
 environment:
 VAULT_DEV_ROOT_TOKEN_ID: ${VAULT_DEV_ROOT_TOKEN:-dev-root-token}
 VAULT_DEV_LISTEN_ADDRESS: 0.0.0.0:${VAULT_PORT:-8200}
 healthcheck:
 test: ["CMD", "vault", "status"]

```

```

 interval: 10s

 timeout: 5s

 retries: 5

 start_period: 10s
restart: unless-stopped

=====
Mem0
=====
mem0:
 image: ${MEM0_IMAGE:-mem0ai/mem0:latest}
 container_name: agent-mem0
 ports:
 - "${MEM0_PORT:-8000}:8000"
 restart: unless-stopped

=====
MySQL (模拟客户企业数据库)
=====
mysql:
 image: ${MYSQL_IMAGE:-mysql:8.0}
 container_name: agent-mysql
 ports:
 - "${MYSQL_PORT:-3306}:3306"
 environment:
 MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:-dev-root-password}
 MYSQL_DATABASE: ${MYSQL_DATABASE:-enterprise_demo}
 MYSQL_USER: ${MYSQL_USER:-demo_user}
 MYSQL_PASSWORD: ${MYSQL_PASSWORD:-demo_password}
 volumes:
 - mysql_data:/var/lib/mysql
 healthcheck:
 test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${MYSQL_ROOT_PASSWORD:-dev-root-password}"]
 interval: 10s
 timeout: 5s
 retries: 10
 start_period: 30s
 restart: unless-stopped

```

```

=====
Migration Service (初始化企业测试数据)
=====
migration:
 image: ${MYSQL_IMAGE:-mysql:8.0}
 container_name: agent-migration
 depends_on:
 mysql:
 condition: service_healthy
 entrypoint: ["bash", "-c"]
 command:
 - |
 echo "[Migration] Waiting for MySQL..."
 until mysqladmin ping -h mysql -u root -p${MYSQL_ROOT_PASSWORD:-dev-root-password}
--silent 2>/dev/null; do
 sleep 2
 done
 echo "[Migration] Executing seed SQL..."
 mysql -h mysql -u root -p${MYSQL_ROOT_PASSWORD:-dev-root-password}
${MYSQL_DATABASE:-enterprise_demo} < /seed/init_enterprise_data.sql
 echo "[Migration] Done. Enterprise demo data loaded."
 volumes:
 - ./testdata/migrations:/seed:ro
 restart: "no"

=====
Volumes
=====
volumes:
 mongo_data:
 milvus_data:
 seaweedfs_data:
 redis_data:
 mysql_data:
 workspace_data:

```

### 17.3.3 测试企业种子数据 (testdata/migrations/init\_enterprise\_data.sql)

```
-- =====
-- 模拟企业数据：电商平台运营数据库
-- 供 SQL Executor Skill 执行查询测试
-- =====

CREATE TABLE IF NOT EXISTS departments (
 id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(64) NOT NULL,
 parent_id INT DEFAULT NULL,
 FOREIGN KEY (parent_id) REFERENCES departments(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS employees (
 id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(32) NOT NULL,
 department_id INT NOT NULL,
 hire_date DATE NOT NULL,
 salary DECIMAL(12,2) NOT NULL DEFAULT 0,
 status ENUM('active','suspended','resigned') DEFAULT 'active',
 FOREIGN KEY (department_id) REFERENCES departments(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS products (
 id INT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(128) NOT NULL,
 category VARCHAR(32) NOT NULL,
 unit_price DECIMAL(10,2) NOT NULL,
 cost_price DECIMAL(10,2) NOT NULL,
 stock INT NOT NULL DEFAULT 0,
 created_at DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS orders (
 id INT PRIMARY KEY AUTO_INCREMENT,
 order_no VARCHAR(32) NOT NULL UNIQUE,
 customer_name VARCHAR(64) NOT NULL,
 product_id INT NOT NULL,
```

```

 quantity INT NOT NULL,

 unit_price DECIMAL(10,2) NOT NULL,

 total_amount DECIMAL(14,2) NOT NULL,

 status ENUM('pending','paid','shipped','completed','cancelled') DEFAULT 'pending',

 order_date DATE NOT NULL,

 FOREIGN KEY (product_id) REFERENCES products(id)

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```
INSERT INTO departments (id, name, parent_id) VALUES
```

```

(1, '总公司', NULL),
(2, '华东分公司', 1),
(3, '华南分公司', 1),
(4, '华北分公司', 1),
(5, '西南分公司', 1),
(6, '销售部', 2),
(7, '市场部', 2),
(8, '销售部', 3),
(9, '技术部', 3),
(10, '销售部', 4),
(11, '运营部', 4),
(12, '财务部', 1);

```

```
INSERT INTO employees (name, department_id, hire_date, salary, status) VALUES
```

```

('张伟', 6, '2019-03-15', 18000.00, 'active'),
('李娜', 6, '2020-07-01', 16000.00, 'active'),
('王强', 7, '2018-11-20', 22000.00, 'active'),
('赵敏', 8, '2021-01-10', 15500.00, 'active'),
('刘洋', 9, '2017-06-05', 25000.00, 'active'),
('陈静', 10, '2020-09-12', 19000.00, 'active'),
('周涛', 11, '2019-04-28', 21000.00, 'active'),
('孙丽', 12, '2016-08-01', 28000.00, 'active'),
('马超', 6, '2022-02-14', 14000.00, 'active'),
('黄芳', 8, '2021-05-20', 15000.00, 'active');

```

```
INSERT INTO products (id, name, category, unit_price, cost_price, stock) VALUES
```

```

(1, '智能手机 X1', '电子产品', 4999.00, 3500.00, 250),
(2, '笔记本电脑 Pro', '电子产品', 8999.00, 6200.00, 80),
(3, '运动跑鞋', '运动户外', 599.00, 320.00, 500),
(4, '办公椅 Ergo', '办公家具', 1299.00, 700.00, 120),

```

```

(5, '蓝牙耳机 ANC', '电子产品', 799.00, 450.00, 300),
(6, '护肤套装', '美妆个护', 399.00, 180.00, 400),
(7, '双肩背包', '箱包配饰', 299.00, 130.00, 600),
(8, '智能手表 S1', '电子产品', 2499.00, 1500.00, 180);

INSERT INTO orders (order_no, customer_name, product_id, quantity, unit_price,
total_amount, status, order_date) VALUES
('ORD202601001', '王大锤', 1, 2, 4999.00, 9998.00, 'completed', '2026-01-15'),
('ORD202601002', '李小萌', 3, 1, 599.00, 599.00, 'completed', '2026-01-16'),
('ORD202601003', '赵大伟', 2, 1, 8999.00, 8999.00, 'shipped', '2026-01-17'),
('ORD202601004', '陈美美', 6, 3, 399.00, 1197.00, 'completed', '2026-01-18'),
('ORD202601005', '刘铁柱', 1, 1, 4999.00, 4999.00, 'completed', '2026-01-19'),
('ORD202601006', '周星星', 5, 2, 799.00, 1598.00, 'shipped', '2026-01-20'),
('ORD202601007', '吴彦祖', 2, 1, 8999.00, 8999.00, 'pending', '2026-01-21'),
('ORD202601008', '章子怡', 7, 1, 299.00, 299.00, 'completed', '2026-01-22'),
('ORD202601009', '孙大圣', 8, 1, 2499.00, 2499.00, 'cancelled', '2026-01-23'),
('ORD202601010', '马小云', 4, 2, 1299.00, 2598.00, 'completed', '2026-01-24'),
('ORD202602001', '王大锤', 3, 2, 599.00, 1198.00, 'completed', '2026-02-01'),
('ORD202602002', '李小萌', 1, 1, 4999.00, 4999.00, 'completed', '2026-02-03'),
('ORD202602003', '赵大伟', 5, 1, 799.00, 799.00, 'shipped', '2026-02-05'),
('ORD202602004', '陈美美', 8, 1, 2499.00, 2499.00, 'completed', '2026-02-06'),
('ORD202602005', '刘铁柱', 2, 1, 8999.00, 8999.00, 'pending', '2026-02-07'),
('ORD202602006', '周星星', 4, 1, 1299.00, 1299.00, 'completed', '2026-02-08'),
('ORD202602007', '吴彦祖', 6, 2, 399.00, 798.00, 'completed', '2026-02-10'),
('ORD202602008', '章子怡', 3, 1, 599.00, 599.00, 'cancelled', '2026-02-12'),
('ORD202602009', '孙大圣', 7, 3, 299.00, 897.00, 'completed', '2026-02-14'),
('ORD202602010', '马小云', 1, 2, 4999.00, 9998.00, 'shipped', '2026-02-15'),
('ORD202603001', '王大锤', 5, 3, 799.00, 2397.00, 'completed', '2026-03-01'),
('ORD202603002', '李小萌', 2, 1, 8999.00, 8999.00, 'completed', '2026-03-05'),
('ORD202603003', '赵大伟', 8, 1, 2499.00, 2499.00, 'shipped', '2026-03-08'),
('ORD202603004', '陈美美', 4, 1, 1299.00, 1299.00, 'completed', '2026-03-10'),
('ORD202603005', '刘铁柱', 6, 2, 399.00, 798.00, 'completed', '2026-03-12');

```

-- 索引

```

CREATE INDEX idx_orders_order_date ON orders(order_date);
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_employees_department ON employees(department_id);

```



### 17.3.4 测试部署一键启动

```
1. 复制测试环境变量
cp .env.test .env

2. 编辑 .env (修改 EMBEDDING_API_KEY 指向真实服务)
vim .env

3. 启动测试环境 (MySQL + LLM Mock + 种子数据)
docker compose -f docker-compose.test.yml up -d

4. 等待 migration 完成
docker compose -f docker-compose.test.yml logs -f migration
预期输出: [Migration] Done. Enterprise demo data loaded.

5. 验证 Mock LLM 可用
curl -X POST http://localhost:8080/mock/v1/chat/completions \
 -H "Content-Type: application/json" \
 -H "Authorization: Bearer test" \
 -d '{"model":"mock","messages":[{"role":"user","content":"test"}],"stream":false}'
返回 Mock 默认响应

6. 注入测试 LLM 响应
curl -X POST http://localhost:8080/mock/responses \
 -H "Content-Type: application/json" \
 -H "Authorization: Bearer admin-token" \
 -d '{"match":"昨天华东区销售额多少? ","response":"华东区昨日销售额 85.3 万元, 环比增长 12%"}'

7. 访问前端
open http://localhost:3000
```

### 17.3.5 测试环境清理

```
停止所有测试服务并清理数据 (包括 workspace、MySQL 种子数据)
docker compose -f docker-compose.test.yml down -v
```

18. 风险与缓解方案

风险	影响	概率	缓解措施	应急方案
ADK 框架 API 变更频繁	Agent 核心逻辑需重构	中	抽象 AgentEngine 接口，降低框架耦合	回退到自研轻量 Agent Loop
AI 模型输出质量不稳定	分析结果错误率高	高	结果置信度标注 + 用户反馈闭环	提供结果编辑功能 + 重新分析
Milvus 内存占用高	部署成本超预算	中	MVP 使用小维度向量(384d) + 定期清理	降级到 Qdrant (更轻量)
大文件索引性能瓶颈	知识库索引进度慢	中	异步队列 + 分块处理 + 并行索引	限制单文件大小上限 (20MB)
安全审查规则覆盖不足	敏感数据泄露	中	多层防护 + 定期安全审计 + 渗透测试	增加人工审批步骤
并发 Session 资源泄漏	服务器性能下降	低	严格超时 + 资源监控告警 + 定期巡检	自动重启脚本 + 手动清理工具
Agent 任务死循环	资源耗尽	低	最大步数限制(50) + 超时强制终止(10min)	全局熔断器自动切断

19. 分阶段开发任务分解

16.1 Phase 1: 基础框架 (Week 1–2)

目标: 搭建项目骨架、基础设施连接层和核心中间件。

Week 1: 项目初始化与基础设施

ID	任务	工时	依赖	关键决策
P1-01	Go 项目初始化 (Module、目录结构、Makefile)	2h	-	Go 1.22+, RFC Section 3.2 目录结构
P1-02	Docker Compose 开发环境 (MongoDB/Milvus/SeaweedFS/Redis)	4h	-	参考 RFC Section 13.2
P1-03	配置管理系统 (Viper/YAML + 环境变量覆盖)	3h	P1-01	开发用 YAML, 生产用环境变量
P1-04	统一日志系统 (slog 结构化日志)	2h	P1-01	Go 1.21+ 原生 slog
P1-05	MongoDB 连接层 (Repository Pattern 封装)	4h	P1-02	按 RFC Section 6.1 集合设计
P1-06	Redis 连接层 (缓存 + Stream 操作)	3h	P1-02	参考 RFC Section 12.1
P1-07	SeaweedFS 连接层 (Bucket CRUD + 文件操作)	3h	P1-02	参考 RFC Section 9
P1-08	Milvus 连接层 (Collection 管理 + 向量搜索)	4h	P1-02	参考 RFC Section 10

Week 2: 中间件与认证

ID	任务	工时	依赖	关键决策
P1-09	JWT 认证中间件（签发、验证、刷新）	4h	P1-04	<code>golang-jwt/jwt</code> ，参考 RFC Section 8.2
P1-10	RBAC 引擎（角色定义、权限校验接口）	6h	P1-05, P1-09	参考 RFC Section 4.1 数据模型
P1-11	HTTP 路由注册（Gin Router + 分组）	2h	-	按 RFC Section 7.1 路由结构
P1-12	审计日志中间件（请求级别记录）	3h	P1-05, P1-09	参考 RFC Section 11
P1-13	安全过滤中间件框架（规则加载 + 过滤注册）	4h	P1-09	参考 RFC Section 8.1 YAML 规则配置
P1-14	错误码体系统一（errcode 包 + HTTP 映射）	2h	-	-
P1-15	CORS / Rate Limit 中间件	1h	-	-
P1-16	健康检查 API（ <code>GET /health</code> ）	1h	P1-05	检查所有基础设施连接

技术决策点:

决策	方案	备注
HTTP Framework	Gin	性能优先
配置格式	YAML + 环境变量覆盖	开发本地 YAML，生产环境变量
日志库	slog (Go 1.21+)	原生支持，零依赖

16.2 Phase 2: 核心服务 (Week 3–4)

目标: Agent 引擎集成、Agent Service 实现（含 Chat 模式 + Agent 批量模式 + Worker Pool）。

Week 3: Agent Engine + Skill Registry

ID	任务	工时	依赖
P2-01	ADK Agent Engine 初始化与配置	4h	P1-16
P2-02	LLM Router 实现（多模型切换、参数配置）	4h	P2-01
P2-03	Skill 接口定义与注册中心（参考 RFC Section 5.3）	4h	P2-01
P2-04	Skill 自动加载器（扫描 skills/ 目录）	3h	P2-03
P2-05	SQL Executor Skill（MCP 对接数据库）	8h	P2-03, P1-05
P2-06	Stats Engine Skill（回归 + 时间序列, gonum）	10h	P2-03
P2-07	分析结果落库模型与 Repository（参考 RFC Section 6.2）	3h	P1-05
P2-08	Artifact 存储模块（SeaweedFS volume + save_artifact Skill, 参考 RFC Section 10）	5h	P1-07, P2-03

Week 4: Agent Service (Chat 模式 + Agent 模式 + Worker Pool)

ID	任务	工时	依赖
P2-09	Session Manager (创建、续期、过期清理, 参考 RFC Section 9)	6h	P1-07
P2-10	Agent Service — Chat 模式 (SSE 流式返回, 参考 RFC §4.2 + §7.2)	6h	P2-01, P2-08
P2-11	快捷提示词 CRUD + 按角色展示	3h	P1-05
P2-12	Agent Task Queue (Redis Stream, 参考 RFC Section 12.1)	4h	P1-06
P2-13	Agent Worker Pool (并发消费, 参考 RFC Section 12.1 Worker)	6h	P2-11
P2-14	任务取消机制 (Context Cancel, 参考 RFC Section 4.3)	3h	P2-12
P2-15	任务进度上报 (WebSocket Push)	4h	P2-12
P2-16	任务完成通知 (Email + In-app)	3h	P2-14

16.3 Phase 3: 知识库与解读 (Week 5-6)

Week 5: 知识库文档处理

ID	任务	工时	依赖
P3-01	文档解析引擎 (PDF/Word/Excel/MD/TXT)	6h	–
P3-02	文档分块策略 (固定 500 字符 + 语义切分, 参考 RFC Section 10.1)	4h	P3-01
P3-03	Embedding Model 对接 (向量生成)	3h	P1-08
P3-04	Milvus Collection 创建与写入 (参考 RFC Section 10.1)	4h	P3-03, P1-08
P3-05	异步索引队列 (Redis Stream → Worker, 参考 RFC Section 10.1)	4h	P1-06
P3-06	MongoDB 全文搜索索引建立	2h	P1-05

Week 6: 搜索 + 解读 + 聚合

ID	任务	工时	依赖
P3-07	Milvus 向量相似度搜索	3h	P3-04
P3-08	混合搜索 (Semantic + Fulltext + 重排序, 参考 RFC Section 10.2)	5h	P3-07, P3-06
P3-09	权限过滤 (搜索结果按用户权限裁剪)	3h	P3-08, P1-10
P3-10	分析结果智能解读 (KB 检索 + 上下文注入)	6h	P3-08
P3-11	多维度聚合层数据定义与管理 (参考 RFC Section 6.3)	5h	P1-05
P3-12	Email Sender Skill (域名白名单)	3h	P2-03
P3-13	知识库 Web API (上传/删除/搜索)	4h	P3-08

16.4 Phase 4: 高级功能 (Week 7-8)

Week 7: 定时任务 + 子 Agent

ID	任务	工时	依赖
P4-01	Scheduler Service (robfig/cron 集成, 参考 RFC Section 4.4)	4h	P2-12
P4-02	定时任务 CRUD + 暂停/恢复	3h	P4-01
P4-03	定时任务执行 → 入队 Redis Stream → Worker 消费 → Agent Service	3h	P4-01
P4-04	失败重试逻辑 + 超时终止	2h	P4-03
P4-05	Sub-Agent 编排基础 (A2A, 参考 RFC Section 5.1 + 9.2)	6h	P2-01
P4-06	子 Agent 会话生命周期管理	4h	P2-08, P4-05

Week 8: 高级统计 + 安全 + OpenAPI

ID	任务	工 时	依赖
P4-07	聚类分析 Skill (K-Means, gonum)	6h	P2-06
P4-08	主成分分析 Skill (PCA, gonum)	4h	P2-06
P4-09	财务分析 Skill (比率计算 + 趋势对比)	6h	P2-06
P4-10	Security Audit Layer 完整实现 (参考 RFC Section 8.1)	6h	P1-13
P4-11	熔断器 Circuit Breaker (参考 RFC Section 8.1)	3h	P4-10
P4-12	OpenAPI → MCP 转换器 (参考 RFC Section 5.5)	8h	P2-03
P4-13	审计日志自动清理 (TTL Index, 参考 RFC Section 11.3)	2h	P1-12
P4-14	用户/Agent 审计日志完善 (IP、Skill 调用记录, 参考 RFC Section 11.1-11.2)	4h	P1-12
P4-15	报告格式校验模块 Report Validator (规则引擎 + Agent 修正循环, 参考 RFC Section 4.6)	6h	P2-09, P1-05



16.5 Phase 5: 管理后台 (Week 9–10)

Week 9: 后台前端搭建 + 核心页面

ID	任务	工时	依赖
P5-01	Next.js 项目初始化 + UI 框架 (Ant Design / Shadcn)	4h	–
P5-02	Admin Layout + 菜单路由	3h	P5-01
P5-03	登录页 + JWT Token 管理	3h	P5-01
P5-04	Dashboard 可视化看板 (ECharts / Recharts)	6h	P5-02
P5-05	用户管理页面 (CRUD + 角色分配 + 启停)	5h	P5-02
P5-06	权限管理页面 (角色 CRUD + 权限映射)	4h	P5-02

Week 10: 功能页面完善

ID	任务	工时	依赖
P5-07	模型配置页面 (LLM 选择 + 参数调整)	3h	P5-02
P5-08	任务管理页面 (列表 + 进度 + 取消)	4h	P5-02
P5-09	知识库管理页 (上传 + 列表 + 搜索 + 状态追踪)	5h	P5-02
P5-10	审计日志查看页 (筛选 + 导出)	4h	P5-02
P5-11	API 转换审核页面	3h	P5-02
P5-12	响应式适配 (移动浏览器兼容)	3h	P5-02
P5-13	前后端联调 + Bug 修复	8h	All above

16.6 Phase 6: 测试与优化 (Week 11–12)

Week 11: 测试

ID	任务	工时	依赖
P6-01	单元测试补充（目标覆盖率 > 70%）	10h	All
P6-02	集成测试（Service 层 + 数据库交互）	8h	P6-01
P6-03	E2E 测试（Playwright: 登录→Chat→Agent→结果）	8h	P6-02
P6-04	golangci-lint 全量检查 + 修复	4h	All
P6-05	依赖安全扫描（govulncheck）	2h	All

Week 12: 优化 + 发布

ID	任务	工时	依赖
P6-06	性能压测 + 瓶颈优化	8h	P6-03
P6-07	安全渗透测试（OWASP Top 10）	4h	P6-05
P6-08	Docker 镜像构建 + K8s 部署配置（参考 RFC Section 13）	4h	–
P6-09	内部 Alpha 发布 + 使用培训	4h	P6-08
P6-10	文档整理（API 文档 + 部署手册 + 用户指南）	6h	All

16.7 工时汇总

Phase	总预估工时	说明
Phase 1	~45h	基础设施和中间件
Phase 2	~73h	Agent Engine + Chat + Agent Service + Artifact
Phase 3	~55h	知识库文档处理 + 搜索 + 解读
Phase 4	~61h	调度器 + 子Agent + 高级 Skill + 安全 + 报告校验
Phase 5	~52h	管理后台前端
Phase 6	~56h	测试 + 优化 + 发布
总计	~342h	3 人团队约 12 周

开发路线图

20. AI 辅助开发流程规范

17.1 工作模式

采用 AI 生成代码 + 人工 Review 的混合开发模式：



Step 1 — Spec 定义: 工程师编写详细技术规格（功能需求、API 定义、数据模型、核心算法/逻辑描述）

Step 2 — 代码生成: AI 根据 Spec 生成 Go 代码 + 单元测试 + 接口文档注释

Step 3 — 人工 Review: 必须检查以下方面： -  代码逻辑正确性 -  安全审查（SQL 注入、越权、敏感数据） ← 重点 -  性能审查（N+1 查询、内存泄漏、goroutine 泄漏） -  错误处理完整性 -  代码风格一致性

Step 4 — CI/CD 自动化: golangci-lint → go test ./... → govulncheck → Build

Step 5 — PR Merge: 至少一人 Code Review + 通过 CI 检查 → Merge to Main

## 17.2 AI 生成规范模板

每次让 AI 生成代码时，必须提供以下上下文：

```
生成要求

Context
- 项目使用 Go 1.22+, Gin framework
- MongoDB driver: go.mongodb.org/mongo-driver/v2
- 代码风格遵循 golangci-lint 默认规则
- 错误处理使用自定义 errcode 包

Input Spec
[粘贴详细的技术规格]

Output Requirements
1. 生成 .go 源文件 (包含 package import path)
2. 生成对应的 _test.go 文件 (table-driven tests)
3. 关键函数添加 Godoc 注释
4. 错误处理必须显式返回，不允许 panic (除非不可恢复)
5. SQL 相关操作必须使用参数化查询
6. 敏感信息不得硬编码

Constraints
- 不修改已有文件 (只新增)
- 遵循现有目录结构 (RFC Section 3.2)
- 接口签名与 spec 保持一致
```

17.3 AI 生成 vs 人工编写比例

类别	AI 生成比例	人工介入程度	说明
项目脚手架	95%	低	模板化代码
CRUD 接口	80%	低-中	标准增删改查
中间件	70%	中	安全/审计相关需审查
Agent 逻辑	50%	高	核心 Prompt + 工具编排
Skill 实现	75%	中	取决于复杂度
安全相关	30%	极高	必须人工逐行审查
测试代码	90%	低	用例由人定，代码 AI 写
前端 UI	85%	低-中	页面布局 + 交互

21. 团队分工建议

最小团队配置

角色	人数	主要职责
后端工程师	2	核心服务开发 + AI 代码 Review
全栈/前端	1	管理后台前端 + API 联调
AI/架构师	1（兼职）	架构决策 + Agent Prompt 设计 + 技术评审

按阶段分配

Phase 1 (W1-2): 2 Backend + 1 Fullstack

- 后端：基础设施 + 中间件
- 前端：UI 框架预研

Phase 2 (W3-4): 2 Backend + 1 Fullstack

- 后端：Agent Engine + Chat/Agent Service
- 前端：Chat 页面原型

- Phase 3 (W5-6): 2 Backend
  - 后端: 知识库 + 解读 + 聚合 (专注后端)
- Phase 4 (W7-8): 2 Backend
  - 后端: 高级功能 + 安全完善
- Phase 5 (W9-10): 1 Backend + 1 Fullstack
  - 前端: 管理后台全面开发
  - 后端: API 支持 + Bug Fix
- Phase 6 (W11-12): 2 Backend + 1 Fullstack
  - 全员: 测试 + 调优 + 发布

## 22. 附录

### 19.1 开源 Skill 推荐

Skill	用途	Repository	ADK 兼容性
sql-executor	SQL 生成与执行	自研 (基于 database/sql)	✓
stats-engine	统计分析 (gonum)	自研	✓
knowledge-search	知识库搜索	自研 (Milvus SDK)	✓
email-sender	邮件发送	自研 (go-mail)	✓
openapi-mcp	API 转 MCP	社区 + 自研	⚠ 需适配
audit-logger	审计日志	go-audit/audit-go	✓
pdf-reader	PDF 解析	ledongthuc/pdf	✓
security-scanner	安全扫描	自研 (regex + keyword)	✓

## 19.2 环境变量配置

```
Server

SERVER_PORT=8080

SERVER_MODE=debug # debug | release

MongoDB (credentials managed by Vault in production)

MONGO_URI=mongodb://localhost:27017

MONGO_DB=data_agent

Milvus

MILVUS_HOST=localhost

MILVUS_PORT=19530

SeaweedFS (credentials managed by Vault in production)

SEAWEEEDFS_ENDPOINT=localhost:8333

SEAWEEEDFS_ACCESS_KEY=seaweedfs-admin

SEAWEEEDFS_SECRET_KEY=seaweedfs-admin

SEAWEEEDFS_WORKSPACE_VOLUME=workspaces

SEAWEEEDFS_ARTIFACT_VOLUME=artifacts

Redis

REDIS_ADDR=localhost:6379

REDIS_PASSWORD=

REDIS_DB=0

Mem0

MEM0_API_KEY=your_key

MEM0_ENDPOINT=http://localhost:8080

Vault

VAULT_ADDR=http://localhost:8200

VAULT_TOKEN=dev-root-token

VAULT_MOUNT_PATH=secret/data-agent # Vault KV v2 mount path

JWT (secret managed by Vault in production; dev fallback below)

JWT_SECRET=dev-secret-key

JWT_EXPIRY=24h
```

```
LLM (API key managed by Vault in production; dev fallback below)

LLM_PROVIDER=openai
LLM_MODEL=gpt-4o
LLM_API_KEY=your_key
LLM_BASE_URL=https://api.openai.com/v1

Email (password managed by Vault in production)

SMTP_HOST=smtp.company.com
SMTP_PORT=587
SMTP_USER=noreply@company.com
SMTP_PASSWORD=your_password
EMAIL_DOMAIN_WHITELIST=company.com,partner.com

Security

SECURITY_INPUT_FILTER_ENABLED=true
SECURITY_OUTPUT_FILTER_ENABLED=true
AUDIT_RETENTION_DAYS=90
```